压缩与验证

复杂软件系统的层级涌现、结构收敛与可验证执行

Compression and Verification:
Layered Emergence, Structural Convergence,
and Verifiable Execution in Complex Software Systems

压缩—保证工作笔记

独立技术报告

https://github.com/ghwinter/assay

通讯请通过仓库的 issue 追踪。
本报告源文件：https://github.com/ghwinter/assay/tree/master/docs/paper

2026 年

摘要

本文研究一个问题的形式化：复杂软件系统在高层的抽象、结构与架构上，是否也像底层 *ABI*、调用约定与栈帧那样收敛？该直觉被拆分为三个强度不同的命题，各给出可判定或可证伪的形式。

四条论题组织全文。**压缩论题**：复杂系统可被有限智能理解，当且仅当它可压缩；理解即任务约束下的有损压缩，其最优下潜深度是率失真拉格朗日量的驻点条件——边际预测增益为零处。**必然性论题**：任何具有并发存活实体与受争用有限资源的系统，若操作语义良定义，则必然已定义六项职能——身份与生命周期、分配与回收、调度、命名与寻址、通信与同步、故障隔离与恢复；六项可自建，可外借给宿主，不可在语义上缺失——而这条主张是一项清单义务，不是" 六项最小" 或" 六项彼此独立" 的主张。**分离论题**：生成可自由、概率、不可信；准入必须确定、机械、可穷尽检视。形式层（规格 $\to$ 实现 $\to$ 二进制 $\to$ 执行）的保证来自极小可信内核的机械判定，意图层（意图 $\to$ 规格）的保证来自对抗性共识，两层由制品绑定缝合。**设计论题**：架构是人为制造的尺度分离——自然界的层级由重整化免费获得，软件的层级必须被工程付费制造；设计是正向压缩，理解是逆向压缩，两者是同一结构的两个读取方向。

形式化结果分三类，全文逐条标注，而这一区分本身是贡献的一部分。作为工作定义：以抽象映射的核刻画层级的无视集，把" 该层可以不关心什么" 变成可被声明、随后可被检验的对象——是声明而非抽取，因为该定义不赋予可计算性，而针对它的检验都是部分的；以契约操作元的侧条件精确定位组合失效之处，其中包含着色与无环性都看不见的资源耦合替换；以读集期望给出架构的可测目标函数；以" 接口是行为的充分统计量" 给出近可分解性判据与泄漏对的差分诊断。作为条件性结果：在观测相容与假设封闭下给出层间可靠性的复合，其中关系情形带有显式的复合前提；把不变量集中化陈述为读集的一个下界，且相对于没有独立证据可结算的那些强制点；在写明的成本模型下给出内部基础设施化的盈亏平衡阈值；以层族区分" 六职能的跨领域同构" 与" 沿问题形状的传输"；提出粗粒化判据，使收敛论题可在单一项目内被证伪；以描述长度差定义技术债；以观测判据回答理解能否收敛，区分观测不足与样本不足；以可治理复杂度 $C = \mathbb{E}[\rho] \cdot \lambda$ 对理解容量 $K = hb$ 给出稳定条件，以及该模型内所需人力的标度律；以 Hoeffding 界给出共识可靠性的上界，证明相关错误模式 q 支配该界，并把 q 归约为一个激励设计问题；以综合保证条件式给出充分条件而非必要条件——因为它的各个合取项是关于证据的条件，而一个系统可以在没有这些证据时依然正确。作为可证伪假设：收敛主张、六职能清单、把理解读作压缩，以及第九章的成本与标度轨迹。

全文的组织性问题不是" 单个组件能否被证明正确"，而是保证能否走完从需求到执行制品的全程。一组已验证组件的复合仍会失效：若规格漏掉一条需求、若各组件的外部假设互不相容、若资源代价未被证明、若规格从未由有资格的主体审定、或若部署运行的是与已验证制

品不同的制品。以上每一种都是不同的失效，各有不同的补救，且没有一种能通过” 把更多组件证明正确” 来修复。

全文给出十四条可证伪假设及其操作化、对照组、证伪条件与风险，一条六阶段实施路线，一张以论文为中心的跨学科文献地图，以及一份不作主张清单。该清单拒绝把 Gödel 不完备定理、Rice 定理、P 与 NP 的未决问题、近可分解性，以及与 Langlands 纲领或 Landauer 原理的类比，当作已证明的普遍定律来承重。

关键词：软件架构；结构收敛；抽象层级；信息压缩；最小描述长度；形式化验证；证明携带代码；语义保持；可信计算基；共识；意图鸿沟

Abstract

We formalise one question. *Do large software systems converge in their high-level abstractions, structures and architectures, in the way that low-level ABIs, calling conventions and stack frames converge?* The intuition is decomposed into three propositions of differing strength, each given a decidable or falsifiable form.

Four theses organise the work. The **compression thesis**: a complex system is comprehensible to a bounded intelligence exactly to the extent that it is compressible; comprehension is task-constrained lossy compression whose optimal descent depth is the stationarity condition of a rate–distortion Lagrangian. The **necessity thesis**: any system with concurrently live entities and contended finite resources, whose operational semantics is well defined, has already defined six functions—identity and lifecycle, allocation and reclamation, scheduling, naming and addressing, communication and synchronisation, and fault isolation and recovery; these may be built or adopted, but not omitted—and the claim is a checklist obligation, not a claim that the six are minimal or mutually independent. The **separation thesis**: generation may be free, probabilistic and untrusted, while admission must be deterministic, mechanical and exhaustively inspectable; formal-layer guarantees come from a minimal trusted kernel, intent-layer guarantees from adversarial consensus, and the two are stitched by artefact binding. The **design thesis**: architecture is manufactured separation of scales—nature obtains levels free through renormalisation, software must pay for them—and design is forward compression while comprehension is inverse compression, two reading directions of one structure.

The formal results are of three different kinds, labelled as such throughout, and the distinction is part of the contribution. As *working definitions*: a level's *oblivious set* as the kernel of its abstraction map, which turns "what this level may ignore" into an object that can be declared and then checked—declared rather than extracted, because the definition confers no computability and the checks against it are partial; a *contract operad* whose side conditions locate exactly where composition fails, including the case of resource-coupled substitution that neither colouring nor acyclicity detects; expected read-set as a measurable architectural objective; and near-decomposability as interface sufficiency, yielding a differential diagnostic for *leak pairs*. As *conditional results*: composition of inter-level soundness under observation compatibility and assumption closure, with the relational case carrying an explicit composition premise; centralisation of invariants as a bound on the read-set, stated relative to those enforcement sites that no independent evidence settles; a break-even threshold for in-

ternal infrastructuralisation under a stated cost model; a *level family* separating cross-domain isomorphism of the six functions from transport along problem shape; a *coarse-graining criterion* making the convergence thesis falsifiable inside a single project; technical debt as a description-length gap; an *observability* criterion for whether comprehension can converge at all, separating insufficient observation from insufficient sampling; *governable complexity* $C = \mathbb{E}[\rho] \cdot \lambda$ against comprehension capacity $K = hb$, giving a stability condition and a scaling law for required headcount within that model; a Hoeffding bound for consensus reliability, dominated by a correlated error mode q that is in turn reduced to an incentive-design problem; and a consolidated assurance condition stated as sufficient rather than necessary, because its conjuncts are conditions on evidence and a system may be correct without them. As *falsifiable hypotheses*: the convergence claims, the six-function checklist, the reading of comprehension as compression, and the cost and scaling trajectories of Chapter 九.

The organising question is not whether individual components can be proved correct, but *whether guarantees survive the journey* from requirements to executed artefact. A composition of verified components still fails if the specification omits a requirement, if the components' environmental assumptions are mutually incompatible, if resource costs are unproved, if the specification was never authorised by anyone with standing to adopt it, or if the deployment runs a different artefact from the one verified. Each of those is a distinct failure with a distinct remedy, and none of them is repaired by proving more components correct.

Fourteen falsifiable hypotheses with operationalisations, controls, falsifiers and risks are given, together with a six-phase roadmap, a paper-centred interdisciplinary literature map, and an explicit list of non-claims rejecting the use of Gödel incompleteness, Rice's theorem, the open status of P versus NP, near-decomposability, and analogies to the Langlands programme or Landauer's principle, as load-bearing universal laws.

Keywords: software architecture; structural convergence; abstraction levels; information compression; minimum description length; formal verification; proof-carrying code; semantic preservation; trusted computing base; consensus; intent gap

目录

摘要 **i**

Abstract **iii**

第一章 导论 **1**

1.1 问题的由来 1
1.2 为什么这些问题尚未被统一处理 1
1.3 核心论题 2
1.4 认识论纪律 3
1.5 主要贡献 3
1.6 全文结构 4
1.7 术语约定 5

第二章 形式框架 **6**

2.1 层级与抽象映射 6
2.2 无视集与压缩率 8
2.3 架构关系型 11
2.4 理解作为任务约束的有损压缩 11
2.5 契约、轨迹与保证 13
2.6 信任的两种生成模式 14
2.7 契约代数与组合 14
2.8 记号汇总 16

第三章 结构收敛 **18**

3.1 三个强度不同的收敛命题 18
3.2 约束系统与解集几何 18
3.3 收敛机制 20
3.4 收敛强度与领域绑定度 21
3.5 关系而非组件 21
3.6 验证设计 22
3.7 反例与边界 23

第四章 内部基础设施化 **24**
4.1 现象与术语 . . . 24
4.2 参与者-资源结构 . . . 25
4.3 集成成本定理与盈亏平衡阈值 . . . 25
4.4 内核六职能 . . . 27
4.5 自建与外借 . . . 28
4.6 跨领域对照 . . . 29
4.7 传输与重新导出 . . . 30
4.8 粗粒化判据 . . . 31
4.9 虚拟机化与深模块 . . . 32
4.10 智能体系统的职能分化 . . . 32

第五章 理解：逆向压缩 **34**
5.1 顺序阅读的失效 . . . 34
5.2 认知图 . . . 34
5.3 理解的操作判据 . . . 35
5.4 阅读顺序 . . . 35
5.5 模块边界的探测判据 . . . 36
5.6 主动辨识与被动阅读 . . . 37
5.7 可观测性判据 . . . 38
5.8 预算与停止条件 . . . 39
5.9 机器理解的推论 . . . 40

第六章 设计：正向压缩 **42**
6.1 读集与目标函数 . . . 42
6.2 近可分解性 . . . 44
6.3 技术债 . . . 45
6.4 可治理复杂度与理解容量 . . . 45
6.5 设计流程 . . . 47
6.6 横切关注点与分层的极限 . . . 47
6.7 设计检查项 . . . 48

第七章 从意图到可验证执行 **50**
7.1 正确性的维度 . . . 50
7.2 可验证细化链 . . . 50
7.3 转换边与证据 . . . 51
7.4 组合条件与失效模式 . . . 52
7.5 意图鸿沟 . . . 53
7.6 验证与生成的不对称 . . . 55

7.7 证据等级 . . . 56
7.8 副作用与约束机制 . . . 56
7.9 运行时保证 . . . 58
7.10 制品绑定 . . . 58
7.11 共识层的可靠性界 . . . 59
7.12 意图层的激励结构 . . . 61
7.13 谁可以审定：道义轴 . . . 61
7.14 规格绑定 . . . 62
7.15 综合保证条件 . . . 63

第八章 层级与验证义务 66
8.1 认知尺度 . . . 66
8.2 层契约 . . . 67
8.3 验证义务的分解 . . . 67
8.4 责任主体 . . . 68
8.5 层级语言的错配 . . . 69

第九章 可证伪的研究纲领 70
9.1 方法论：从解释框架到可检验假设 . . . 70
9.2 十四条可证伪假设 . . . 70
9.3 三个优先研究方向 . . . 73
9.4 六阶段实施路线 . . . 75
9.5 成功判据 . . . 76
9.6 最小可行子集与采纳风险 . . . 76

第十章 文献地图与定位 77
10.1 六组文献 . . . 77
10.2 按数学功能重排的五层 . . . 78
10.3 空缺的交叉点 . . . 78
10.4 与既有理论的关系 . . . 79

第十一章 反对意见与限度 81
11.1 不作主张清单 . . . 81
11.2 被移出承重结构的论证 . . . 82
11.3 可能的反对意见与回应 . . . 83
11.4 尚未解决的问题 . . . 84
11.5 方法论局限 . . . 85

第十二章　结论 **86**

12.1 四个问题的回答 86

12.2 压缩作为主线 86

12.3 核心命题 87

12.4 结语 87

附录 A　补充证明与论证 **88**

A.1 命题 3.5 的说明 88

A.2 定理 4.4 的完整论证 88

A.3 定理 4.9 的逐项论证补充 89

A.4 定理 6.3 的读集下界 89

A.5 命题 7.12 的三条根据是否穷尽 89

A.6 定理 7.29 的界是否紧 90

A.7 命题 7.30 的一个更强的形式 90

A.8 命题 5.8 的说明 90

附录 B　模板 **91**

B.1 架构地图模板 91

B.2 模块契约模板 91

B.3 决策记录模板 92

附录 C　术语表 **93**

表格

2.1 两种信任生成模式的对照 14
2.2 全文记号 17

4.1 同一组管理职责在不同领域的独立实现 30
4.2 智能体系统对六职能的重新发现 33

5.1 子系统认知记录的最小字段集 40

6.1 大型系统设计检查表 48

7.1 五个正确性维度及其可机械检查性 50
7.2 五条转换边、典型证据与关键缺口 52
7.3 证据等级 56
7.4 制品证据清单 57
7.5 副作用分类与相应的机械机制 57

8.1 层契约四元组在一个关系数据库引擎上的实例化 67
8.2 保证类型在各层级上的分布（示例） 68

9.1 假设与实验设计汇总 73
9.2 研究实施路线 75

10.1 按数学功能重排的五个层次 79
10.2 本文与既有理论的关系 79

11.1 被移出承重结构的论证 82

B.1 架构地图 91

第一章　导论

1.1　问题的由来

以下三个问题指向同一结构。它们最初以工程直觉的形式提出；本节给出可讨论、可证明、可证伪的形式。

问题 1.1 (底层确定性的可推广性)**.** 软件底层的运行过程具有高度的一致性：ABI 与调用约定、函数调用与栈帧、堆与对象生命周期、控制流与数据流、同步原语与线程调度。这些机制在不同软件之间几乎相同，其一致性来自共同的计算模型、硬件架构、语言运行时与操作系统接口。问题是：这种确定性是否可以推广到高层？大部分正常的、偏置不多的大型复杂软件系统，是否也会在抽象、结构与架构上收敛到相同或相似的形态？

问题 1.2 (规模的强制性)**.** 当软件代码量或协作复杂度跨过某一量级，系统是否会必然开始针对同类问题进行设计与优化，从而在内部长出一套统一的基础设施？Linux 内核会定义内存如何管理、如何分配、进程与线程如何定义与调度，并为上层提供运行时接口；浏览器、游戏引擎、异步运行时、分布式存储与关系数据库引擎也会考虑同一类问题，只是所处的层次各不相同。代码量一旦上去，这些系统是否都会在代码中体现出" 将功能统一、接口统一、管理规划、运行时结构" 这类思考？

问题 1.3 (智能的理解与设计)**.** 软件是人类与 AI 编写的，但软件本身并不容易理解。大型复杂软件几乎不可能从微观开始顺序阅读理解。那么应当从什么抽象层级理解？如何由理解进入开发，而不陷入代码海洋？智能不能陷入无穷的底层信息量，而须保留对底层基本原理的理解（记为 L_0）与上层的 L_1、L_2、L_3……，即被压缩过的信息。反过来，究竟应当如何设计一个大型复杂软件系统，使目的得以达成？

第三个问题有一个尖锐的当代变体，它把前两个问题从" 如何理解" 推向" 如何保证"：

问题 1.4 (无人审阅条件下的正确性)**.** 如果 AI 使编码成本趋近于零，人不再逐行审阅代码，那么如何机械地、非自指地保证正确性？即：如何保证人提出的初始目标被实现，且实现过程中没有额外的、未被授权的副作用？一个具体的例子是：原本手写一次调用耗时 1 ns，AI 生成的实现耗时 1 μs，功能完全一致，目的达成，却产生了性能上的副作用。

1.2　为什么这些问题尚未被统一处理

上述问题中的每一个都在既有文献中有局部对应：架构收敛对应模式语言与复杂系统理论，规模的强制性对应信息隐藏、近可分解性与模块化理论，理解与设计对应认知模型与多视

图建模，正确性对应程序逻辑、精化演算与验证编译。但把它们作为一个整体处理时，会出现三处结构性缺口。

缺口一：收敛命题的强度未被区分。 ”大型软件架构会收敛”这句话至少包含三个强度差异极大的命题：局部架构模式的反复出现（有大量工程经验支持）；在适当抽象下呈现高于随机基线的可测量结构相似性（很可能成立，值得系统验证）；几乎所有正常运行的大型软件最终收敛到同一种高层架构（证据不足，且可能过强）。多数讨论把三者混为一谈，因而无法被检验。

缺口二：必然性主张缺少承重结构。 ”规模强制产生统一基础设施”是一个必然性主张，但通常以经验观察的形式陈述，因而无法回答：必然性从何而来？阈值由什么决定？为什么有些同样巨大的系统没有长出内核？本文的答案是：必然性来自操作语义的完备性，阈值来自摊销不等式，而”没有长出内核”的系统是把内核外借给了宿主。

缺口三：理解、设计与保证是同一结构的三次出现。 理解被当作认知问题，设计被当作工程问题，保证被当作逻辑问题，三者分别在不同的文献传统中处理。三者是同一个对象的不同读取方向：层级就是保证义务的分解单元，理解是恢复这套分解，设计是规定这套分解，而保证是在这套分解上逐层施加约束。

1.3　核心论题

本文以四条论题组织全部内容。每条论题后面括号内标注了它在本文中的形式化位置与强度等级：[**P**] 表示有完整证明，[**A**] 表示有严格论证但依赖明确假设，[**H**] 表示作为可证伪假设提出。

原则 1.5 (压缩论题)**.** [**A**] 一个复杂系统可被有限智能理解，当且仅当它是可压缩的。理解被定义为任务约束下的有损压缩：在保持任务相关预测能力的约束下最小化描述长度。由此，下潜的最优深度是率失真拉格朗日量的驻点条件，即边际预测增益为零处。

原则 1.6 (必然性论题)**.** [**A**] 任何具有并发存活实体与受争用有限资源的系统，若其操作语义良定义，则必然已定义六项职能：身份与生命周期、分配与回收、调度、命名与寻址、通信与同步、故障隔离与恢复。这六项职能可以被自建，也可以被外借给操作系统、语言运行时或框架，但不可以在语义上缺失，缺失即意味着存在未定义行为。

原则 1.7 (分离论题)**.** [**A**] 生成可以自由、概率、开源、不可信；准入必须确定、机械、可穷尽检视。形式层（规格 $\to$ 实现 $\to$ 二进制 $\to$ 执行）的保证来自极小可信内核的机械判定；意图层（意图 $\to$ 规格）的保证来自对抗性共识，且这一层不存在机械解。两层由制品绑定缝合：只有被验证过的那个二进制被允许运行，且环境假设在其上被检查或监控。

原则 1.8 (设计论题)**.** [**A**] 架构就是人为制造的尺度分离。自然界的层级由尺度分离与重整化免费获得；软件的层级不会自然涌现，必须由接口、封装与不变量付费制造，并且随时会漏。

因此设计是正向压缩，理解是逆向压缩，两者是同一结构的两个读取方向。架构的可测量目标函数是一次典型变更所需的读集期望 $\mathbb{E}[\rho]$。

四条论题及其形式化结果合称**压缩—保证框架**。名称取自主张的两端：压缩是理解与设计的机制，保证是可信性的锚定方式。框架的核心断言是：一个复杂系统能被有界主体理解，当且仅当它可被压缩；而它可被压缩，当且仅当它存在一套尺度分离，使每一层都拥有一个被显式声明的无视集。

1.4　认识论纪律

本文处理的材料跨越工程直觉、复杂系统理论、信息论、数理逻辑与社会认识论，越界风险很高。为控制风险，全文遵守以下纪律。

(i) **强度标注**。每个主张标注其为定理、命题、猜想、假设或启发式。拒绝把启发式写成定理。

(ii) **不作主张清单**。第十一章显式列出本文不主张的内容，包括被本文拒绝的若干” 权威定理”。

(iii) **假设显式化**。凡结论依赖环境假设，必须在陈述中写明；凡保证有边界，必须在边界处标注。

(iv) **可证伪性**。第九章把核心命题降格为十四条可证伪假设，并给出操作化、对照组、证伪条件与风险。

(v) **不与结论同构的验证**。凡提出” 生成不可信、验证需独立” 的地方，该原则同样适用于本文自身：本文的核心承重命题必须可被独立检验，且不得依赖任何未证明猜想。

第 (v) 条约束本文自身。若干修辞性主张被刻意从承重结构中卸下：不用 Gödel 不完备定理或 Rice 定理去证明” 意图与规格之间存在原则性裂缝”（理由见第七节 7.5），不用 $\mathrm{P} \neq \mathrm{NP}$ 的未决性去支撑” 验证比生成便宜”（理由见第七节 7.6），也不把 Simon 的近可分解性当作普遍定律（见第三章与第十一章）。这几条由卸载而非加固获得强度：拆掉未经证明的引用之后，核心命题不再依赖任何猜想。

1.5　主要贡献

形式化贡献。

(C1) 以抽象映射的核刻画层级的无视集，使” 这一层可以不关心什么” 从修辞变为可计算对象（定义 2.8）。

(C2) 以 Galois 连接刻画层间语义保持，并给出层级可靠性的条件式（定义 2.2、定义 2.3、命题 2.4）。

(C3) 以契约操作元的良定义性把局部保证的可组合性归约为接口图的良类型着色与无环性（定义 2.27、命题 2.29）。

(C4) 以层族 $\pi : \mathcal{L} \to \mathcal{P}$ 把内核六职能的跨域同构与沿问题形状的迁移区分开来（定义 4.16、命题 4.17）。

(C5) 以粗粒化判据使收敛论题可以在单个项目内部被证伪（定义 4.20、猜想 4.21）。

(C6) 以一条摊销不等式比较 $\Theta(nm)$ 的直接交互与 $\mathcal{O}(nk+m)$ 的基座交互，给出内部基础设施化的盈亏平衡阈值（定理 4.4）。

(C7) 以操作语义完备性论证内核六职能的条件必然性，并由此导出”自建 vs 外借”的判定准则（定理 4.9、推论 4.12）。

(C8) 以抽象映射的像空间维度给出过约束/欠约束判据，从而把”底层唯一、高层多解”形式化为解集几何（命题 3.5）。

(C9) 以读集期望 $\mathbb{E}[\rho]$ 给出架构质量的可测量目标函数，并给出一个界，表明不变量集中化会降低读集，且该界是相对于那些没有任何独立证据能够确定的强制点陈述的（定义 6.1、定义 6.2、定理 6.3）。

(C10) 以描述长度差给出技术债的精确定义（定义 6.12），并给出重构的率失真解读。

(C11) 一个可辨识性判据，用于判断理解能否收敛，它把观测不足与采样不足区分开来，而线性观测秩是它在有限维下的特例（定义 5.9、命题 5.10）。

(C12) 以可治理复杂度的稳定性条件，以及所需人力的标度律（定义 6.15、命题 6.16）。

(C13) 以 Hoeffding 界给出共识层可靠性的指数上界，并证明当存在相关错误模式 q 时该界被 q 支配、增加节点数无益，从而确立”多样性而非数量”为共识设计的核心变量（定理 7.29、命题 7.30、第 7.12 节）。

(C14) 给出完整的保证条件式，把功能、资源、副作用、系统组合与目标表达五类正确性统一为一条可追溯的条件蕴含（定义 2.24、命题 7.42）。

方法论贡献。

(M1) 一份七步”下行协议”，把理解大型系统从顺序阅读改造为分层建模与主动辨识。

(M2) 一组可执行的”缝探测器”清单，用于在陌生代码库中定位真实模块边界。

(M3) 一份七步”正向压缩”设计流程，把设计目标从形容词改造为可优化目标函数。

(M4) 十四条可证伪假设及实验设计。

(M5) 一份把社会认识论与证明携带代码耦合起来的交叉研究方向，即文献地图上最空白的可占据位置。

1.6　全文结构

第二章建立统一的形式框架：层级、抽象映射、无视集、契约、轨迹、保证与信任。第三章处理问题 1.1，把收敛拆成三个强度不同的命题并给出强度定律。第四章处理问题 1.2，证明六职能的条件必然性，并推导出基座收回自身成本的盈亏平衡阈值。第五章处理问题 1.3 的前半，给出逆向压缩的理解理论。第六章处理问题 1.3 的后半，给出正向压缩的设计理论。第七章处理问题 1.4，给出从意图到可验证执行的七段链与双层信任结构。第八章把层级从认知辅助升级为验证义务的分解单元。第九章给出可证伪的研究纲领。第十章给出文献地图与定位。第十一章给出反对意见与限度。第十二章收束。

注 1.9 (阅读指南). 只关心架构收敛的读者可读第二、三、四章；只关心理解与设计的读者可读第五、六章；只关心正确性与验证的读者可读第七、八章；关心可检验性的读者应直接读第九章。

1.7　术语约定

本文使用若干在中英文语境中含义漂移的术语，此处一次性约定，完整术语表见附录 C。

术语	本文含义
层级 / level	一组（表示空间、行为集合、观察映射、环境假设、成本模型、不变量），相对研究对象与当前任务定义，不是代码的物理目录。
无视集 / oblivious set	某一层在原理上永远无法区分的差异集合；等于该层抽象映射的核。
尺度分离	上下层之间能够各自成立有效描述、且下层的实现细节不在上层语义中出现。
内部基础设施化	系统把重复出现的管理问题提升为统一机制、并让上层依赖这些机制的过程。
内核六职能	身份与生命周期、分配与回收、调度、命名与寻址、通信与同步、故障隔离与恢复。
读集 / read-set	完成一次典型变更所必须阅读的模块集合。
保证 / assurance	在明确前提与明确范围内，某性质成立的、可追溯的证据关系。
机械保证	由一个极小、确定、可穷尽检视的判定器给出的保证。
共识保证	由多个独立、对抗性参与方给出的、统计学意义上可靠的判断。
制品绑定	被验证的工件与实际执行的工件之间的同一性关系。
意图鸿沟	自然语言（或更一般的）意图与其形式化投影之间的失真。

一个术语上的重要提醒：本文所说的”层”是*认知尺度*，而非物理层级。同一个组件在不同研究任务中可以处于不同层：一个异步运行时对普通应用开发者是基础设施，对其自身开发者则是需要深入修改的复杂系统。层级是相对于研究对象与当前任务定义的（见第八章）。

第二章　形式框架

本章建立全文共用的形式语言，把工程直觉中已在使用但未被命名的对象显式化：一层可以无视什么，层间语义保持在什么条件下成立，理解应下潜到什么深度，保证在什么范围内成立。

2.1　层级与抽象映射

定义 2.1 (层级)**.** 一个**层级**是一个六元组

$$\Lambda \;=\; \big(X,\ \mathcal{B},\ \mathcal{O},\ \mathcal{A},\ \mathcal{C},\ \mathcal{I}\big),$$

其中

- X 是**表示空间**（该层允许出现的状态集合）；
- $\mathcal{B} \subseteq X^{\mathbb{N}}$ 是**行为集合**（该层允许的执行轨迹）；
- $\mathcal{O}$ 是**观察映射**，把本层行为投影到上层可分辨的观察代数；
- $\mathcal{A}$ 是**环境假设**集合，即该层结论成立所依赖的外部条件；
- $\mathcal{C} : \mathcal{B} \to \mathbb{R}^{k}_{\geq 0}$ 是**成本模型**（时间、空间、能耗、带宽等 k 个分量）；
- $\mathcal{I} \subseteq \mathcal{B}$ 是**不变量集合**，即在该层任何允许的转换下都必须保持的性质。

这个定义中的六项并非等权重。X 与 $\mathcal{B}$ 描述" 存在什么"，$\mathcal{O}$ 描述" 看不见什么"，$\mathcal{A}$ 描述" 依赖什么"，$\mathcal{C}$ 描述" 代价是什么"，$\mathcal{I}$ 描述" 什么不能坏"。第八章将论证：$\mathcal{O}$ 与 $\mathcal{I}$ 是决定一个层级是否真实的两项；一个层级若说不出自己的 $\mathcal{O}$ 与 $\mathcal{I}$，它不是一个层级，只是一个目录。

定义 2.2 (抽象与具体化)**.** 设 Λ_i 与 Λ_{i+1} 为相邻两层（下标小者为上层）。**抽象映射** $\mathcal{O}_i : \mathcal{B}_{i+1} \to \mathcal{B}_i$ 把下层行为投影为上层行为；**具体化映射** $\gamma_i : \mathcal{B}_i \to 2^{\mathcal{B}_{i+1}}$ 把上层行为展开为下层的实现集合。若存在偏序 $\sqsubseteq_i$ 与 $\sqsubseteq_{i+1}$ 使得

$$b \sqsubseteq_{i+1} \gamma_i(b') \quad\Longleftrightarrow\quad \mathcal{O}_i(b) \sqsubseteq_i b', \qquad \forall b \in \mathcal{B}_{i+1},\ b' \in \mathcal{B}_i, \tag{2.1}$$

则称 $(\mathcal{O}_i, \gamma_i)$ 构成一对 **Galois 连接**，记为 $\mathcal{O}_i \dashv \gamma_i$。

Galois 连接的意义在于：它精确刻画" 抽象化丢掉了什么、保住了什么"，并且是 Cousot 抽象解释框架的核心工具 [20]。它在本文中的角色是确切的，但比通常以为的更窄：Galois 连接使语义保持条件成为一个良定义、具有典范证明形态的义务，而不是一个可自动判定的条件。要判定它，仍然要求两个论域都有有效表示、二者之间的关系可计算、所涉性质落在某个

可判定的片段内，并且分析能够终止。当 Galois 连接无法建立时，必须退回到模拟关系、双模拟、上下文等价或人工论证，而它们各自带有自己的组合义务，如注 2.5 所述。

定义 2.3 (层间可靠性)**.** 设 $A \in \mathcal{A}_i$ 为上层声明的一条环境假设，$\mathcal{B}_{i+1} \restriction A$ 表示下层在假设 A 下的行为子集。称 Λ_i 相对于 Λ_{i+1} 在假设 A 下**可靠**，若

$$\mathcal{O}_i\big(\mathcal{B}_{i+1} \restriction A\big) \;\subseteq\; \mathcal{B}_i. \tag{2.2}$$

命题 2.4 (可靠性的组合条件)**.** *设存在一条链 $\Lambda_0, \Lambda_1, \dots, \Lambda_n$（下标递增表示向下具体化），且对每个 i 存在假设 $A_i \in \mathcal{A}_i$ 使式 (2.2) 成立。若*

(a) 观察映射相容：$\mathcal{O}_i \circ \mathcal{O}_{i+1} = \mathcal{O}_i^{(i+2)}$，其中 $\mathcal{O}_i^{(i+2)}$ 是 $\mathcal{B}_{i+2}$ 到 $\mathcal{B}_i$ 的直接观察；

(b) 环境假设闭合：$\mathcal{O}_i(\mathcal{B}_{i+1} \restriction A_i) \subseteq \mathcal{B}_i \restriction A_{i-1}$，即下层满足的假设足以支撑上层的假设；

则 $\mathcal{O}_0^{(n)}\big(\mathcal{B}_n \restriction \bigcap_i A_i\big) \subseteq \mathcal{B}_0$。

证明. 对 n 归纳。$n = 1$ 即式 (2.2)。(b) 保证归纳步中上层的环境假设在上层行为集合内成立，从而式 (2.2) 可逐层套用；(a) 保证复合观察与逐层观察一致，故结论中的包含关系对最上层成立。 ■

注 2.5 (关系情形下的条件 (a))*.* 命题 2.4 是对函数式观察映射陈述的，因为条件 (a) 是复合的等式，$\mathcal{O}_i \circ \mathcal{O}_{i+1} = \mathcal{O}_i^{(i+2)}$。注 2.6 已记录：在若干转换边上，层间关系是一般关系 $R_i \subseteq \mathcal{B}_{i+1} \times \mathcal{B}_i$ 而非函数。二者在这里不可互换，这一差别值得确切写出，而不是留给读者。

在关系情形下，条件 (a) 的自然替代物是*复合相容性包含*

$$R_{i+1} \circ R_i \;\subseteq\; R_i^{(i+2)}, \tag{2.3}$$

其中 $R_i^{(i+2)}$ 是 $\mathcal{B}_{i+2}$ 到 $\mathcal{B}_i$ 的直接关系，而 $b_{i+2}\,(R_{i+1} \circ R_i)\,b_i$ 成立，当且仅当存在某个 b_{i+1} 满足 $b_{i+2} R_{i+1} b_{i+1}$ 且 $b_{i+1} R_i b_i$。

关于式 (2.3) 有三点必须说明。

(a) 它是一条包含，不是等式。归纳步用到的正是包含；等式是严格更强、且不必要的前提。

(b) 它不被函数式条件所蕴含，也不能由每个 R_i 分别是（比如说）一个模拟推出。复合相容性是关于族 $\{R_i\}$ 的独立闭合性质，或需假设，或需证明。

(c) 因此，命题 2.4 在本文中只对函数式情形给出证明。它的关系版本是有条件的：它在式 (2.3) 之下成立，而后者是附加前提，不是推论。

对第七章真正重要的读法是：从函数式抽象改用前向模拟、后向模拟、双模拟或上下文等价，就承担了为所使用的关系族建立式 (2.3) 的义务。在实践中这一义务由相应理论的复合引理兑现，而这恰恰是该要求在被非正式地写下时容易被忽略的原因。

注 2.6 (与源研究报告记号的关系)*.* 本文的六元组定义 2.1 是源研究报告四元组 $M_i = (X_i, \mathrm{Beh}_i, \mathrm{Obs}_i, A_i)$ 的扩展：本文显式加入 $\mathcal{C}$ 与 $\mathcal{I}$ 两项，因为第六、七章需要把资源保证与不变量强制纳入层级对象。源报告以*解释关系* $R_i \subseteq \mathrm{Beh}_{i+1} \times \mathrm{Beh}_i$ 描述层间语义，并要求对成功转换 $x_{i+1} = T_i(x_i)$ 的每个允许目标行为 b_{i+1}，都存在源行为 b_i 使 $R_i(b_{i+1}, b_i)$ 成立。这是一个*模拟条件*，比式 (2.2) 的集合包含更一般：当 R_i 由函数诱导（即 $R_i = \{(b, \mathcal{O}_i(b))\}$）时二者一致。本文在需要一般关系时沿用 R_i，在只处理函数式抽象时使用 $\mathcal{O}_i$。源报告特别强

调：对不同语言与不同观察域，R_i 可分别是前向模拟、后向模拟、双模拟、上下文等价或精化关系，不能假定所有转换都使用同一种关系。这一限定本文完全接受。

注 2.7. 命题 2.4 是” 局部保证如何组合成端到端保证” 的精确形式，也是它为什么常常失败的精确形式：组合需要两个条件同时成立，而工程实践中 (a) 常因观察映射不一致而破坏（例如” 事务提交成功” 在应用层与存储层指代不同的观察），(b) 常因环境假设不闭合而破坏（例如上层假定” 写入即持久”，而下层的持久化保证依赖一个未被声明的刷盘策略）。第七章将把这两条失效模式作为保证边界表的骨架。

2.2　无视集与压缩率

以下两个定义是本文形式化努力的核心。它们把” 这一层可以不关心什么” 从一句工程修辞变成一个可计算对象。

定义 2.8 (无视集). 层级 Λ_i 相对于下层 Λ_{i+1} 的**无视集**定义为抽象映射的核：

$$\mathrm{Obl}(\Lambda_i) \;:=\; \ker \mathcal{O}_i \;=\; \big\{(b, b') \in \mathcal{B}_{i+1}^2 \;:\; \mathcal{O}_i(b) = \mathcal{O}_i(b')\big\}. \tag{2.4}$$

若 $(b, b') \in \mathrm{Obl}(\Lambda_i)$，称 Λ_i **对 b 与 b' 之间的差异无知**。

命题 2.9 (无视集的基本性质). $\mathrm{Obl}(\Lambda_i)$ 是 $\mathcal{B}_{i+1}$ 上的等价关系，且商集满足

$$\mathcal{B}_{i+1} / \mathrm{Obl}(\Lambda_i) \;\cong\; \mathcal{O}_i(\mathcal{B}_{i+1}).$$

证明. 等价关系的三条性质由 $=$ 的性质直接得到。满射 $\mathcal{O}_i : \mathcal{B}_{i+1} \to \mathcal{O}_i(\mathcal{B}_{i+1})$ 诱导双射 $\mathcal{B}_{i+1}/\ker \mathcal{O}_i \to \mathcal{O}_i(\mathcal{B}_{i+1})$，由商集的泛性质即得。 ■

命题 2.9 给出一个直接而重要的结论：**抽象就是按核做商**。一个层级暴露给上层的全部信息，就是下层行为在这个核下的商。因此：

- ” 这一层保证什么” = 商集 $\mathcal{O}_i(\mathcal{B}_{i+1})$ 的元素；
- ” 这一层不保证什么” = 被商掉的差异，即 $\mathrm{Obl}(\Lambda_i)$；
- ” 这一层可以无视什么” = 与 $\mathrm{Obl}(\Lambda_i)$ 相容的一切下层细节。

第三条的后果最深：它使” 无知” 成为一种可以被主动声明、被检查、被追责的性质。一个从未声明自己无视集的模块，其契约是不完整的。

注 2.10 (无视集相对于任务与性质). 定义 2.8 从单个对象（观察映射 $\mathcal{O}_i$）定义了 $\mathrm{Obl}(\Lambda_i)$。这是正确的定义（正是它使命题 2.9 为真），但它压缩掉了一个在使用中重要的依赖关系：$\mathcal{O}_i$ 不是由系统给定的。它是被选择的，并且是相对于任务被选择的：一个实现细节可能与业务结果完全无关，却对尾延迟、对安全边界或对能耗具有决定性作用；同一次改动每次使一方变好、使另一方变差，就演示了这一点。

在依赖关系重要之处，记号应当把它显示出来。本文记 $\mathrm{Obl}(\Lambda; T)$ 为层级 Λ 在任务 T 下的无视集，记 $\mathrm{Obl}(\Lambda; \varphi)$ 为相对于所要保证的性质 φ 可以被无视的差异。不带参数的 $\mathrm{Obl}(\Lambda)$ 于是表示在所考虑的全部任务上的交，这正是一个模块所声明的无视集若要对其尚未遇见的调用者成立所应取的值。

这里有一个值得提炼的口号，因为人们实际使用的口号是错的。有效的抽象不是尽可能多地删除信息，而是删除那些不可能影响所要得出的结论的差异。一个隐藏了很多、却保住了对它提出的各种性质的层级，优于一个隐藏得更多、却保住得更少的层级；单看 κ 无法区分二者，这正是为什么压缩率是一种测量，而目标是定义 2.19 的约束优化，不是这个比率本身。

注 2.11 (核的定义给出了什么、没有给出什么). 命题 2.9 是关于一个核的陈述；对于从它能推出什么，值得说得确切，因为有两件事常被假定为可推出，而它们并不能。

它不给出可计算性。$\mathrm{Obl}(\Lambda_i)$ 是等价关系，这由 $\mathcal{O}_i$ 是函数得到；但该关系是否可判定、其等价类能否被枚举，还额外要求 $\mathcal{O}_i$ 有效可计算，且其输出是否相等可判定。对于一个观察映射由无穷行为集合上的操作语义定义的层级，这两点都不是自动成立的。因此这个定义是关于无视集是什么的规格，而不是产生无视集的算法。

它也不提供构造。本节没有任何内容说明如何从一个真实系统的执行语义、接口与已观察行为得到它的 $\mathcal{O}_i$，也没有说明如何计算或近似该层级隐藏了哪些差异。这项工作并非因篇幅原因被推迟：它是一个独立的问题；本文立场的诚实表述是，无视集是一个应被声明、随后被检查的设计制品，而不是一个可被提取的分析制品。声明本身就是那个制品；可用于检验它的，是原则 6.11 的两条边界判据：以泄漏对检验的充分性，与以非干扰检验的完备性，而两者都是部分的。同时通过两者的声明并未被证明为正确，只是尚未被反驳。

注 2.12 (声明可能是不完备的：第二条边界判据). 定义 2.8 把 $\mathrm{Obl}(\Lambda_i)$ 定义为已声明的观察映射 $\mathcal{O}_i$ 的核。该层级看不见的一切按构造对 $\mathcal{O}_i$ 不可见，因此没有任何失效可以定位在 $\mathrm{Obl}(\Lambda_i)$ 自身之内。可能发生的失效位于上一层，在声明之中：$\mathcal{O}_i$ 可能不是上层实际能观察到的全部。若一个被 Λ_i 声明为可无视的差异仍然改变了时间、资源占用或失败行为（这三者从上层可见，却不在 $\mathcal{O}_i$ 之中），那么这个声明是假的，尽管用 $\mathcal{O}_i$ 作出的每一句陈述仍然为真。两个行为在 $\mathcal{O}_i$ 下相等，在实践中却可区分。

这个问题很古老，为它命名可以换来一整套技术。设 $D \subseteq \mathrm{Obl}(\Lambda_i)$ 是该层级声明为可无视的差异集合，并设上层实际可用的观测装置为 $\mathcal{J}$，它可能严格大于 $\mathcal{O}_i$。要求 D 中没有任何差异经由 $\mathcal{J}$ 可见，正是 Goguen 与 Meseguer 的非干扰条件 [42]：一个论域中的变化不得使另一个论域的观察发生变化。用本文的语汇：

信息流概念	本文中的对应物
高层输入	被声明为可无视的差异，$D \subseteq \mathrm{Obl}(\Lambda_i)$
低层观察	上层实际可用的观测装置，$\mathcal{J}$
非干扰	D 不改变 $\mathcal{J}$-观察：该声明是完备的
隐蔽信道	一条位于 $\mathcal{O}_i$ 之外、却仍使 D 可见的 $\mathcal{J}$-路径

第五章提供了边界忠实性的另一半：接口对外部行为是否充分，其检验方式是寻找一个泄漏对。本注提供的是充分性无法提供的东西，因为泄漏对是关于 $\mathcal{O}_i$ 的陈述，对 $\mathcal{O}_i$ 遗漏了什么一字未说。第 6.2 节把两条判据并列陈述。

实践上的后果是一套借来的工具箱，而不是一个新理论：差分测试是充分性方向上的工具，污点追踪、信道审计与时间通道分析是完备性方向上的工具。已经编目的反模式”用类型系统强制多租户隔离”（例 8.12(d)）正是完备性失效在其天然栖息地中的形态：类型系统约束了已声明的信道，而未被声明的信道仍然敞开。

注 2.13 (模式实在性的判据). 命题 2.9 与一个独立的哲学判据同构。Dennett 的*真实模式*判据断言：一个模式是真实的，当且仅当用该模式描述系统比描述原始数据更经济 [63]。在该判据下，” 这一层是否存在” 不是一个本体论问题，而是一个*压缩率*问题，恰是定义 2.14 所度量的量。

这一判据后来获得了一个更锐利、且明确是信息论的表述；它值得引用，因为它比 Dennett 的原始陈述更接近定义 2.14。Ladyman 与 Ross 主张” 存在即是一个真实模式”：一个模式是真实的，仅当它有一个模型，该模型在一个*逻辑深度*低于该结构的位图编码的编码中承载关于某个结构的信息；并且他们认为，奥卡姆剃刀所建议的是：只把那些使归属于世界的逻辑深度最小的模式计为真实的 [81]。这是一个用编码语汇、而不是用非正式的经济性语汇表述的存在性压缩判据，而定义 2.14 的压缩率正是它所要问的量。这一等同是双向的，并非没有代价：若一个层级是真实的，当且仅当承认它带来压缩，那么一个不带来压缩的层级就不仅是不方便的，而是不真实的；这比本文所需的主张更强，本文不作此主张。本文采用的是较弱的、条件式的读法：*若*一个层级所声明的观察映射带来压缩，则它的核是一个真实的等价关系；压缩多少才算够是一个设计决定，而不是一个本体论阈值。

这一等价有三项后果，本文在后续各章中分别使用：

(a) 层级不是被发现的自然种类，也不是任意的约定；它们是压缩算符的不动点。

(b) ” 架构收敛” 命题（第三章）不需要独立的哲学辩护：若两个系统在相近约束下独立压缩，其不动点相近是压缩算子性质的推论，而不是关于世界的额外假设。

(c) 判据是定量的，因此” 某一层级是否真实” 是可测的，而不是可争论的。

定义 2.14 (压缩率). 设下层是经由一个*有限编码*的行为描述被观察的：一个可测映射 $\mathrm{enc} : \mathcal{B}_{i+1} \to \mathcal{X}$，其中 $\mathcal{X}$ 是可数集合，μ 是 $\mathcal{X}$ 上的概率分布，H 是离散分布的 Shannon 熵。层级 Λ_i 的**压缩率**定义为被商掉的信息量

$$\kappa_i \;=\; H_{\mu}\big(\mathrm{enc}\big) \;-\; H_{\mathcal{O}_i \# \mu}\big(\mathcal{O}_i \circ \mathrm{enc}\big) \;=\; H_{\mu}\big(\mathrm{enc} \mid \mathcal{O}_i \circ \mathrm{enc}\big), \tag{2.5}$$

最后一个等号成立，因为对离散随机变量的函数 f 有 $H(X) - H(f(X)) = H(X \mid f(X))$。这就是在 $(\mathcal{O}_i, \gamma_i)$ 下不可恢复的信息量。这一表述直接借用信息论 [55] 与算法信息论 [57] 的语汇。

这个定义附带三项限定，因为每一处都是该量容易被误用的地方。

第一，编码是数据的一部分，不是呈现细节。κ_i 是相对于固定的 enc、固定的 μ 与固定的计数单位定义的；三者中任何一个改变，它都随之改变。行为集合是轨迹的集合，因而是无穷对象，在它们之上没有直接定义的熵：被测量的是一个有限描述的熵。凡本文后文说某一层级” 带来压缩”，或说一个抽象优于另一个抽象，该比较都是相对于一个已声明的编码与一个已声明的任务分布而言的，否则它根本不是一个比较。

第二，式 (2.5) 中的恒等式不能经受向连续变量、或向表示空间变更的过渡。对微分熵，$H(X) - H(f(X)) = H(X \mid f(X))$ 的对应物不成立（微分熵在重参数化下不是不变的），因此一个把状态空间换成连续空间的读者会在不知不觉中换掉所测的量。本文并不阻止这种替换；它需要另给一个定义。

第三，κ_i 不是描述长度，也绝不能与描述长度等同。本文在别处（定义 2.19 与第六章的技术债落差）使用描述长度 ℓ，取的是最小描述长度意义，其中被度量的对象是一个在固定编码下的模型。被一个抽象商掉的量与一个模型的最短描述的长度，是定义在不同自变量上的不同泛函；关于其中一个的结果不能迁移到另一个。二者只通过编码定理相联系，且只在本文未建立的条件之下相联系。

压缩率本身不是目标：把系统压缩成” 什么都不保证” 可以获得任意大的 κ。目标是在保持任务相关预测能力的前提下最大化 κ。这引出第 2.4 节的形式化。

2.3　架构关系型

仅用” 层” 描述架构是不够的。第三章将论证：跨系统的相似性主要存在于关系而非组件之中。因此需要把关系本身类型化。

定义 2.15 (架构关系型)**.** 一个系统的**架构关系型**是一个带类型的有向多重图

$$\mathcal{G} \;=\; \big(V,\; E_{\mathrm{abs}} \sqcup E_{\mathrm{dep}} \sqcup E_{\mathrm{cmp}} \sqcup E_{\mathrm{ctl}} \sqcup E_{\mathrm{ctr}}\big), \tag{2.6}$$

其中 V 为构件集合，五类边分别表示：

(a) E_{abs} **抽象边**：$u \to v$ 表示 v 的多个细节被 u 统一表达。附带的标注是”u 保留了 v 的哪些语义、隐藏了哪些细节”。

(b) E_{dep} **依赖边**：$u \to v$ 表示 u 为完成自身职责需要 v。标注为” 依赖抽象接口还是具体实现”。

(c) E_{cmp} **组合边**：$u \to v$ 表示 u 由 v 构成，标注为” 拥有（own）还是仅使用（use）”。

(d) E_{ctl} **控制边**：$u \to v$ 表示执行权在何时机从 u 转移到 v，标注为” 同步调用、异步消息、事件回调或数据驱动”，以及” 谁负责调度”。

(e) E_{ctr} **契约边**：$u \to v$ 表示 u 向 v 承诺什么、要求 v 满足什么前置条件，标注为前置条件、后置条件、不变量与失败语义。

把五类关系分开记录不是形式主义。一个常见的理解错误是把它们混为一谈：如果只看” 调用图”，那么” 线程池组合了线程与队列”” 线程池依赖同步原语”” 线程池通过提交接口被调用”” 线程池承诺关闭后不再接受新任务” 这四件事会被压成同一类边，从而丢失全部结构性信息。第五章的缝探测器本质上就是在恢复这五类边的区分。

定义 2.16 (抽象骨架)**.** 设 ϕ 为一个**骨架映射**：把构件替换为其在某一职责分类系统中的角色标签，把五类关系保留其类型而抹去其具体实例。称 $\phi(\mathcal{G})$ 为 $\mathcal{G}$ 的**抽象骨架**。

第三章的核心命题可以用 ϕ 表述：$\phi(\mathcal{G}_1) \approx \phi(\mathcal{G}_2)$ 可以成立，即使 $\mathcal{G}_1$ 与 $\mathcal{G}_2$ 在源码层面毫无相似之处。

2.4　理解作为任务约束的有损压缩

定义 2.17 (任务)**.** 一个**任务**是一个二元组 $T = (Q, d)$，其中 Q 是一族**查询** $q : \mathcal{B} \to \mathcal{A}_q$（把系统行为映射到可观察的答案集），$d$ 是答案上的损失泛函。

查询的典型实例包括：”若我修改此处，哪些行为会改变？”“在什么条件下这条路径会失败？”“这个不变量由谁强制？”“增加一种新的实体类型需要修改哪些模块？”

定义 2.18 (T-充分性)**.** 设 M 为系统 S 在某一抽象层级上的模型。称 M 对 S 是 **T-充分的**，若

$$\forall q \in Q: \quad q\big(\mathcal{B}(M)\big) = q\big(\mathcal{B}(S)\big), \tag{2.7}$$

或（近似版本）$\mathbb{E}_{q\sim Q}\big[d\big(q(\mathcal{B}(M)), q(\mathcal{B}(S))\big)\big] \le D_T$。

定义 2.19 (描述长度与率失真目标)**.** 该定义建立在最小描述长度原则之上 [56, 40, 41]。记 $\ell(M)$ 为模型 M 的描述长度（在固定编码方案下），$L_T(M) = \mathbb{E}[d(q(\mathcal{B}(M)), q(\mathcal{B}(S)))]$ 为任务损失。理解 S 的**率失真目标**为

$$M^\star \;=\; \arg\min_{M}\; \ell(M) \qquad \text{s.t.} \qquad L_T(M) \le D_T, \tag{2.8}$$

或等价的拉格朗日形式 $\min_M\ \ell(M) + \lambda\, L_T(M)$。

命题 2.20 (连续松弛下的下潜停止准则)**.** *设模型族不由离散的下潜深度、而由一个实参数 $t \in [0,1]$ 参数化，t 增大时 M_t 保留更多的下层细节；并设 $\ell(M_t)$ 与 $L_T(M_t)$ 对 t 可微，且 $\mathrm{d}\ell/\mathrm{d}t \ge 0$，$\mathrm{d}L_T/\mathrm{d}t \le 0$。则在拉格朗日量 $\ell + \lambda L_T$ 的内点最优 $t^\star$ 处，驻点条件*

$$\left.\frac{\partial L_T}{\partial \ell}\right|_{t^\star} \;=\; -\frac{1}{\lambda} \tag{2.9}$$

成立；且若在某区间上 $\mathrm{d}L_T/\mathrm{d}t = 0$ 而 $\mathrm{d}\ell/\mathrm{d}t > 0$，则继续下潜被严格占优。

证明. 对 $\mathcal{L}(t) = \ell(M_t) + \lambda L_T(M_t)$ 求导并令其为零，得 $\mathrm{d}\ell/\mathrm{d}t + \lambda\,\mathrm{d}L_T/\mathrm{d}t = 0$，故在 $\mathrm{d}\ell/\mathrm{d}t \neq 0$ 之处有 $\mathrm{d}L_T/\mathrm{d}\ell = -1/\lambda$。若在某区间上 $\mathrm{d}L_T/\mathrm{d}t = 0$ 而 $\mathrm{d}\ell/\mathrm{d}t > 0$，则在该区间上 $\mathrm{d}\mathcal{L}/\mathrm{d}t = \mathrm{d}\ell/\mathrm{d}t > 0$，目标函数上升，故继续下潜被严格占优。 ∎

限定为连续松弛是刻意的，而且这正是该命题的全部内容。真实代码库中的下潜深度是离散的：一个模块要么被读到、要么没有被读到，不存在使描述长度平滑变化的半途深度。在离散族上，式 (2.9) 中的导数不存在，拉格朗日量不必是凸的，一阶最优性也无从定义；仍然成立的只有命题的后半部分：若越过某个深度后任务损失不再改善、而描述长度继续增长，则该深度之后的下潜被占优。这个较弱的陈述完全不需要可微性，而原则 2.21 用的正是它。率失真目标 (2.8) 不受这一限定影响：有条件的是微分形式，不是优化问题本身。

由此得到两条实践推论，它们也是所报告实验应当采取的做法。第一，模型族最好被当作一个离散的候选模型集合，其 (ℓ, L_T) 点对落在帕累托前沿之上或附近，而在前沿点之间的取舍是关于 D_T 的判断，不是一次求导。第二，比较相邻两个深度是一次测量，不是一次计算：再下潜一层的边际预测增益由有限差分、留一层消融，或过往改动的实测代价来估计；于是式 (2.9) 的微分规则被读作对前沿形状的描述，而不是一份计算配方。

原则 2.21 (下潜到”必须违反的不变量可见为止”)**.** *要把某个改动做对，只需下潜到管辖该改动的那条不变量变为可见的那一层，然后停止。继续下潜只增加描述长度，不降低任务损失。*

这条原则同时解释了为什么”阅读全部源码”不是理解，以及为什么”只看架构图”也不是理解：前者越过驻点进入被占优区域，后者尚未到达 T-充分性。

2.5　契约、轨迹与保证

定义 2.22 (行为契约)**.** 一个**行为契约**是一个八元组

$$S \;=\; \big(P,\, Q,\, E^{+},\, E^{-},\, R,\, I,\, F,\, A\big), \tag{2.10}$$

其中 P 为前置条件；Q 为后置条件；E^{+} 为允许的副作用集合；E^{-} 为禁止的副作用集合；R 为资源预算（时间、空间、带宽、能耗）；I 为执行期间必须恒真的不变量；F 为失败、取消、重试与恢复语义；A 为环境假设（硬件、操作系统、负载、外部服务）。

式 (2.10) 不是一种公认的规格格式。它的意义在于：正确性不再被压缩为一次返回值，而被定义为一组对整条执行过程的约束。这一点是第七章全部论证的起点。

定义 2.23 (轨迹与允许集)**.** 设 B 为一个可执行工件，E 为一个运行环境（处理器、指令集、操作系统、加载器、配置、外部服务与设备）。记 $\mathrm{Traces}(B, E)$ 为 B 在 E 中可能产生的可观察执行轨迹集合（包含输入输出、系统调用、状态修改、I/O、并发事件与失败行为），$\mathrm{Allowed}(S)$ 为契约 S 允许的轨迹集合。称 B **在** E **中满足** S，若

$$\mathrm{Traces}(B, E) \;\subseteq\; \mathrm{Allowed}(S). \tag{2.11}$$

式 (2.11) 比”输出正确”严格得多：它约束的是所有可能行为，包括不被期望的副作用。

定义 2.24 (保证与其条件式)**.** 一个**保证**是一个四元组

$$\mathfrak{A} = (\text{被保证的性质},\ \text{适用的对象},\ \text{依赖的前提},\ \text{证据}),$$

称性质 φ 在范围 Σ 内成立，当且仅当下列条件式成立：

$$\begin{aligned}
&\text{环境假设成立}\\
&\wedge\ \text{契约已明确且被授权采用}\\
&\wedge\ \text{验证器接受实现及其证明}\\
&\wedge\ \text{实现转换保持所需语义}\\
&\wedge\ \text{执行边界有效}\\
&\wedge\ \text{实际执行的对象即被验证的对象}\\
&\quad\Longrightarrow\ \text{在所声明的范围}\Sigma\ \text{内满足}\varphi.
\end{aligned} \tag{2.12}$$

式 (2.12) 的最后一项（本文称为**第四支柱**）在既有讨论中经常被遗漏，但它不是边角细节：即使验证器正确接受了一个二进制，如果部署系统实际运行的是另一个文件，或者加载时环境改变了关键前提，前面的结论不能自动套用。其形式化条件是一个同一性断言：

$$\mathrm{digest}\big(B_{\mathrm{deployed}}\big) \;=\; \mathrm{digest}\big(B_{\mathrm{verified}}\big) \quad\wedge\quad E_{\mathrm{deployed}} \models A_{\mathrm{claimed}}. \tag{2.13}$$

式 (2.13) 是内容寻址、可复现构建与加载链完整性之所以属于理论组成部分而非部署阶段收尾工作的原因。

2.6　信任的两种生成模式

定义 2.25 (可信计算基)**.** 对一个保证体系，其**可信计算基** TCB 是必须被无条件信任才能使该保证成立的组件集合：证明检查器及其执行、所用逻辑与语义定义、证明所绑定的程序与实际执行的程序之间的同一性、编译加载运行机制、以及所假设的硬件与环境条件。

TCB 最小化是本文对" 非自指" 这一直觉的严格重述。注意它的准确含义：机械系统不会背叛，但会有缺陷、故障与不完整的模型。因此信任的基础不是" 机械绝无错误"，而是" 机械的判定不依赖生成者的主观意愿"。这两者不能混同。

定义 2.26 (机械保证与共识保证)**. 机械保证**由单个确定性的、可穷尽检视的判定器给出，其对象是有限、静态、无歧义的符号对象（如证明项、类型推导、约束系统）。**共识保证**由多个独立参与方的对抗性审查给出，其对象是高维、动态、部分自反的判断（如" 这份规格是否忠实表达了意图"）。

这两种模式不是竞争关系，而是互补覆盖，其关键差异见表 2.1。

表 2.1: 两种信任生成模式的对照

维度	机械保证	共识保证
生成方式	极小判定器机械检查符号对象	多独立方对抗性审查并签署
覆盖命题	可形式化的性质	不可形式化的判断
保证强度	在所声明前提下绝对	统计意义上可靠，非绝对
失败模式	判定器缺陷、前提失效	合谋、利益捕获、认知同构
失败形态	离散（有缺陷或没有）	连续（独立性越高，失败率越低）
成本规模	与命题复杂度成比例	与参与方数量及独立性成比例
学科类比	数学证明检验	同行评审、陪审团、司法

表 2.1 中最重要的一行是失败形态。机械保证的失败是离散的，不是因为它更可靠，而是因为它的对象是有限静态对象，检查可以穷尽；共识保证的失败是连续的，因为它的对象是意向性的，只能被一个网络逼近。第 7.11 节将给出这种" 连续" 的精确形式，并证明一个反直觉结论：节点数量不是共识可靠性的支配变量，错误的独立性才是。

2.7　契约代数与组合

命题 2.4 给出了组合的充分条件，但那些条件是逐条列举的。本节给出契约的代数结构，使组合条件成为该结构的良定义性条件，而非一张检查表。

定义 2.27 (契约操作元)**.** 一个**契约操作元** $\mathcal{O}$ 由下列数据构成：

(i) **颜色**集合 Col，其元素为行为契约（定义 2.22）；

(ii) 对每个 $n \geq 0$ 与颜色 $S_0, S_1, \ldots, S_n$，一个**运算**集合 $\mathcal{O}(S_1, \ldots, S_n; S_0)$，其元素是以 $S_1, \ldots, S_n$ 为子组件契约、并满足 S_0 的复合体；

(iii) **代换**：由 $f \in \mathcal{O}(S_1,\dots,S_n;\,S_0)$ 与 $g_i \in \mathcal{O}(T_{i,1},\dots,T_{i,m_i};\,S_i)$（$i = 1..n$）得到

$$f \circ (g_1,\dots,g_n) \;\in\; \mathcal{O}\big(T_{1,1},\dots,T_{1,m_1},\dots,T_{n,1},\dots,T_{n,m_n};\,S_0\big);$$

(iv) **单位** $\mathrm{id}_S \in \mathcal{O}(S;\,S)$。

并要求代换满足结合律与单位律。该定义是标准的（无色）操作元定义在颜色集合上的版本[67]。直观上：颜色是" 这一块承诺什么"，运算是" 怎么把它搭起来"，代换是" 把子块换成更细的实现而不改变外层承诺"。

然而，结合律与单位律只对资源占用互不相交的子组件族成立；这是对象本身的一个真实限制，而不是其表述上的技术细节。式 (2.10) 中的资源分量 R 不是一个可加量。当两个子组件取用同一资源时，合并它们的预算涉及摊销与干扰（缓存、争用、队头阻塞），此时 $f \circ (g_1, g_2)$ 与 $(f \circ g_1) \circ g_2$ 可能给同一个复合体分配不同的预算。结合律失效，随之失效的是" 该结构对这些复合体而言是一个操作元" 这一主张。诚实的回应有两条：细化颜色，使其携带一个资源占用向量，并给出一个使合并满足结合律的资源代数；或者把操作元限制在资源互不相交的族上。本文没有资源代数可提供，因此取第二条，并把第一条记为正在进行的工作。

定义 2.28 (接口图与自由性)**.** 设复合体 $f \in \mathcal{O}(S_1,\dots,S_n;\,S_0)$。以子组件为顶点，若子组件 u 的保证被 v 的假设所引用，则连一条有向边 $u \to v$。所得有向图称为 f 的**接口图**。若接口图无环，称 f 的接口图是**无环的**；此时称 f 由接口图**自由生成**。

命题 2.29 (组合条件的结构刻画)**.** 命题 *2.4* 的条件 *(a)* 与 *(b)* 分别等价于契约操作元上的两个结构条件：

(a) **着色良类型**：观察映射相容（命题 *2.4* 条件 *(a)*）$\iff$ 代换在颜色上良定义，即复合体不要求子组件的契约提供其颜色未声明的观察；

(b) **环境闭合**：接口图无环是环境假设闭合（命题 *2.4* 条件 *(b)*）的充分条件，且其充分性只相对于一种拓扑证明策略而言，即按拓扑序自下而上构造复合体的那种策略。它不是必要条件。当图中存在环时，良基的构造不可用，但复合体并不因此失去意义：它可以被赋予不动点语义，或者契约满足关系可以被加强，使每个组件的保证被要求对其环境的一切允许行为成立，而不只是对环所提供的那一个行为成立。

论证. (a) 若观察映射相容，则每个子组件在接口上的投影与复合体的观察一致，代换不引入新的颜色需求；反之，若代换在颜色上良定义，则复合体的每个可观察量必可追溯到某个子组件颜色所声明的观察，即观察映射相容。

(b) 若接口图无环，则存在拓扑序，沿拓扑序逐层代换即得唯一的构造，环境假设的闭合性沿该序逐层验证。反之，若存在环 $u_1 \to u_2 \to \cdots \to u_k \to u_1$，则 u_1 的假设依赖 u_k 的保证，而 u_k 的保证又（经环）依赖 u_1 的假设，代换式的两侧互相定义，复合体按该序没有良基的构造。要使其有意义，必须额外引入不动点语义，即为该环单独声明一个存在唯一性条件；或者加强满足关系，如下面的注所述。 ■

注 2.30 (无环是充分条件，不是必要条件)**.** 无环互联是最简单的组合论证得以成立的条件：按拓扑序、一次一个组件，每个组件都针对已经建立的保证来兑现自己的假设。它不是关于契约

组合的一般条件；把它写成一般条件会夸大结论，而这种夸大在实践中是有后果的，因为带反馈的系统是常态，不是病态。

这一区分在契约文献中是已知的，且有标准形态。契约框架通常至少承认两种满足关系：*弱满足*，其中组件的保证只需对其环境实际表现出的行为成立；以及*强满足*，其中保证必须对环境被允许表现出的*一切*行为成立。该一般框架的综述见 [74]。这一区分不是装饰性的，它恰好解决了争点。对构成有向无环图的互联，弱满足已经足够，并按拓扑序复合；对含环的互联，弱满足不再足够，需要强满足 [73]。同一项工作给出了连续时间系统下的对应条件：互联必须是*良置的*，而良置性是加在环上的条件，它由不动点论证兑现，而不是靠排除该环。

对本文而言，随之而来的是两个主张的收窄，而不是方向的改变。第一，命题 2.29(b) 建立的是拓扑策略下的充分性，而本文对它的使用（重试、缓存回写与事件回环使既有的组合论证失效）按原样仍然正确，因为这类改动破坏的是那个论证，而不是组合本身。第二，悬而未决的问题不是” 如何避免环”，而是第九章已经以另一种形式提出的那个问题：如何把功能性性质、环境假设、资源预算、失败恢复与重试语义纳入同一个契约系统，使其组合规则覆盖带环的互联。把环当作失败并止步于此，会使这个问题被过早地关闭。

命题 2.29(b) 解释了第 7.4 节三个组合失效反例中的两个，也解释了两处既有做法为何必须引入额外假设：

- 动态系统的契约组合必须额外假设互联是*良置的*（well-posed），这正是为反馈环补上不动点语义；
- 软件契约在网络重试回路上失效，因为重试把一个无环的调用图变成了带环的接口图，而契约的着色未随环的出现而重新验证。

注 2.31 (第四类组合失效). 第七章把组合失效分为三类：颜色外延过小、接口图改变，以及契约分量缺失。定义 2.27 中的附加条件指出了第四类，而它不是那三类的特例。资源分量*确实*在颜色之中，接口图*确实*无环，每个子组件*确实*满足自己的预算，而复合体却不满足，因为两个子组件取用了同一资源。

时延预算是干净的例子。设 g_1 与 g_2 在孤立情况下各自满足 $10\,\mu\mathrm{s}$ 的预算，并设 f 在二者中任一被装入时都满足自己的预算；同时运行两者，复合体却不达标，因为二者争用同一个缓存。任何着色检查与任何无环性检查都看不到这一点，重新验证每一个颜色也同样看不到。

第七章把缓存争用一例读作契约分量缺失，这一读法在其限度内是正确的：资源分量当时根本没有进入颜色。第四类是那次修复*之后*仍然剩下的东西。把 R 放进颜色之后，仍然存在预算无法组合的复合体，而本该使组合良定义的代数恰恰是失效之处。这就是为什么该限制属于定义，而不属于组合定理的一条脚注。

注 2.32 (反馈是结构性的，不是工程细节). 命题 2.29 把” 能不能组合” 归约为” 接口图有没有环”。这一归约有一个非平凡后果：*引入反馈不是局部改动*。任何把无环接口图变为带环的改动（重试、缓存回写、双向同步、事件回环），都会使原先合法的组合论证失效，无论各组件自身是否仍然正确。这与工程直觉一致：这类改动是回归问题的高发区。

2.8　记号汇总

表 2.2: 全文记号

记号	含义
$\Lambda_i = (X, \mathcal{B}, \mathcal{O}, \mathcal{A}, \mathcal{C}, \mathcal{I})$	第 i 层（定义 2.1）
$\mathcal{O}_i \dashv \gamma_i$	层间 Galois 连接（定义 2.2）
$\mathrm{Obl}(\Lambda_i) = \ker \mathcal{O}_i$	无视集（定义 2.8）
κ_i	压缩率（定义 2.14）
$\mathcal{G}$	架构关系型（定义 2.15）
$\phi(\mathcal{G})$	抽象骨架（定义 2.16）
$T = (Q, d)$	任务（定义 2.17）
$\ell(M)$	模型 M 的描述长度（定义 2.19）
$L_T(M)$	任务损失（定义 2.19）
S	行为契约：八元组（定义 2.22）
$\mathrm{Traces}(B, E)$	工件 B 在环境 E 中的可观察轨迹集（定义 2.23）
$\mathrm{Allowed}(S)$	契约允许的轨迹集（定义 2.23）
$\mathfrak{A}$	保证四元组（定义 2.24）
TCB	可信计算基（定义 2.25）
$\rho(c)$	变更 c 的读集（定义 6.1）
$\mathcal{K}_6$	内核六职能（定义 4.8）
D	技术债（定义 6.12）
$\mathcal{O}$, Col	契约操作元与其颜色集（定义 2.27）
$C(t)$, K	可治理复杂度、理解容量（定义 6.15）
$\mathcal{J}$	观测算子（定义 5.9）
$\mathrm{rank}_{\mathcal{T}}\mathcal{J}$	任务子空间上的观测秩（定义 5.9）

第三章　结构收敛：过约束解集与工程吸引子

本章处理问题 1.1。高层结构确实收敛，但收敛机制与底层不同，不同强度的命题必须分开陈述；收敛强度依赖领域绑定度，其载体是关系而非组件。本章末尾给出可证伪的验证设计。

3.1　三个强度不同的收敛命题

假设 3.1 (弱命题：局部模式复现)**.** 不同领域的软件反复使用同一批局部结构解法：分层与阶段化、机制与策略分离、控制与数据分离、事件与状态机、声明式描述与执行引擎分离、前端与后端分离、资源所有权与生命周期管理。这些结构的重要性不在于名称相同，而在于它们具有相同的*结构性动机*。

弱命题有大量工程经验支持，其强度也最低：它几乎不可能被证伪，因此解释力有限。

假设 3.2 (中命题：抽象后结构相似性)**.** 对于来自不同领域、由不同团队开发、具有不同实现细节的大型软件系统，在消除实现语言、组件命名与领域语义差异之后，其高层架构在某个可定义的抽象空间中呈现出*显著高于随机基线*的结构相似性。

中命题是可操作化的核心命题。它可被操作化（第 3.6 节），且其成立与否不依赖任何形而上假设。

假设 3.3 (强命题：单点收敛)**.** 几乎所有正常偏置的大型软件最终都会收敛到同一种高层架构。

强命题证据不足，而且很可能过强。它的主要问题在于把” 存在吸引子” 与” 只有一个吸引子” 混为一谈。

本文以式 3.2（中命题）为核心命题，把式 3.1（弱命题）视为其退化形式，把式 3.3（强命题）视为过强表述而予以拒绝。

3.2　约束系统与解集几何

定义 3.4 (约束系统与解集)**.** 设设计空间为 d 维流形 $\mathcal{D}$，一个设计问题由 c 个独立约束 $g_1, \dots, g_c : \mathcal{D} \to \mathbb{R}$ 定义，其**解集**为 $\mathcal{S} = \{x \in \mathcal{D} : g_j(x) = 0,\ j = 1..c\}$。

命题 3.5 (过约束与欠约束：在一个正则性假设之下)**.** *设 $\mathcal{S} \subseteq \mathcal{D}$ 被描述为映射 $g : \mathcal{D} \to \mathbb{R}^c$ 的零点集，0 是 g 的正则值，且 g 的 c 个分量相互独立。则*

(a) 当 $c < d$ 时，若 $\mathcal{S} \neq \emptyset$，则它是维数为 $d - c$ 的子流形；

(b) 当 $c = d$ 时，$\mathcal{S}$ 是离散的——零维的——并且仅在一个附加假设（例如 $\mathcal{S}$ 的紧致性）之下才是有限的；

(c) 当 $c > d$ 时，$\mathcal{S}$ 在一般位置下为空。

证明. 在 g 关于 $\{0\}$ 横截的条件下，g 的每个独立分量把 $\mathcal{S}$ 的余维数降低一，故由横截性定理（Thom）得 $\operatorname{codim}\mathcal{S} = c$。这在 $d - c > 0$ 时给出维数 $d - c$；在 $c = d$ 时给出离散集合；而在 $c > d$ 时不可能成立，因为 d 维流形的子集不可能有超过 d 的余维数——故原像在一般位置下为空。$c = d$ 时的有限性不能由维数计数推出：非紧流形上的零维闭子集可以是无限的，$\mathcal{S}$（或 $\mathcal{D}$）的紧致性必须另行假设。 ■

对这一命题的读法需要两处更正，而这两处都比命题本身更重要。

第一，$c \geq d$ 的情形并不给出”一般为有限或空集”。零维给出的是离散性，而离散性只有在紧致性之下才给出有限性。更弱而正确的一般陈述是：当 $c \geq d$ 时解集没有内点——它是低维的层（stratum），而不是一个开区域——这既是该论证所需的全部，也是它所能支撑的全部。

第二，也是更实质的一点：这一前提假设很强，而所讨论的领域并不是该假设所描述的那类对象。横截性加独立性是关于流形之间光滑映射的条件，而软件设计空间两者都不是：它是离散的，其约束常常是不等式、枚举、兼容性关系以及彼此之间的依赖，其可行集通常是有限或可数的候选集合，而不是流形。因此该命题是底层情形的一个模型——一个有用的模型，因为它解释了为什么增加约束会减少自由度，以及为什么底层显得被钉死——而不是一条关于软件架构的定理。它应当被读作：在设计空间容许光滑描述、且约束相互独立之处，自由度随约束数目线性收缩，并在 $c = d$ 时消失。

本章关于层级高低的实际主张在这次削弱之后完好无损，因为承载这一论证的从来不是维数计数。经验主张是：在低层，约束源自已经存在的对象，因而彼此独立；而在高层，若干表面上的约束只是同一个偏好的不同表述，因而无法满足独立性（附录 A 给出了同一点，并加以推广）。维数计数是对这一不对称性的一幅图像，而不是它的证明；这一不对称性可检验的形式是假设 H_2，它问的是一个被测的领域绑定度是否单调地预测相似性——这是一个关于数据的问题，而不是关于横截性的问题。

命题 3.5 给出”底层唯一、高层多解”的一个干净解释，但需要用词谨慎。它不是在说底层真的被形式化为一个约束系统，而是说底层设计空间的有效维数很低、有效约束很多，因此其解集在几何意义上退化。

例 3.6 (底层). 考虑为一个固定指令集架构定义函数调用约定。设计自由度包括参数的传递位置（寄存器或栈）、寄存器的保存责任、栈帧布局、返回地址存放、可变参数的传递、对齐要求、以及与既有二进制、调试器、异常展开表、系统调用接口和动态链接器的兼容性。这些约束中绝大多数不是”可协商的偏好”，而是被已有对象固定下来的：既有二进制必须继续工作，调试器必须能回溯栈，异常必须能展开，内核必须能接收系统调用。因此 $c \geq d$，解集退化为极少数可行点，在实践上常常退化为一。

例 3.7 (高层)**.** 考虑为一个任务处理系统选择架构。设计自由度包括：单体中的函数调用、多进程消息传递、微服务 RPC、事件驱动异步处理、Actor 模型、分布式工作流引擎，以及它们在下列维度上的连续参数：部署粒度、状态归属、一致性模型、同步或异步、失败边界、数据分区方式。此时问题层面的约束显著少于自由度，$\mathcal{S}$ 是一个高维流形。

那么为什么高层仍然收敛？因为存在第二组约束：它们不来自问题，而来自制造与维护该系统的人类组织：

原则 3.8 (组织约束对解集的压缩)**.** 问题层面欠约束的高维解流形，会被组织层面的约束压缩成少数几个尖峰。这些约束包括：团队沟通成本、认知负荷上限、部署与运维能力边界、技术人才市场、既有技术栈的沉没成本、以及监管与合规要求。

由此得到一个推论：高层收敛不是朝向工程最优，而是朝向”有界组织可导航的结构”。这一点在后面的强度定律中会被精确化。

3.3　收敛机制

(M1) **复杂性约束**。设系统有 N 个组件，若依赖不受组织，潜在有向依赖数达 $N(N-1)$，无向达 $N(N-1)/2$。系统的可维护性因此要求对依赖施加结构：模块化、分层、显式接口、依赖方向约束、子系统边界。这些技术不完全相同，但都在解决同一个问题：限制复杂性传播。模块化与信息隐藏的经典论证见 [49]。

(M2) **信息流约束**。大量软件可被抽象为” 输入 → 处理 → 状态变化 → 输出”（并叠加反馈、并发事件、外部副作用与持续运行的状态机）。当不同职责具有不同的变化频率、错误模式与资源需求时，分离它们具有实际收益：解析与执行分离、业务规则与数据访问分离、纯计算与副作用分离、控制平面与数据平面分离。

(M3) **状态管理约束**。系统必须回答：谁拥有某个状态？谁可以修改它？它何时创建和销毁？多个组件如何观察它的变化？并发访问如何协调？出错时如何恢复一致性？这些问题具有很强的普遍性，因而反复产生相似的结构：事务与锁管理、进程与句柄管理、GUI 的状态所有者与事件循环、复制状态机、实体生命周期与系统更新。

(M4) **认知约束**。大型软件不仅受机器约束，也受开发者的理解能力约束。若所有组件之间都存在隐式关系，则即使系统在计算意义上可以运行，开发者也无法预测修改的影响。因此人倾向于构建层次、命名空间、模块、明确接口、可组合抽象与局部可理解的状态。软件架构不仅是在组织计算，也是在有限认知能力下组织可理解性。

(M5) **演化约束**。大型软件不是一次性设计完成的，而经历初始实现、功能增长、性能优化、故障修复、团队交接、模块重构与兼容性维护。某些决策会随演化逐渐显露成本，于是产生筛选效应。现实中的演化不保证最优，历史包袱、兼容性要求、组织结构与沉没成本都可能让不理想的架构长期存活。

机制 (M1)–(M3) 来自问题的客观结构，机制 (M4)–(M5) 来自人类与历史。这一区分对应第 3.4 节的强度定律：前三种机制产生跨领域相似性，后两种产生领域内相似性与路径依赖。

机制 (M4) 与 (M5) 的经典表述分别是认知负荷下的层级分解 [39] 与沟通结构对系统结构的决定作用 [50]。” 无终态” 架构观与多稳定形态之间的振荡见 [61]；组织形态通过团队拓

扑反向驱动架构的实践见限界上下文与领域驱动设计 [62]。

3.4　收敛强度与领域绑定度

定义 3.9 (领域绑定度)**.** 设层级 Λ 的语义依赖一组领域公理 $\mathrm{Ax}(\Lambda)$（例如" 账户余额可以为负" 或" 帧必须在 16.6 ms 内提交"）。定义 Λ 的**领域绑定度**为

$$\mathrm{Dom}(\Lambda) \;=\; \frac{\big|\mathrm{Ax}(\Lambda)\big|}{\big|\mathrm{Ax}(\Lambda)\big| + \big|\mathrm{Ax}_{\text{通用}}(\Lambda)\big|}, \tag{3.1}$$

即领域特有公理在该层全部公理中的占比。$\mathrm{Dom} = 0$ 表示该层完全由通用公理决定（如内存局部性、并发协调成本），$\mathrm{Dom} = 1$ 表示该层完全是领域约定。

猜想 3.10 (收敛强度定律)**.** *设 $\mathrm{Conv}(\Lambda)$ 为层级 Λ 在跨系统比较中的结构相似性度量（第 3.6 节给出操作化）。则 Conv 关于 Dom 单调递减：*

$$\Lambda_1 \preceq \Lambda_2 \implies \mathrm{Conv}(\Lambda_1) \;\geq\; \mathrm{Conv}(\Lambda_2) \qquad \text{当} \qquad \mathrm{Dom}(\Lambda_1) \leq \mathrm{Dom}(\Lambda_2). \tag{3.2}$$

猜想 3.10 有可观察的定性支持：越靠近物理与网络层，趋同越强（分布式计算的八条谬误是无协商余地的硬约束）；越靠近业务域，趋同越弱（保险核心系统与推荐系统在领域模型上几乎不会同构）；越靠近组织结构，越受沟通结构支配；越靠近技术成熟期，形态越收敛；越靠近核心与高风险模块，越保守趋同。

本文不把式 (3.2) 称为定律，因为它依赖 Dom 的可定义性与 Conv 的可测性，二者都需要在实验中被建立。它的价值是把一个模糊的方向性判断转化为一个可被证伪的单调性假设。

3.5　关系而非组件

原则 3.11 (关系优先)**.** 跨系统的结构共性主要存在于关系之中，而不是组件之中。不同系统不一定具有相同的模块，但可能具有相似的关系型：

- 哪些部分负责决策，哪些部分负责执行；
- 哪些部分拥有状态，哪些部分只能读取状态；
- 哪些部分定义稳定契约，哪些部分允许变化；
- 哪些部分协调全局，哪些部分独立工作；
- 哪些依赖可以跨越边界，哪些依赖必须被禁止。

用定义 2.15 与 2.16 的语言：需要寻找的不是一个通用的架构模板，而是一组能在不同系统中反复出现的结构关系及其成立条件。这与第四章的内核六职能是同一条思路的两个层次：六职能给的是管理职责的坐标，关系型给的是职责之间的坐标。

例 3.12 (数据库与浏览器的抽象骨架)**.** 一个数据库引擎包含 SQL 解析器、查询优化器、执行引擎、缓冲池、事务管理器与存储引擎；一个浏览器包含 HTML/CSS/JavaScript 解析器、样式与布局引擎、脚本执行引擎、渲染引擎、网络子系统与缓存持久化机制。二者的模块名称、内部算法与具体职责都不同。但在适当的骨架映射 ϕ 下，二者都表现为：输入与接口层（接收请求、解析输入、建立执行上下文）、解释与规划层（把外部意图转换为内部可执行表示）、

执行与协调层（调度工作、协调子系统、管理执行过程）、状态与资源层（管理数据、缓存、生命周期与持久化）。

例 3.12 的价值有限：这个骨架已经宽泛到几乎可以描述任何系统。因此需要一条对抽象层级的附加要求：

原则 3.13 (抽象层级的双向约束)**.** 有研究价值的抽象层级必须同时满足两个方向：向上，它必须能识别跨系统的共同结构；向下，它必须保留足够的区分能力，使不同系统在该层级上仍然可被区分。越抽象，实现差异越被消除，但描述也越容易失去区分力；存在一个恰当的中层。

第四章的六职能坐标正是为满足这一双向要求而设计的：它比”输入-处理-输出”窄（因此有区分力），比任何具体模块列表宽（因此跨领域可比）。

3.6　验证设计

中命题（假设 3.2）要成为科学命题，必须能回答”与什么比较”。一个完整的检验设计包含五步。

第一步：建立架构图。　为每个系统建立图 $\mathcal{G} = (V, E)$，节点代表组件或职责，边代表调用、依赖、控制流或数据流。按定义 2.15，不同类型的关系应分开记录，避免把完全不同的关系混为一谈。

第二步：定义抽象映射。　将具体组件映射到更一般的类别，例如：输入与解析、规划与决策、执行与协调、状态与持久化、通信与边界、资源与生命周期管理。这些类别不能预先定义得过于宽泛，否则任何系统都会显得相似。

第三步：选择有区分力的结构指标。　可用的指标包括：依赖图的模块化程度、层次结构深度、跨模块依赖比例、状态所有权的集中程度、控制流与数据流的耦合程度、核心组件的网络中心性、以及不同职责之间的依赖方向。

第四步：构建合理的对照组。　不能只比较几个著名软件然后发现它们都存在模块化。必须将观察到的相似性与适当的随机基线比较，并控制系统规模、领域、平台、架构风格和历史技术栈等因素。

第五步：区分真正的收敛与共同来源。　如果两个系统采用相同的架构模式，可能是因为：它们独立解决了相似问题；它们使用了相同的框架；它们受到相同的标准影响；它们借鉴了相同的设计文献；它们继承了相同的历史架构。只有在控制这些因素之后，才更有理由讨论独立的结构收敛。

第九章把这一设计写成假设 H_1 与 H_2。

3.7　反例与边界

必须明确列出使收敛命题失效或退化的情况：

(a) **路径依赖**。一旦某种结构被生态、工具链或标准锁定，它可能长期存留，即使次优。锁定的强度与迁移成本成正比，与被锁定结构的质量无关。

(b) **规模相变**。在 10^2 与 10^9 个实体两种规模下，最优结构是定性不同的：一致性模型、分片策略、故障模型全部改变。因此不存在一个” 大系统架构”，只存在” 某个规模区间内的架构”。这直接限制了任何” 所有大系统都收敛” 的表述。

(c) **组织政治**。沟通结构对系统结构的决定作用（Conway 定律）意味着，即使技术问题完全相同，不同组织仍会产生不同的系统边界。这既是收敛的部分原因（当组织形态趋同时），也是发散的部分原因（当组织形态不同时）。

(d) **强耦合系统**。当系统内部耦合无法被局部化时（例如紧耦合的实时控制系统或单体内核），模块边界可能不是设计选择而是被迫的，此时比较” 架构” 本身的意义下降。

(e) **抽象层级选择的人为性**。第三步与第四步的指标选择本身包含研究者自由度。不同的职责分类系统会给出不同的相似性结论。这是该路线最脆弱的方法论环节，必须通过预注册指标与多分类系统稳健性检验来缓解。

本章的可证伪部分是假设 3.2 与猜想 3.10：若在控制规模、领域、平台与框架之后，跨系统结构相似性不显著高于随机基线，或相似性关于 Dom 的单调性不成立，则本章的核心主张被削弱（假设 H_1 与 H_2，第九章）。弱命题 3.1 与原则 3.11 不受此影响，因为二者是解释性框架而非经验断言。

第四章　内部基础设施化：内核六职能的条件必然性

本章处理问题 1.2。核心主张是：任何足够复杂的软件系统都会在自己的问题域内重新发明一个操作系统。其原因不是工程师在模仿内核，而是一组语义义务在规模上无法被回避。本章给出该主张的前提、证明与判据，并给出一个可在单一项目内实施的可证伪检验。

4.1　现象与术语

观察到的现象是：当系统规模增长到一定程度，它会开始把原本分散在各处的机制提升为统一的系统级能力。具体而言：

- 多个功能都需要分配和释放资源；
- 多个子系统都需要管理对象生命周期；
- 多个组件都需要执行异步任务；
- 多个模块都需要共享状态并协调修改；
- 多个功能都需要统一的错误处理、日志与诊断机制；
- 多个执行路径都需要遵循相同的线程安全与并发规则。

如果每个模块独立解决这些问题，系统就会出现大量重复实现、不一致的行为与难以控制的耦合。因此系统开始将这些问题集中处理，形成公共机制、接口、管理器、运行时与执行模型。

定义 4.1 (内部基础设施化)**.** **内部基础设施化**指一个系统依次经历下列三个阶段：

(i) **共性识别**：把重复出现的需求识别为同一类问题（问题空间的归并）；

(ii) **机制统一**：把共性提升为公共能力，使不同功能通过公共接口访问共享资源，遵循共同的生命周期、错误处理与并发规则（机制的集中化与标准化）；

(iii) **结构统一**：上层功能不再直接解决全部底层问题，而在统一规则之上构建（结构上的统一）。

简记为 $\boxed{\text{共性识别} \to \text{机制统一} \to \text{上层组合}}$。

机制统一不意味着所有逻辑都集中在一个中心模块中。成熟的架构也可以通过协议、接口与分布式所有权实现统一。统一的是规则与契约，不一定是实现与所有权。这一区分是避免”上帝对象”的关键，第六章会把它写成一条设计原则。

4.2　参与者-资源结构

定义 4.2 (参与者-资源结构)**.** 一个**参与者-资源结构**是一个五元组

$$\Pi = \big(E,\ \mathcal{R},\ \to,\ \mathrm{Ready},\ \mathrm{Fault}\big), \tag{4.1}$$

其中 E 是并发存活实体的集合，$\mathcal{R} = \{r_1, \dots, r_m\}$ 是有限共享资源集合，$\to$ 是操作语义的转移关系，$\mathrm{Ready} \subseteq E \times \mathrm{Conf}$ 指出在给定配置下哪些实体可执行，Fault 是故障事件集合。

关键的两个量是 $n = |E|$ 与 $m = |\mathcal{R}|$。第 4.3 节将证明：决定是否需要统一基础设施的自变量是 $n \times m$ 的结构复杂度，而不是代码行数。

定义 4.3 (统一基座)**.** Π 的一个**统一基座**是一组机制 $\mathcal{U}$，使得每个实体 $e \in E$ 与每个资源 $r \in \mathcal{R}$ 的交互都通过 $\mathcal{U}$ 声明的接口进行，而不是由 e 直接实现。

4.3　集成成本定理与盈亏平衡阈值

定理 4.4 (集成成本)**.** *设参与者-资源结构 Π 有 n 个实体与 m 个共享资源。假设*

(a) 在无基座情形下，每个实体若参与资源 r，必须独立实现与 r 的正确交互，包括对 r 的不变量的独立处理；

(b) 在最大耦合情形下，每个实体参与全部 m 个资源；

(c) 有基座情形下，每个实体至多使用 k 个基座接口，且基座自身对 m 个资源各定义一次语义；基座的固定成本为 F（设计、实现与长期维护）。

则无基座情形的交互对数为 $\Theta(nm)$，有基座情形为 $\mathcal{O}(nk + m)$。记 $a = \sum_e |\mathcal{R}(e)| \le nm$ 为实体-资源关联数，c 为单次直接交互的单位成本，c_s 为单次基座接口的单位成本，C_{cons} 为维持一个资源的 n 份独立不变量实现彼此一致的成本，F 为基座的固定成本，则两种安排的成本为

$$C_{\text{直}} \;=\; c\,a + C_{\mathrm{cons}}, \qquad C_{\text{基}} \;=\; F + c_s\,(nk + m). \tag{4.2}$$

基座是期望意义上更好的安排，当且仅当 $C_{\text{基}} < C_{\text{直}}$，即当且仅当

$$c\,a + C_{\mathrm{cons}} \;>\; F + c_s\,(nk + m). \tag{4.3}$$

证明. 无基座情形下交互对数为 $\sum_e |\mathcal{R}(e)| = a \le nm$，在假设 (b) 下取等号；每对需要成本 c 才能被正确实现，此外，对每个资源 r，n 份独立实现必须对 r 的不变量达成一致，这耗费 C_{cons}，其失败模式数目为 $\Omega(n)$（任一实现偏离即破坏全局不变量）。有基座情形下 m 个资源的语义各定义一次，每个实体通过至多 k 个接口与基座交互，固定成本 F 只付一次，即得式 (4.2)。比较两式即得式 (4.3)。 ∎

需要对式 (4.3) 的形式作出两点说明，因为那些显而易见的简化各自以不同方式出错。

第一，两边必须是成本，而不是计数。本结果早先的表述把对数 nm 直接与 $c(nk+m)+F$ 相比；这在量纲上不一致，因为左边在数交互次数，右边在为它们定价。正确的比较是在 $C_{\text{直}}$ 与 $C_{\text{基}}$ 之间，如式 (4.3) 所示。若取两个单位成本相等，$c = c_s = c$，则不等式化为

$$a + \frac{C_{\mathrm{cons}}}{c} \;>\; nk + m + \frac{F}{c}, \tag{4.4}$$

而在假设 (b) 下，此时 $a = nm$，

$$nm \;>\; nk + m + \frac{F}{c} - \frac{C_{\text{cons}}}{c}. \tag{4.5}$$

右端三项分别是基座的按实体接口开销、按资源开销，以及以一次交互为单位计的固定成本；最后一项是基座所消除的一致性成本。舍去最后一项会使式 (4.5) 偏于保守。

第二，a 不必等于 nm。假设 (b) 是最坏情形，而不是典型情形。若实体 e 只参与 $|\mathcal{R}(e)|$ 个资源，且 $\mathbb{E}_e[|\mathcal{R}(e)|]$ 不随 m 增长，则 a 按线性而非按乘积增长，基座就完全不必收回自身成本。因此，资源使用是弥散还是集中，是一个关于系统的经验问题，而不是代数推导的后果；附录 A 记录了乘积形式得以成立所要求的增长条件。

推论 4.5 (盈亏平衡阈值)**.** *由于 k 是每个实体的有界接口使用量，$\mathcal{O}(nk+m)$ 关于 n 是线性的，而 $\Theta(nm)$ 关于 n 与 m 都是线性的乘积。因此在假设 (b) 与单位成本相等的条件下，只要 n 或 m 之一显著增长，式 (4.5) 即被满足。盈亏平衡点位于*

$$c\,(a - nk - m) \;\sim\; F,$$

即当省下的交互工作量按单位交互成本计价后超过基座的固定成本时。在假设 (b) 下且低阶项被支配时，此式读作 $nm \sim F/c$，故起支配作用的量是实体数与资源数的乘积。

该推论被有意命名为盈亏平衡，而不是相变。两条成本函数相交，决定的是一种安排开始变得更便宜的那一点；而相变还额外要求系统的组织在该点的某个邻域内发生结构变化。这种变化是否发生——越过盈亏平衡点的系统是否真的长出一个基座——是一个经验问题，即假设 H_3；这里的代数给出的是使该问题变得尖锐的那个阈值，而不是问题的答案。

推论 4.5 精确化了”代码量达到某个数量级”这一直觉，并给出了它的正确形式：

原则 4.6 (阈值不是代码行数)**.** *决定是否必须建立统一基座的自变量是活跃实体数与共享资源数的乘积，以及重复问题的出现频率、状态与资源之间的交互数量、并发执行的复杂程度、模块的演化频率与对一致性、性能与可靠性的要求。代码行数只是这些量的代理变量。*

这解释了一个本该令人困惑的事实：两个各有一百万行代码的软件，内部架构可能完全不同。一个可能是大量彼此独立的生成代码、数据文件或线性业务逻辑（m 小，n 小）；另一个可能有复杂的共享状态、异步任务与大量互相依赖的子系统（n、m 都大）。只有后者会出现对统一运行时、资源管理与协调机制的需求。

核心归并（4.2）在这里给出第二个解释：当系统中活跃实体的数量从 10^2 跃迁到 10^6 时，朴素管理方式发生相变。管理 100 个对象时，条件分支与裸分配是足够的；管理 100 万个并发任务、缓存条目或连接时，实体间的潜在交互是 $O(N^2)$，复杂度不可行，于是被迫引入池化、索引、分层、异步、批量与缓存局部性，而这套手段的集合恰恰构成”内核技术”的全部清单。

例 4.7. 一个关系数据库引擎自建内存上下文（memory context）体系，是因为通用分配器在”每条查询创建并销毁数千个元组”的负载下会产生碎片且无法追踪泄漏。于是它构造了上下文树的分配与批量回收机制，这是数据库域内的对象池。一个异步运行时自建工作窃取调度器，是因为操作系统线程在百万并发面前过重，于是构造了 $M{:}N$ 调度，这是用户态的分时调度。数量级跃迁强制引入内核技术，这是第一重必然。

4.4　内核六职能

定义 4.8 (内核六职能). 对参与者-资源结构 Π，定义**内核六职能** $\mathcal{K}_6$ 为

(F1) **身份与生命周期** (identity & lifecycle)：实体的身份标识、创建、存活与终止关系；

(F2) **分配与回收** (allocation & reclamation)：有限资源的分配策略与回收策略；

(F3) **调度** (scheduling)：在就绪实体之间选择下一个执行者的选择函数；

(F4) **命名与寻址** (naming & addressing)：使实体可被发现、可被引用、可被遍历的机制；

(F5) **通信与同步** (communication & synchronisation)：实体之间的交互原语与并发转移的复合规则；

(F6) **故障隔离与恢复** (isolation & recovery)：故障的包含关系与状态的可重放性。

定理 4.9 (六职能的条件必然性). *设 $\Pi = (E, \mathcal{R}, \to, \mathrm{Ready}, \mathrm{Fault})$ 为一个参与者-资源结构，其操作语义良定义（即在每个可达配置下，下一步转移是被确定的，或其所允许的非确定性已被显式规定）。若 $|E| > 1$ 且 $\mathcal{R}$ 有限且受争用，则 $\mathcal{K}_6$ 的六项职能必然已被定义，定义位置或在该系统自身的代码中，或者在其宿主（操作系统、语言运行时、框架）中。*

结构论证. 逐项说明缺失任一职能将导致语义不完备（存在未定义行为）。

(F1) 身份与生命周期：若实体不可区分，则 Ready 无法被定义为一个关于实体集合的函数；若实体无创建与终止关系，则 E 随时间变化而未被规定，$\to$ 在” 实体出现/消失” 的步骤上无定义。

(F2) 分配与回收：$\mathcal{R}$ 有限且受争用，故” 同时请求超过可供给量的资源” 是一个可达状态。若缺少分配策略，$\to$ 在该状态下无定义。终止实体的资源若不回收，则可用资源单调递减，系统在有限步内进入资源耗尽状态；此时 $\to$ 在前瞻意义下无定义。

(F3) 调度：若 $|E| > 1$ 且多个实体同时就绪，则 $\to$ 不再是一个函数而是一个关系。若不规定就绪者之间的选择，则” 允许的非确定性” 未被界定，语义不完备。

(F4) 命名与寻址：交互原语（见 F5）必须指定交互的对象。若无引用机制，则交互的宾语不可表达。

(F5) 通信与同步：若实体之间存在交互，则全局转移不是局部转移的简单积；缺少复合规则时，并发转移的语义未被定义。

(F6) 故障隔离与恢复：$\mathrm{Fault} \neq \varnothing$（故障是物理与工程的既成事实）。若缺少包含关系，则一次局部偏离具有全局语义，等价于系统在故障下没有良定义的行为；若缺少恢复规则，则系统在首次故障后不再回到规范状态。

综上，任一职能缺失都会使” 良定义” 这一前提被破坏。　■　■

注 4.10 (定理的性质). 定理 4.9 是一个条件必然性结果，而非无条件定理。它的条件是 $|E| > 1$、$\mathcal{R}$ 有限且受争用、以及故障可达。它的证明是结构性的枚举论证，而非某个形式系统中的推导。它可被证伪的方式是：给出一个满足前提却缺少六项之一的良定义系统。该判断本身可被挑战。

注 4.11 (逃逸通道，以及如何穿过它做检验). ” 或者在该系统自身的代码中，或者在其宿主中” 这一措辞，既使该定理可用，也使它容易被不诚实地辩护。粗心读来它会变得不可证伪：

任何缺少六项之一的系统都被说成是从其宿主那里获得了该职能，而宿主总在某处，于是该主张不可能失败。这一措辞不是一个漏洞——定理的内容是该职能在语义中的某处被定义，这是关于语义的主张，而不是关于文件所有权的主张——但它确实转移了反驳的负担，因此值得说明这一负担如今落在何处。

一次反驳必须给出一个系统与一个职能，使得 (i) 操作语义在定理 4.9 的意义下良定义，(ii) 前提成立，且 (iii) 该职能既从该系统、也从它运行的每一个宿主中缺席。第 (iii) 条是难以满足的那一条，而它之所以难，原因是有信息量的而非回避性的：有名字的宿主在相关职能上是稠密的。操作系统定义了调度、分配与隔离；语言运行时定义了身份与生命周期，且往往还有通信；浏览器则定义了全部六项。因此一次反驳必须指出一个其整个执行环境都缺少该职能的系统，这在实践上意味着：要么是一个完全没有宿主的系统——裸机加静态分配，而这已被定理 4.9 的前提排除——要么是一个其宿主已被证明缺少该职能的系统，而后者是关于那个宿主的、可核查的主张。

由此得出两条关于该主张应当如何使用的后果。第一，该定理最好被读作一份清单式义务，而不是一项关于世界的发现：对六项中的每一项，说明它在何处被定义，若答案是” 宿主”，则说明是哪个宿主、依据什么证据。一份填不满某一行的层级清单并不因此被反驳；它只是因此不完整，而这是一个可用的状态。第二，该定理并未确立的三条性质应当被分离出来、当作假设处理，而不是折进主张之中，因为它们是可独立检验、也可独立置疑的：

(a) **充分性**：六项已经足够，即语义良定义不需要六项之外的任何东西。这正是猜想 4.21 从粗粒化一侧处理、并由假设 H_{10} 检验的主张。

(b) **独立性**：六项之中没有哪两项可以用其余各项定义。该清单的动机来自彼此不同的语义义务——可区分性、有限性与争用、就绪集合的基数、交互的宾语、复合的方式，以及故障的传播范围——但义务的彼此不同并不等于实现的彼此独立，系统经常同时实现其中若干项（一张能力表同时是命名、分配与隔离）。

(c) **最小性**：六项无法再被约简。论证表明去掉任意一项都会在前提之下破坏良定义；它并未表明六项在所有分解中最小，只表明这一分解相对于其自身前提是冗余自由的。

消融实验是检验这三条的自然工具：对每个职能，把它从一个满足前提的系统中移除，观察语义是否真的变得不良定义，以及余下的职能能否吸收它的义务。该实验尚未被做过。

定理 4.9 的措辞” 或者在该系统自身的代码中，或者在其宿主中”，是关键性的。它把” 职能是否必然出现” 与” 职能是否必须自建” 两个问题分离开来。

4.5　自建与外借

推论 4.12 (自建/外借二分)**.** *由定理 4.9，六职能必被定义，但定义的位置是一个设计选择。该选择由不变量的领域关键性决定：*

(a) 若某职能所承载的不变量是领域关键的，即该不变量必须由本系统拥有的保证才能成立，则该职能必须自建；

(b) 若某职能所承载的不变量是通用的，即通用宿主能够提供强度足够的保证，则应当外借；

(c) 对通用职能自建实现，将产生本文称为通用件重造的失败模式：以更高的长期理解与维护成本，重新实现一个更差的通用机制。

这一判据解释了”应用”与”平台”的边界为什么正好落在它现在的位置：*你被迫自己拥有的不变量集合，就是你内核的边界。*

例 4.13. 一个游戏引擎必须自建实体-组件存储与帧调度，因为通用宿主无法提供”亚毫秒确定性帧预算”这一保证。一个数据库必须自建存储引擎与预写日志，因为通用文件系统无法提供”崩溃后恢复到确定状态”这一保证。反之，一个 Web 应用应当外借垃圾回收、线程调度与内存分配，因为这些职能的不变量是通用的，自建只会得到更差的版本。

注 4.14 (与企业理论的平行，以及它在哪里停止). 推论 4.12 回答的问题，在软件之外有一个发展成熟的对应物：一个组织应当*自建什么，又应当外购什么*？科斯的回答是：企业的边界划在”在市场中交易的成本超过在内部组织同一活动的成本”之处 [75]，而后续关于自建/外购决策的文献把这一点转化为治理成本的可操作比较 [76]。这一结构上的平行足够接近，值得明确说出：在两种情形中，边界都不是关于工作的技术事实，而是哪些成本落在边界的哪一侧的后果。

这里的判据在一个方面更锐利、在另一个方面更狭窄，而这一差别正是引用该平行而非依赖它的理由。更锐利：科斯的比较是在两种成本之间，而这两种成本出了名地难以测量；而推论 4.12 先问一个答案确定的定性问题——该不变量是否是只有本系统才能拥有的那一个？——然后才考虑成本。一个拥有领域关键不变量的企业没有自建/外购决策可做，无论市场替代品多么便宜。更狭窄：该推论没有涉及若干系统各自拥有某不变量的一部分、或所有权可以按契约分割的情形，而那正是经济学文献大部分内容所在之处。

这一平行带来的不是结论，而是一个警告。如果软件边界像企业边界那样由治理成本划定，那么它会表现出同样的病理：在理由消失之后仍然存续的边界；出于控制而非成本的理由被选中的集成；以及地位受边界自身的转换成本、而非所供之物质量保护的供应商。这些在软件中是可以观察到的，而一个声称解释该边界的框架应当能预测它们。本文没有做到这一点，这是一个缺口，而不是一项主张。

注 4.15 (一个更锐利的判据：单调性). 推论 4.12 用不变量是否领域关键来判定自建还是外借。对”协调”这一类职能，存在一个更锐利且可静态检查的判据：*计算是否单调*。在单调性成立时 [69]（输出随输入增长而增长，且不依赖否定或聚合的全局知识），协调不是必需的，可以下推到各参与者自行完成；非单调性（唯一性约束、否定、阈值聚合、非交换更新）则强制某个位置承担协调。

这把第 4.4 节的职能 (F5)（通信与同步）与 (F3)（调度）的一部分从”规模到了就必须集中”精确化为”非单调就必须集中”。对单调子系统，集成成本定理（定理 4.4）中的 k 可以被降到接近常数，因为参与者之间不需要就共享状态达成一致。

4.6　跨领域对照

定理 4.9 预测：横看每一列是一家不同公司、不同语言、不同时代、不同领域的系统；纵看每一行是同一个器官。表 4.1 给出对照。该表不主张这些机制可以一一等同，而是把它们放在相同的*管理职责*维度上比较。

表 4.1：同一组管理职责在不同领域的独立实现

职能	Linux	Chromium	Tokio	Bevy	Unreal	PostgreSQL
身份/Lifecycle	task_struct, inode, file	Frame, Document, Node	Task, JoinHandle	Entity, Component	UObject, Actor	backend, xid, tuple
分配/回收	buddy, slab, page cache	PartitionAlloc, Oilpan	slab 式任务分配	archetype 列存	FMemory, GC	MemoryContext, shared_buffers
调度	EEVDF, workqueue	task queue, RendererScheduler	work-stealing, waker	Schedule/System 并行执行器	TaskGraph, GameThread	postmaster, autovacuum
命名/寻址	kobject, sysfs, fd	DOM, Web IDL	Handle, waker	Entity ID	FName, 反射元数据	OID, 系统目录
通信/同步	syscall, netlink, IPC	Mojo IPC	mpsc, oneshot	Event, Resource	委托, 复制	libpq, LWLock, 扩展钩子
隔离/恢复	namespace, cgroup, OOM	site isolation, 渲染进程沙箱	panic 传播, 取消	状态机隔离	crash reporter	MVCC 快照, WAL 重放

反过来说，这六个器官在不同宿主中呈现的形态差异极大，而这正是第三章” 主干趋同、枝叶各异” 的具体内容。形态差异的根源可以被系统地列出：

(a) **宿主内存模型**。手工内存管理的系统（C/C++/Rust）必须自建分配器甚至自建回收器（引擎对象回收、浏览器 Blink 的垃圾回收）；具备托管内存的系统把同样的精力花在停顿行为上，因为分配是现成的，痛点变成了延迟尾部分布。

(b) **宿主并发模型**。运行时模型决定调度器官的形状：游戏引擎是帧驱动的协作式调度（可预测延迟优先于吞吐），服务器是事件驱动的抢占式调度（吞吐优先）。因此帧调度器与工作窃取调度器是同一个器官的两个亚种。

(c) **领域原语粒度**。通用内核给出的是页、线程、文件描述符、事件轮询；但游戏引擎需要帧、组件与资产，数据库需要缓冲池、事务与版本链，浏览器需要文档节点、安全上下文与跨进程对象。原语粒度错配逼着每个领域自建基础设施。

(d) **演化阶段**。这些内部操作系统本身也在持续被重构，且每次重构都是朝着” 更完整的六职能” 收敛而非发散。

连操作系统自己也会被其用户绕过：高性能网络与存储栈在用户态轮询重做 I/O，异步 I/O 接口被重新设计，连操作系统都逃不过被自己的用户态” 重新发明” 的命运。这不是反常，而是推论 4.5 的直接结果：当用户态的 $n \times m$ 越过阈值，用户态就会长出新的内核。

4.7　传输与重新导出

表 4.1 显示同一组职能在不同领域中反复出现。但” 反复出现” 有两种截然不同的解释，它们的预测完全不同：

(a) **同构**：六职能是同一个结构的六次实现，可以彼此翻译；

(b) **沿基底的传输**：六职能是某个更一般结构在不同问题形状上的纤维，形状改变时结构必须重新导出，不能搬运。

原文只讲了 (a)。区分二者需要显式引入” 问题形状” 这个基底。

定义 4.16 (问题形状与层族)**.** 称一个二元组 $\pi : \mathcal{L} \to \mathcal{P}$ 为一个**层族**，其中

- $\mathcal{P}$ 是**问题形状**的集合：由该领域的资源形态、并发结构、失效模式与时间尺度决定；
- $\mathcal{L}$ 是层实现的集合；
- 纤维 $\pi^{-1}(p)$ 是在问题形状 p 下可行的层实现族。

π 把" 某个实现" 映到" 它解决的是什么形状的问题"。π 的重索引给出**沿基底的传输**：给定 $p \to p'$，把 p 上的实现搬到 p' 上。

命题 4.17 (传输的保持范围)**.** *设 $p \to p'$ 是问题形状之间的映射。若该映射是等价（相同的资源形态、并发结构、失效模式与时间尺度），则 $\mathcal{K}_6$ 的槽位（每个职能所必须回答的问题）沿传输保持，实现形态可以改变但不增加或减少槽位。若该映射不是等价，则至少一个槽位的语义要求发生改变，实现必须重新导出。*

论证. $\mathcal{K}_6$ 的每一项由定理 4.9 的语义义务唯一确定：身份由实体的可区分性确定，分配由资源的有限性与争用确定，调度由就绪集合的基数确定，命名由交互的宾语确定，通信由并发转移的复合方式确定，隔离由故障的传播范围确定。这六项义务只依赖问题形状的四个分量。当映射保持这四个分量时，义务集合不变；当任一分量改变时，对应义务的语义要求随之改变。 ■

推论 4.18 (一个可观测的判别)**.** *由命题 4.17，形态差异可以分成两类：同一槽位在不同基底上的实现差异（例如帧驱动与事件驱动的调度器，基底的时间尺度不同），与不同槽位的差异。前者不构成反例，后者构成。*

推论 4.18 使第三章的中命题（假设 3.2）可以被更严格地检验：比较两个系统时，必须先在基底上对齐，再比较槽位。在基底上不对齐而直接比较模块，是既有架构比较研究最常犯的错误。

注 4.19 (这一节的地位)*.* 定义 4.16 与命题 4.17 是一份结构提案，不是关于范畴论的既有定理的应用。它们的作用是把" 同构还是传输" 这个此前未被提出的问题显式化，并给出一个可判别的推论（推论 4.18）。把 π 提升为严格的纤维化并证明重索引函子的性质，是尚未完成的工作（第十一章 O 项）。

4.8　粗粒化判据

定理 4.9 论证了六职能必然存在，但没有说明它们是否是最显著的结构。一个更强的命题是：六职能是系统结构在粗粒化下的存活集。

定义 4.20 (粗粒化序列)**.** 设系统的架构关系型为 $\mathcal{G}$（定义 2.15）。一个**粗粒化**是一个商映射 $q : \mathcal{G} \to q(\mathcal{G})$，它合并构件并保留关系类型。称一列商映射 $q_1, q_2, \ldots$（每一步的粒度严格变粗）为一个**粗粒化序列**。结构 $\mathcal{S}$ 在该序列下**存活**，若存在 N 使得对所有 $n \geq N$，$\mathcal{S}$ 在 $q_n(\mathcal{G})$ 中仍可识别。

猜想 4.21 (六职能是相关结构)**.** *在一个正常偏置的大型系统的粗粒化序列下，存活的结构恰为 $\mathcal{K}_6$ 的槽位及其关系型；其余结构（具体模块、算法选择、数据布局细节）在足够粗的粒度上被冲掉。*

猜想 4.21 的价值在于它给出一个单项目内的证伪程序：不需要跨项目统计比较，只需对一个代码库反复粗粒化，观察存活集。若存活集不是六职能，猜想被否定，且第三章的收敛论题需要一个不同的解释。

原则 4.22 (实施形式)**.** 粗粒化可以完全机械地实施：以模块依赖图为基础，反复施加三类操作：(i) 合并扇入扇出低于阈值的节点；(ii) 折叠可传递的依赖边；(iii) 抹去不携带不变量（定义 6.2 中 $\mathrm{S}(\iota) = \varnothing$）的节点。每一步记录存活结构的变化。整个实验只需一个仓库与一个依赖图，不需要标签、不需要人工分类，因此不受第 3.7 节指出的” 职责分类系统选择自由度” 影响。

第九章把猜想 4.21 写成假设 H_{10}。

注 4.23 (与重整化的关系)**.** 原则 4.22 的操作与重整化群的粗粒化在形式上相同：消去短程自由度，观察长程结构是否收敛到不动点。这里的不动点就是六职能槽位。必须明确的是，这不意味着软件具有重整化群流，因为软件没有尺度分离（原则 4.24 之前的讨论），因而不存在真正意义上无关算符的指数衰减。相同的是操作，不是机制。

4.9　虚拟机化与深模块

” 压缩复杂度、为上层提供高级语言与符号” 这一表述可以精确化。每个基础设施层的核心工作都是同一种动作：

原则 4.24 (虚拟机化)**.** 基础设施层把物理的、连续的、易变的东西，翻译成离散的、稳定的、可组合的领域级符号。

例 4.25. 操作系统：物理内存 $\to$ 虚拟地址空间；块设备 $\to$ 文件；硬件中断 $\to$ 信号。数据库：物理页 $\to$ 多版本元组 $\to$ SQL 中的” 行”。浏览器：字节流 $\to$ 文档树；套接字事件 $\to$ 事件循环回调。游戏引擎：内存中的裸字节 $\to$ 组件与资源；一帧的时间 $\to$ 调度表。异步运行时：事件轮询位图 $\to$ `async/await` 中的 Future。

用模块设计的术语 [49, 53]，这些全是深模块：接口极小，实现极深。一行分配调用背后是整个分配器体系；一行任务创建背后是整个调度器。基础设施层的职责就是持续生产深模块，把复杂度封在下面，把干净的符号暴露在上面。

这一点把它与第五章的理解理论直接连接起来：深模块正是压缩率（定义 2.14）极高的层级，它商掉了大量底层差异，同时保持了上层所需的预测能力。这也解释了为什么好的基础设施同时是” 高性能的” 与” 易理解的”：两者都是同一个商映射的后果。

4.10　智能体系统的职能分化

如果定理 4.9 成立，那么最新出现的复杂计算域应当正在重演这一演化。表 4.2 给出观察。

这一观察的价值在于它是一个自然实验：没有人在刻意模仿内核，是复杂度本身把同样的解剖结构压了出来。它同时构成假设 H_3 的经验基础（第九章）。

表 4.2: 智能体系统对六职能的重新发现

职能	在智能体域中的对应物
分配与回收	上下文窗口作为快速存储；检索增强作为缺页处理；外置记忆作为交换区；上下文压缩作为内存紧缩
通信与同步	工具调用或函数调用作为系统调用原语
调度	智能体主循环、多智能体编排
身份与生命周期	会话、任务与子目标的生命周期管理
命名与寻址	工具注册与发现协议
隔离与恢复	沙箱、重试、检查点与回滚

定理 4.4 与 4.9 在给定前提下成立，其可挑战之处在于前提与结构论证的严密性。经验上可证伪的是：六职能的出现是否由 $n \times m$ 而非代码行数预测（假设 H_3 与 H_{10}）；以及是否存在满足前提却长期缺少某一职能的良定义系统。

第五章　理解：逆向压缩

本章处理问题 1.3 的前半：面对无法从微观通读的大型系统，应当从什么层级理解，以及如何由理解进入开发。核心命题是理解即逆向压缩，其最优深度由率失真驻点条件给出，其可行性由观测秩决定。

5.1　顺序阅读的失效

一个自然的阅读方式是追随源代码的物理顺序与控制流：

$$\texttt{main()} \to \texttt{module_a} \to \texttt{module_b} \to \texttt{function_c} \to \texttt{helper_d} \to \texttt{allocator} \to \texttt{operating_system} \to \cdots$$

然后不断向下追踪，最终陷入无限递归。其失败不是耐心不足，而是结构性的：

原则 5.1 (调用关系不等于认知依赖)**.** 函数调用关系是实现的拓扑；认知依赖关系是理解的拓扑。二者不同构。研究事务隔离问题时，你可能需要知道事务管理器如何使用锁与快照，但不一定需要理解操作系统整个虚拟内存子系统；反过来，排查内存使用异常时，事务管理器的高层业务语义可能暂时不重要。

更根本的原因是生成方向与理解方向的关系。系统是被自顶向下设计的（先有意图，再有架构，最后才有代码展开）；因此逆向过程应当取同方向。自底向上读代码，等于通过清点所有分子去理解一只猫：原则上可行，实践上荒谬。

而” 原则上可行” 这一让步有实际后果：顺序阅读不是不可能，而是被占优。命题 2.20 给出其精确形式：越过驻点后，描述长度增加而任务损失不再下降。

5.2　认知图

面对一个大型单体仓库，不应先建立文件地图，而应先建立概念地图。二者的分工是：

- 文件地图回答” 代码在哪里”；
- 概念地图回答” 系统由什么组成、为什么存在这些部分、谁依赖谁、什么规则使它们能够协作”。

定义 5.2 (四张认知图)**.** 对一个系统 S 与任务 T，四张认知图分别是：

(i) **系统全景图**：把 S 压缩为 5–15 个主要职责域，并回答：最重要的职责域是什么？每个职责域为什么存在？它们之间最重要的依赖关系是什么？哪些机制被多个职责域共享？哪些状态与行为决定整个系统的正确性？

(ii) **关键子系统图**：对每个关键子系统回答八个问题：职责、边界（负责什么与不负责什么）、状态（拥有什么状态、谁有权修改）、机制、接口、约束、失败行为、实现入口（哪些源码与测试可以验证上述结论）。

(iii) **关键执行路径图**：把静态架构转换为动态行为模型。例如一条查询路径” 输入 → 解析 → 规划 → 执行 → 访问存储 → 返回” 背后的真实问题：查询计划何时创建？执行期间状态存放在哪里？事务快照由谁管理？存储访问失败时错误如何向上传递？哪些步骤会持有锁？哪些操作可能触发磁盘 I/O？查询取消时哪些资源必须释放？

(iv) **契约图**：按定义 2.15 的五类边分别记录构件之间的关系，特别是每层的*无视集*，即该层被允许永远不关心的内容。

第 (iv) 张图是本文相对于常规架构文档的补充。它的必要性直接来自命题 2.9：一个没有声明无视集的契约，等于没有声明它的压缩边界，从而无法判断上层可以依赖它到什么程度。

5.3　理解的操作判据

” 理解” 必须被操作化，否则无法判断是否已经理解。本文采用五能力判据。

定义 5.3 (理解判据)**.** 称一个智能体（人或机器）**理解**了子系统 M 到任务 T 所需的程度，当且仅当它能够做到：

(U1) **解释**：说清这个子系统为什么存在；

(U2) **预测**：预测某个输入、状态变化或错误会导致什么行为；

(U3) **定位**：出现故障时判断应该检查哪些组件；

(U4) **修改**：要添加功能时找到正确的扩展点与必须遵守的约束；

(U5) **验证**：知道哪些测试或实现细节能够证明自己的判断。

本文称达到 (U1)–(U5) 的状态为**可操作的抽象理解**。

定义 5.3 与定义 2.18 的关系是直接的：(U2) 是 T-充分性的行为化表述；(U1)、(U3)、(U4) 是 T 中具体查询族；(U5) 是” 能够追溯到更底层证据” 的要求，即定义 2.19 中” 足够准确” 与” 能够验证” 两条。

一个重要的诊断推论：如果只能复述架构图，却无法预测一个改动会影响什么，那么当前的抽象还不够。这与第 5.5 节的缝探测器是同一件事的两个方向。

5.4　阅读顺序

原则 5.4 (下行协议)**.** 自顶向下形成模型，按需下钻验证，永远不要顺序阅读。

具体步骤如下。

学科层。　读 README、架构文档、架构决策记录、设计论文、内核文档目录。用约 1% 的时间建立约 80% 的骨架模型。产出是一张手绘的器官图：系统有哪几个大块、各自职责、数据如何流动。

运行观测。　先跑起来，再读。运行时的行为暴露结构，比静态代码诚实得多：观察进程与线程模型；打断点或跟踪一次典型请求的完整旅程；看日志与指标，日志的词汇表就是系统的自述。一次端到端的跟踪，抵得上一万行静态阅读。

接口边界。　按依赖顺序看模块的头文件、公共接口与导出符号，而不是实现。接口是压缩边界（命题 2.9），它告诉你这一层承诺什么；至于它如何兑现承诺，暂时不需要。此刻要的是器官图谱，不是组织切片。

数据形状。　数据模型是最稳定的抽象，控制流最易变。先弄清楚：核心实体是什么、生命周期由谁拥有、状态机有哪些、谁可写谁只读。握住数据的形状之后，控制流只是数据随时间的重排。

按需下钻。　只带着具体任务下钻到代码：修这个缺陷、加这个功能。此时阅读是有指向的检索而非漫游。下钻遵循" 调用点 → 实现 → 底层机制" 的 V 字形：能解释当前问题即收手，然后回到高层重估模型。

预测与验证。　在每一层，先写下心智模型（" 我猜执行器是这样取下一条元组的")，然后去验证。能预测行为就是理解。预测对了，压缩模型有效；预测错了，那个位置恰好就是系统最反直觉之处。

回写契约图。　把新获得的知识写回定义 5.2 的契约图，填入该层的新增保证、新增假设、以及新增的无视集。这一步是防止认知漂移的唯一手段：若不回写，分层模型会逐渐脱离真实系统。

注 5.5 (一个可操作的比例). 深入理解约 10% 的代码，同时知道另外 90% 在哪里、归谁管。这个比例不是精确的目标，而是对" 理解不等于穷尽" 这一原则的数量化提醒。

5.5　模块边界的探测判据

架构就是" 改动被局部化的那些地方"。因此理解的关键操作是找缝。以下判据按可靠性排序，均可机械执行或半机械执行。

(S1) **类型停止穿越处**。数据在哪里被重新编码、包装或降级为通用类型（如字节数组、字符串、映射）？这是模块边界的强信号：类型系统停止提供保证的地方，就是契约必须由人补上的地方。

(S2) **可隔离测试的单位**。哪里存在一个可以在不启动整个系统的前提下跑完的测试套件？这是最锋利的判据：能被隔离验证的单位才是真模块。它的可靠性来自定义 2.18：一个单元测试本质上是在断言" 接口观察是行为的充分统计量"。

(S3) **序列化、版本号与拷贝边界**。哪里有版本字段、序列化、深拷贝、跨进程或跨网络调用？跨越这类边界使契约变得可见且可证伪。

(S4) **不变量的集中强制点**。哪里有断言、数据库约束、类型级约束、契约检查？不变量被集中强制的地方通常就是层级边界的所在。

(S5) **构建系统的独立目标**。构建配置里哪里存在独立的编译目标或包？这是边界的必要而非充分条件：独立目标未必是真模块，但真模块几乎总有独立目标。

(S6) **共同变更图与事故史**。这是最被低估的技术。考察历史提交记录：哪些文件倾向于在同一个变更中被一起修改？共同变更图给出的是语义耦合，而导入图给出的是语法耦合。二者不一致的地方，正是架构文档与现实不符的地方。事故与缺陷历史则给出真实的不变量缺口。

原则 5.6 (缝的判据总结)**.** 一个模块是真的，当且仅当它的接口观察是其外部行为的充分统计量；一个边界是真的，当且仅当跨过它之后，另一侧的变化不再需要被理解。

第六章将把原则 5.6 反过来用作设计目标。

5.6　主动辨识与被动阅读

原则 5.7 (理解即系统辨识)**.** 理解一个复杂系统的本质是*系统辨识*：构造一个能预测其行为的模型，并量化模型的不确定性。系统辨识的最高效方式是”扰动–观察”循环。

被动阅读是一种极差的采样方式，因为它没有误差信号。阅读产生的每一个判断都无法被证伪，因此无法被更新。主动探测则相反：先形成一个可证伪的预测（”如果我改这里，应该只有这两个模块受影响”），做一个最小改动或跑一次测试，然后观察预测误差。误差就是你看不见的那部分结构。

命题 5.8 (主动探测的优势)**.** 设理解过程是模型 M 在一个假设类 $\mathcal{H}$ 上的迭代更新，并称一次设计 d 为一次观测方式的选择，取自一个设计空间 $\mathcal{D}$，而 $\mathcal{D}$ 包含被动阅读可用的那些设计。记 $I(\mathcal{H}; o \mid d, M)$ 为设计 d 在当前模型下关于该假设的期望信息增益。则：

(a) 最优设计的表现不劣于任何被动设计，因为被动设计都在 $\mathcal{D}$ 中；

(b) 最优设计严格更优，当且仅当 $\mathcal{D}$ 中存在一个设计，能在当前知识状态下分离相互竞争的假设——即当且仅当可用的设计对所涉区分是可辨识的。

论证. (a) 是直接的：在一个包含全部被动设计的集合上取最大，不可能比该集合中的任何成员更差。对 (b)，若 $\mathcal{D}$ 中每个设计对两个在 M 下仍然存活的假设 $h \neq h'$ 都诱导出相同的观测似然，则没有任何设计能区分它们，期望信息增益对所有设计都相同，无论主动还是被动；反之，若某个设计能区分它们，选择该设计就得到严格为正的增益，而无信息的设计得到零。 ■

这一改写不是修辞性的，而它所移除的两样东西都是过度主张。第一，它不再断言期望信息增益与预测误差 $\mathbb{E}|y - \hat{y}(M)|$ 成正比：这样的正比关系一般并不成立。一次探测可以很容易挑起很大的预测误差——一个模型处理得很差的异常输入——却对分离真正在竞争的假设毫无作用；而一次平均误差很小的探测，却可以大幅降低关于某个要紧参数的不确定性。支配增益的是该设计是否具有区分力，而不是当前模型错得有多厉害。第二，它把该优势条件化于设计空间的一项可辨识性质，而这是一条可能失败的前提：若智能体只能执行所有假设都预测相同的观测，那么无论探测多么卖力都买不到任何东西。这与 §5.7 的观测秩亏是同一个失败，只是从设计一侧而非观测一侧抵达。

保持不变的是那个实用主张，而它正是本节其余部分所需要的：只要存在可负担的、具有区分力的设计，主动探测就是更好的工具，且其优势随需要分离的存活假设数目而增长。对 AI 而言该结论的分量比对一个人类读者更重：AI 的优势不在于能”读”更多代码，而在于能高吞吐地做探测，生成假设、跑测试、做最小改动、看共同变更、读事故记录，然后更新模型。这是人类难以匹敌的频率。

该主张可检验的形式相应地是一项研究设计而不是一条定理。列出候选模型及其不确定性；选择能区分相互竞争假设的操作；测量期望信息增益、实验成本与模型的实际改进；并与被动阅读、随机探测和常规调试相比较。要被检验的主张是：在同等成本下，有意的区分胜过无指向的观察——而不是信息增益与预测误差成正比。

5.7　可观测性判据

第 2.4 节把理解定义为任务约束的压缩，命题 2.20 给出下潜的停止点。这两者都预设了”继续观测能改进模型”。本节指出一个更强的限制：*观测不足时，增加观测量的数量不改变任何东西。*

定义 5.9 (观测算子与任务子空间)**.** 设 $\mathcal{B}$ 为系统行为空间，$\mathcal{T} \subseteq \mathcal{B}$ 为*任务相关差异子空间*，即定义 2.17 中查询族 Q 能够区分的差异所张成的子空间。设 $\mathcal{J} : \mathcal{B} \to \mathcal{Y}$ 为智能体可获得的观测算子：它把行为映射到它实际能读取的东西（源码、日志、跟踪、测试结果、运行时指标）。定义 $\mathcal{T}$ 上的**观测秩**

$$\operatorname{rank}_{\mathcal{T}} \mathcal{J} \;=\; \dim \mathcal{J}\big(\mathcal{T}\big), \tag{5.1}$$

即任务相关差异经过 $\mathcal{J}$ 之后仍然可区分的维数。当行为空间与观测空间都是有限维线性空间、且 $\mathcal{J}$ 为线性时，式 (5.1) 特化为线性系统理论中的观测秩 [72]；下面的一般情形并不预设这一结构。

命题 5.10 (可观测性判据)**.** *智能体能够把系统辨识到 T-充分（定义 2.18），当且仅当观测算子对任务是可辨识的：对所有在任务相关子空间上不同的行为 b, b'，*

$$\mathcal{J}(b) = \mathcal{J}(b') \implies q(b) = q(b') \quad \text{对每个查询} q \in Q. \tag{5.2}$$

当式 (5.2) 不成立时，存在两次执行 $e_1 \neq e_2$，其全部可获得观测相同而在任务相关行为上不同；此时，任何数量的同类额外观测都不能降低任务损失，因为新增观测仍在同一 $\mathcal{J}$ 的值域内。

证明. 若式 (5.2) 成立，每个查询的取值都是观测的函数，故由观测建立的模型对每个查询的回答与系统相同，由定义 2.18 即得 T-充分性。若不成立，则存在 $e_1 \neq e_2$ 使 $\mathcal{J}(e_1) = \mathcal{J}(e_2)$，且存在某个 q 使 $q(e_1) \neq q(e_2)$；任何 $\mathcal{J}$-可测的判决规则对二者给出相同答案，因此至少在其中一个上错误。追加同类观测不改变 $\mathcal{J}$，故不能区分二者。 ■

判据的秩形式是式 (5.2) 在下列情形下的特化：$\mathcal{T}$ 是有限维向量空间，$\mathcal{J}$ 为线性，且 dim 就是维数；此时 $\mathcal{J}(b) = \mathcal{J}(b')$ 恰好等价于 $b - b' \in \ker \mathcal{J}$，而式 (5.2) 约化为 $\operatorname{rank}_{\mathcal{T}} \mathcal{J} = \dim \mathcal{T}$。把一般判据表述为秩条件，对软件而言是一个范畴错误：轨迹、日志、测试结果与源码观察并

不自然地构成向量空间，也没有维数可算。保留秩的表述，是因为它在适用之处有用——在有限状态模型中、在资源行为的线性抽象中，以及作为一种关于”观测装置能分辨多少个独立方向”的直觉——而不是因为它就是一般情形。

还有一处区分属于这里，因为本节早先的一种表述把它弄模糊了。式 (5.2) 的失败是观测机制的性质，而不是样本的性质：不能分离两种行为的某一类数据，无论多少数量都不能分离它们。这与”对一种能够分离它们、只是采集得太少的数据采集得太少”不同；在后一种情形中更多数据确实有用，而相关的界是备注 5.12 的样本复杂度。只有前一种情形才许可”继续阅读是徒劳的”这一结论。

命题 5.10 把”理解不足”分成两种在工程上完全不同的情形：

原则 5.11 (两种理解失败的区分)**.** (a) **秩亏**：观测算子本身不足以区分任务相关差异。此情形的唯一疗法是*改变观测*，增加不变量检查、增加跟踪点、增加断言、暴露内部状态。这是设计动作，不是阅读动作。

(b) **秩满但样本不足**：观测足以区分，但尚未采集。此情形的疗法是*增加探测*，由命题 5.8 支配其效率。

把二者混淆是”读更多代码”这类建议失效的主要原因：当问题属于 (a) 时，增加阅读量在信息论上无效。

注 5.12 (理解成本的正确计量)**.** 情形 (b) 的成本是一个*样本复杂度*，而不是阅读量。若把契约假设限定在一个假设类 $\mathcal{H}$（定义 2.27 中的运算集合提供了一个自然的候选），则要以误差 ε 辨识到 T-充分，所需探测次数的下界由 $\mathcal{H}$ 在 T 上的区分能力决定，典型形式为 $\Omega(d/\varepsilon)$，其中 d 为该假设类的容量维数。

这一计量方式给出一个可检验的预测：*设计探测优于穷尽阅读的倍数是可量化的*，且该倍数随假设类容量的增长而增长。第九章的假设 H_5 由此获得一个定量形式。

例 5.13 (秩亏的典型形态)**.** 一个服务把全部状态保存在进程内存中，对外只暴露请求的返回值。若任务是判断”并发的两个请求是否可能读到彼此的部分更新”，则任何数量的返回值观测都不足以判定，因为该差异完全落在返回值之外。此时正确动作不是读更多代码，而是引入一个可观测量：一个并发不变量的运行时断言，或一次带交错注入的测试。此例说明原则 5.11 的区分是实际的而不只是分类学的。

5.8　预算与停止条件

第 2.4 节已经给出停止准则的形式（命题 2.20）。此处给出它的工程化版本与预算分配。

原则 5.14 (下潜预算分配)**.** 把理解预算按”信息增益/成本”排序分配，而不是按代码量或目录顺序分配。典型的高收益入口依次是：对外接口定义与数据模型 > 端到端执行路径的一次跟踪 > 不变量强制点 > 共同变更热点 > 具体实现细节。低收益入口是：顺序阅读实现体、通读工具函数库、通读生成代码。

原则 5.15 (停止条件)**.** 满足下列任一条件即停止下潜：

(a) 你已能命名管辖当前改动的全部不变量；

(b) 继续下潜不再改变你对”这个改动会影响什么”的预测；
(c) 下一步的证据获取成本超过了本次改动的预期返工成本。

条件 (a) 对应原则 2.21；条件 (b) 对应命题 2.20；条件 (c) 是经济性判断：理解不是越多越好，而是够用即可。

5.9　机器理解的推论

AI 面临的困难不只是上下文长度有限。即使模型可以读取大量代码，仍然存在三个不同的问题：信息太多使重要信息被淹没；局部信息之间存在复杂依赖；对细节的理解不一定能自动形成正确的系统级模型。因此仅仅扩大上下文窗口并不能解决全部问题。

原则 5.16 (AI 的软件认知架构)**.** 一个有效的机器认知架构应当是四层的：
(i) **全局认知模型**：系统目标、架构图、关键不变量、主要数据流、核心设计决策；
(ii) **子系统认知模型**：组件职责、接口、状态所有权、依赖关系、生命周期与错误语义；
(iii) **实现级索引**：关键函数、类型定义、调用链、源码位置、测试与运行时证据；
(iv) **验证与反馈**：编译、测试、静态分析、执行跟踪、性能剖析，用于修正上层模型。
其关键性质是：*上层模型不需要复制所有下层信息，只需要保留足够的信息，使智能体在需要时能够找到正确的下层证据。*

定义 5.17 (可追溯的子系统认知记录)**.** 每个子系统的认知记录至少应包含九个字段，见表 5.1。

表 5.1：子系统认知记录的最小字段集

字段	作用
Purpose	这个子系统为什么存在
Contract	对外提供什么保证（即 $\mathcal{O}_i(\mathcal{B}_{i+1})$）
Oblivious	该层被允许永远不关心什么（即 $\mathrm{Obl}(\Lambda_i)$）
State	拥有什么状态，谁有权修改
Dependencies	依赖哪些其他组件
Invariants	哪些条件绝不能被破坏，在何处强制
Lifecycle	关键对象如何创建、运行与销毁
Execution paths	重要请求或任务如何通过系统
Evidence	哪些源码、测试或实验支持上述结论
Uncertainty	哪些内容尚未验证，哪些假设可能错误

表 5.1 中最后两项与 Oblivious 一项是本文的补充，理由如下。

关于 *Uncertainty*：AI 不能把自己生成的一份架构总结当作事实。它需要知道哪些结论来自实际源码，哪些来自测试，哪些只是暂时的推断。这与第十一章的认识论纪律是同一条要求。

关于 *Oblivious*：如上所述，它是压缩边界本身。没有这一项，认知记录只描述了”有什么”，没有描述”可以不关心什么”，因而无法支撑 (U4) 修改能力。

关于动态更新：当代码发生变化时，认知记录必须同步更新。因此有用的机器理解系统库系统不应只是静态知识库，而应是源码与抽象模型之间*可双向追溯*的关系。

原则 5.18 (可读性从美德变为基础设施)**.** 系统的”理解接口”正在成为一等工程产物。过去文档是给人看的附属品；现在机器可读的架构描述、接口即文档的实践、以及面向智能体的工作规范，正在成为系统必须设计的组件，因为代码的读者中多了一个非人类的智能，而它只依赖压缩层。

本章的经验内容是假设 H_5（第九章）：以分层模型加主动探测为策略的理解过程，在单位成本上的任务成功率优于穷尽式阅读。命题 2.20、5.8 与 5.10 在给定模型下成立；定义 5.3 是一个工作定义，其价值取决于它能否被用于区分”理解”与”复述”，这需要实验检验。

第六章　设计：正向压缩

如果理解是逆向压缩，设计就是正向压缩：为未来的读者预先造好压缩边界。本章把这一对称性转化为可优化的目标函数，给出其动力学约束，并说明分层在何处失效。

6.1　读集与目标函数

架构的可测量目标不是” 解耦”” 高内聚” 这些形容词，而是一个可以被统计出来的数字。

定义 6.1 (变更读集)**.** 设代码库由模块集合 $\mathcal{M}$ 组成，即 $\bigcup_{c\in\mathcal{C}}\rho(c)$ 中 $\mathcal{C}$ 为变更集合。对一次变更 c，其**读集** $\rho(c)\subseteq\mathcal{M}$ 定义为：为正确完成 c 而必须被阅读的模块集合。系统的**架构质量指标**定义为

$$Q(\mathcal{M}) \;=\; \mathbb{E}_{c\sim\mathcal{C}}\big[\,|\rho(c)|\,\big], \tag{6.1}$$

即以历史变更分布加权的平均读集大小。

定义 6.1 是可操作的：它可以直接从版本历史中估计：统计每个合并变更触及的顶层模块数、文件数与目录数，取其分布。这不是完美的代理（触及不等于必须阅读），但它是可测量的，而” 模块化程度” 不是。

定义 6.2 (不变量的强制点)**.** 设 $\mathcal{I}$ 为系统的不变量集合。对 $\iota\in\mathcal{I}$，其**强制点集** $\mathrm{S}(\iota)\subseteq\mathcal{M}$ 定义为所有必须正确实现才能保证 ι 成立的模块集合。称强制是**有效的**，若 ι 在任一执行中不被违反。其**未审定部分** $\mathrm{S}_{\mathrm{un}}(\iota)\subseteq\mathrm{S}(\iota)$ 是那些不阅读就无法确认其正确性的强制点构成的子集：既无机器可检查的证明、又无被强制的类型级约束、又无可单独检查的契约、也无已被接受的证书的强制点。$\mathrm{S}_{\mathrm{un}}(\iota)$ 之外的强制点仍然是必要的——若它们出错，ι 就会失效——但它们可被其证据替代，意思是智能体可以依赖该保证而无须检视实现。

定理 6.3 (不变量集中化)**.** 设不变量 ι 的强制有效，且变更 c 可能破坏 ι（即存在一次合法变更序列使 ι 在缺少对 $\mathrm{S}(\iota)$ 的理解时被破坏）。则

$$|\rho(c)| \;\geq\; \big|\mathrm{S}_{\mathrm{un}}(\iota)\big|. \tag{6.2}$$

因此，在其余条件不变的前提下，最小化 $\mathbb{E}[|\rho|]$ 要求最小化 $\sum_{\iota\in\mathcal{I}}|\mathrm{S}_{\mathrm{un}}(\iota)|$，其下界在每个强制点要么是唯一的一个、要么是可单独检查的时候达到。

证明. 若 c 可能破坏 ι，则执行 c 的智能体必须能够判断该变更是否安全。对 $\mathrm{S}_{\mathrm{un}}(\iota)$ 中的每个强制点，该判断都要求阅读该强制点，因为按定义没有其他证据能了结它；故 $\rho(c)\supseteq\mathrm{S}_{\mathrm{un}}(\iota)$，得式 (6.2)。$\mathrm{S}(\iota)\setminus\mathrm{S}_{\mathrm{un}}(\iota)$ 中的强制点可以经由其证据被依赖，无须被阅读。对 ι 求和：$\mathbb{E}[|\rho|]\geq$

$\mathbb{E}[|\bigcup_\iota \mathrm{S_{un}}(\iota)|]$，且在这些集合不相交的乐观情形下取等号。该下界由 $|\mathrm{S_{un}}(\iota)| = 0$ 达到，即每个强制点都带有无须阅读即可检查的证据——类型系统、私有构造函数、能力令牌、数据库约束或证明。 ■

从 $|\mathrm{S}(\iota)|$ 改为 $|\mathrm{S_{un}}(\iota)|$ 并不是为了稳妥而引入的弱化。它是对一处真实混淆的纠正，而它所划出的区分此前在论证中是缺失的。有三个不同的集合被当成了一个：对 ι 而言其正确行为是必要的那些模块（即 $\mathrm{S}(\iota)$）；证明某次变更安全所需的证据；以及智能体实际必须阅读的源文件。第一个集合并不约束第三个，除非经由第二个，而第二个恰恰是 $\mathrm{S_{un}}$ 所分离出来的东西。一个拥有机器可检查的证明、或拥有稳定的可单独检查的接口的模块，无论其正确性多么必要，都不进入读集。

由此得出两条后果，而它们是实践上有用的那两条。第一，该界解释了为什么把不变量编码进类型具有超出表达力的工程价值，而且是定量地解释的：它降低 $|\mathrm{S_{un}}(\iota)|$，而不只是把不变量挪个位置。第二，它告诉维护者读集实际上来自何处：不是来自不变量起作用的处所的数量，而是来自那些其正确性无所可查的处所的数量。那是一个更小、也更可行动的数字，也是重构应当瞄准的数字。

推论 6.4 (强制点的最优位置)**.** *在满足有效性约束的所有候选中，使式 (6.2) 的期望读集最小的强制点是扇入最小的模块，即那些被最少其他模块依赖、因而最不可能被绕过修改的模块。这一最优性的范围仅限于读集这一目标；见下面的备注。*

注 6.5 (定理6.3 的限度)**.** 该定理有一个前提：强制必须有效。现实中，把不变量集中在单一位置常常失败，因为调用者可以绕过该位置。这正是” 不变量散落” 在工程中反复出现的真实原因。它不是无知，而是对绕过风险的防御。因此定理 6.3 的正确读法是：*不变量散落是一种有代价的防御；若不变量可以真正被集中（例如通过类型系统、私有构造函数、能力令牌或数据库约束使绕过不可能），则集中严格优于散落。*这解释了为什么” 把不变量编码进类型” 在工程上有超出表达力的额外价值：它把 $|\mathrm{S_{un}}(\iota)|$ 降到趋近 0 而不增加绕过风险。这两种机制彼此独立且都重要——类型免除了阅读的必要，有效强制免除了信任的必要。

注 6.6 (集中化只优化了若干目标中的一个)**.** 推论 6.4 识别出的是读集最优的强制点。不应当把它读作识别出了最好的强制点，因为读集只是若干目标中的一个，而其余目标在模型中完全没有被表示。把一条不变量移入单一模块，同时也把负载移到了该模块上，给每一个本可以是本地的操作增加了一跳，集中了*失效影响*——若该强制点失效，所有依赖者一起失效——并制造了*变更瓶颈*，因为此后每一次触及该不变量的变更都要串行地经过同一个所有者。

这些正是全文坚持不得被合并的那些维度（原则 6.22），因此对该推论诚实的陈述是有条件的：*给定在所涉强制点上有效性是可达到的，且给定其他四个维度在该处是可接受的，则扇入最小对读集而言是最优的。*当它们不可接受时，正确的问题是一个多目标问题——正确性、变更影响、延迟与失效域——而读集之界是其中的一项约束，而不是目标本身。一次把” 把不变量集中起来” 当作无条件建议的设计评审，是把该推论用在了它的范围之外。

6.2　近可分解性

Simon 的近可分解性 [39] 是层级结构之所以稳定的经典解释：层内交互频繁，层间交互稀薄。但” 交互稀薄” 本身不可检验。本节给出一个可检验的等价刻画。

定义 6.7 (近可分解)**.** 设模块 M 有一个接口 I。对 M 的一次执行 e，记 $\mathrm{trace}_I(e)$ 为 e 在 I 上的交互序列，$\mathrm{behav}_{\mathrm{out}}(e)$ 为 e 在 M 之外可观察到的行为。称 M 关于 I **近可分解**，若存在函数 f 使得

$$\forall e: \quad \mathrm{behav}_{\mathrm{out}}(e) \;=\; f\big(\mathrm{trace}_I(e)\big), \tag{6.3}$$

即 I 上的观察是 M 外部行为的充分统计量。

定义 6.7 与定义 2.18（T-充分性）的关系是直接的：取任务 T 的查询族 Q 为” 外部行为的观察”，则近可分解性正是 M 关于其接口的 T-充分性。由此，Simon 的定性原理被翻译成一个可判定条件。

命题 6.8 (泄漏对)**.** *M 关于 I 不近可分解，当且仅当存在两次执行 e_1, e_2 使得*

$$\mathrm{trace}_I(e_1) = \mathrm{trace}_I(e_2) \qquad \text{而} \qquad \mathrm{behav}_{\mathrm{out}}(e_1) \neq \mathrm{behav}_{\mathrm{out}}(e_2). \tag{6.4}$$

*称这样的一对为**泄漏对**。*

证明. ($\Leftarrow$) 若存在这样的 e_1, e_2，则任何函数 f 都无法同时满足 $f(\mathrm{trace}_I(e_1)) = \mathrm{behav}_{\mathrm{out}}(e_1)$ 与 $f(\mathrm{trace}_I(e_2)) = \mathrm{behav}_{\mathrm{out}}(e_2)$，因为两边的自变量相等而函数值不等，故式 (6.3) 不成立。($\Rightarrow$) 若式 (6.3) 不成立，则不存在满足条件的 f，由函数的定义，必然存在两个自变量相同而应取函数值不同的点，即式 (6.4)。 ■

推论 6.9 (诊断程序)**.** *命题 6.8 给出一项可执行的诊断：构造两个在接口层面完全相同的场景，但在内部状态或外部环境上不同，观察外部行为是否分歧。若分歧，则该接口不是充分统计量，模块边界在此处泄漏。*

推论 6.9 把” 这个模块的抽象是否漏了” 从一个审美判断变成一次差分测试。这一结论的实用价值在于：*抽象是否泄漏是可测的，而不只是可感的。*工程上常见的” 阴影状态”” 隐式全局”” 跨请求缓存污染”，全部是泄漏对的具体实例。

原则 6.10 (设计的等价目标)**.** *由推论 6.9，设计的可检验目标可以写成：对每一个声称的模块边界，其接口观察应当是外部行为的充分统计量；泄漏对的存在即边界的失效。*

原则 6.11 (对一个声称的边界有两条判据，而不是一条)**.** 一个模块与其环境之间被声称的边界作出两项彼此分离的主张，而它们会各自失败。

(a) **充分性**：接口观察决定外部行为。这就是定义 6.7；它的失败是一个泄漏对（命题 6.8），由差分测试（推论 6.9）发现。

(b) **完备性**：边界所声明的观察映射就是跨过该边界可观察到的一切。它的失败是一个被边界声明为无关的差异无论如何变得可见——经由时间、资源占用或失效行为——而它由信息流分析的技术而非差分测试发现（备注 2.12）。

二者彼此独立，且两个方向都独立。一个边界可以是充分而不完备的：每一对接口相同的执行都具有相同的外部行为，然而一个被声明为无关的差异却可以经由时间通道从上方读出。它同样可以是完备而不充分的：没有任何东西经由未声明的通道泄漏，而所声明的接口却过于粗糙，不足以确定行为。

推论 6.9 与原则 6.10 只处理第一条判据。第二条判据才是静默失败的那一条，因为一个经由未声明通道泄漏的边界会通过每一项针对所声明接口写成的差分测试——该测试永远不会构造出那对冒犯性的执行，因为这一对在边界声称无关的某处上不同。

6.3　技术债

”技术债”在工程语境中通常是隐喻。本节给出一个精确但仍有用的定义。

定义 6.12 (技术债)**.** 给定编码方案，系统代码的描述长度为 $\ell(\text{code})$，其**最短生成描述**为 $\ell^\star = \min\{\,|d| : d \text{ 生成 code}\,\}$（即 Kolmogorov 意义下该代码的最短生成程序）。定义**技术债**

$$D \;=\; \ell(\text{code}) \;-\; \ell^\star. \tag{6.5}$$

这个定义是可辩护的但不可计算的（$\ell^\star$ 不可计算），因此它的价值在于给出*方向*与*判据*，而不是数值。

原则 6.13 (技术债的产生与偿还)**.** 技术债在*变更未重新抽象*时增长：每次修改若不重新抽象，代码与”最短生成描述”之间的差距就扩大。因此：

(a) 重构 = 重新压缩，是把压缩率维持在可理解阈值之上的生存必需；
(b) 不重构的系统不是停止生长，而是在积累不可理解性；
(c) 判断一次重构是否值得，等价于判断它是否显著降低 D，即是否让系统更容易被更短的描述生成。

原则 6.13(c) 给出一条实践判据：如果一次重构不能让你用更少的话说清这个模块做什么，那么它很可能只是格式整理。

注 6.14 (重构的率失真解释)**.** 把 $(\ell(M), L_T(M))$ 看作率失真平面上的点 [71]（定义 2.19）。系统的当前状态是曲线上或曲线外的一点：*曲线外意味着存在同描述长度下损失更低的模型*，即纯粹的改进空间；重构即沿曲线移动或把点搬回曲线。

这一解释给出一个可检验预测：*率失真曲线的斜率 $\partial L_T/\partial \ell$ 应当预测哪些重构划算*，斜率为零处的重构不降低任务损失，因而不值得做；斜率大的位置即使代价高昂也值得。若该预测力在真实项目中不成立，则本章的目标函数选择需要被修改（第九章假设 H_{12}）。

6.4　可治理复杂度与理解容量

第 6.1 节把架构质量定义为一个静态量 $\mathbb{E}[\rho]$。但真实系统有*速率*：变更以某个频率到达，而组织的阅读能力有限。本节补上这一动力学，它是全文唯一一处时间变量。

定义 6.15 (可治理复杂度与理解容量)**.** 设 $\lambda(t)$ 为变更到达率（单位时间的变更数），$\mathbb{E}[\rho]$ 为期望读集（定义 6.1）。定义**可治理复杂度**

$$C(t) \;=\; \mathbb{E}\big[\rho\big] \cdot \lambda(t), \tag{6.6}$$

即维持系统所需的信息吞吐，单位为" 模块-次/单位时间"。设组织的**理解容量**为

$$K \;=\; h \cdot b, \tag{6.7}$$

其中 h 为可投入的理解人数，b 为单人理解带宽（模块-次/单位时间）。称系统处于**可治理状态**，若 $C(t) \le K$ 对所有相关 t 成立。

命题 6.16 (容量模型下所需人力的标度指数)**.** 设系统有 n 个特性，每个特性以速率 λ_0 独立变更，故 $\lambda \propto n$。设读集 $\mathbb{E}[\rho]$ 关于 n 的标度为 n^{α}，其中 $\alpha \ge 0$ 是该模型的参数，$\alpha = 0$ 对应完全局部化，$\alpha = 1$ 对应未受约束的耦合。则可治理复杂度标度为

$$C \;\propto\; n^{1+\alpha}. \tag{6.8}$$

由 $C \le K$ 且 $K = hb$、b 为常数，使系统保持可治理所需的人力标度为 $h \propto n^{1+\alpha}$。因此，在该模型之内：

(a) 若 α 被压到 0，人力随系统规模线性增长，系统可长期存续；

(b) 若 $\alpha = 1$，人力须随规模平方增长，系统在某个规模上必然失稳。

证明. 由定义 6.15，$C = \mathbb{E}[\rho] \cdot \lambda$。代入 $\lambda = n\lambda_0$ 与 $\mathbb{E}[\rho] = \Theta(n^{\alpha})$ 得 $C = \Theta(n^{1+\alpha})$。由 $C \le K = hb$ 得 $h \ge C/b = \Theta(n^{1+\alpha})$。 ■

有两件事是该命题并未确立的，而早先的标题——架构决定标度指数——以暗示的方式断言了它们。第一，它没有表明该指数由架构决定；它假定 $\mathbb{E}[\rho]$ 按 n 的幂标度，并把 α 当作该标度的参数。α 是否由架构决定，以及用于估计它的读集代理是否有效，都是经验问题——后者是假设 H_4，前者正是假设 H_{14} 直接追问的问题。第二，它没有表明真实组织的人力会遵循该指数。它表明的是：若组织要在所声明的容量模型下保持可治理，人力就必须遵循该指数；这是关于一个理解容量模型的条件陈述，而不是关于招聘的预测。其经验内容是：估计出的 α 更高的系统，应当要求每单位增长对应不成比例地更多的协调投入；这是可检验的，第九章提出了检验的方式。

推论 6.17 (架构的实际作用)**.** 命题 *6.16* 给出架构的一个精确职责：它不降低单个变更的成本，而是降低成本随规模增长的指数。这解释了为什么架构收益在小系统上不可见、在大系统上决定存亡，也解释了为什么" 架构过度设计 " 的批评在小规模下总是显得成立。

推论 6.18 (两个经典结论的导出)**.** (6.8) 式给出两个经典论断的导出：

(a) **组织规模与系统规模的关系**：由 $h \propto n^{1+\alpha}$，组织规模必须随系统规模增长，且增长指数由架构决定。这是组织结构与系统结构对应关系 *[50]* 的动力学版本，前者是静力学陈述（结构镜像组织），后者给出可治理的上限。

(b) **加人使项目更慢**：增加 h 使 K 线性上升，但同时提高在途变更数从而抬高 λ，并通过新增接口抬高 $\mathbb{E}[\rho]$。当后两项的增幅超过前一项时，C/K 反而恶化。这是对”向延期项目加人使其更延期”这一经验规律的一个模型内解释。

原则 6.19 (设计含义)**.** 由命题 6.16，架构审查的判据不应是” 读集是否小”，而应是 ” 读集关于规模的标度指数是多少 ”。一个读集目前很小但随着规模增长很快的系统，比一个读集当前较大但标度平坦的系统更危险。这使假设 H_4 需要被加强：读集的价值主要在于其标度，而不在于其水平。

注 6.20 (本章的动力学是唯一的)**.** 全文此前是静力学的：结构、不变量、读集、契约都是时不变的量。定义 6.15 是唯一的动力学变量，因为它是唯一一个” 组织的处理能力” 与” 系统的要求” 必须在速率上匹配的地方。其余各处的时间都可以被消去。

6.5　设计流程

(D1) **先固定不变量**。在写任何代码之前，写下这个系统必须永远为真的命题（例如” 账务总额守恒””I/O 线程永不阻塞”” 版本号单调递增”）。不变量是层级的引力，一切结构围绕它结晶。由定理 6.3，每条不变量应当指定一个唯一的强制点，并说明绕过为何不可能。

(D2) **选原语**。原子、组件、消息、事务、实体……原语一旦选定，就成为整个系统后续讨论的共享语言。选原语是这个层级最重要的单次决策，因为它决定了此后所有表达式的形状。

(D3) **定数据模型**。实体、关系、生命周期、所有权。数据模型比逻辑稳定一个数量级，因此应当先冻结它。

(D4) **切器官**。模块化的依据不是” 功能相似”，而是会一起变化的信息被藏在一起 [49]。每个模块的检验标准是：接口小到能一口气记住，实现深到可以整体替换 [53]。接口是你刻意维护的压缩摘要，实现是允许被重写的压缩内容。

(D5) **让物理定律可执行**。不变量写在文档里会腐烂，编码进类型系统、断言、契约测试与性质测试里才长存。类型是编译器强制执行的架构文档。这是” 设计可理解性” 与” 设计正确性” 的合流点：能被机器检查的不变量，人和 AI 都不必靠记忆维护。

(D6) **沉淀决策记录**。记录” 为什么这么选、放弃了什么”。代码是结果，决策记录是生成规则的一部分，读者逆向压缩时，决策记录能省掉最贵的猜测成本。

(D7) **重构 = 重新压缩**。见原则 6.13。

贯穿全部七条的元规则：

原则 6.21 (为读者的心智带宽设计)**.** 为你的读者的心智带宽而设计，而非为你的手速。代码被读的次数远多于被写的次数，架构的用户是未来的理解者。

6.6　横切关注点与分层的极限

必须明确承认分层的失败模式。有一类关注点无法被局部化到任何一层：

- 安全与权限（每个模块都可能引入漏洞）；
- 多租户隔离（跨越存储、计算与网络）；

- 分布式追踪（必须贯穿所有调用路径）；
- 事务与一致性（横跨状态所有者的边界）；
- 可观测性与配额（在每个资源消费点都需要）。

原则 6.22 (横切关注点是独立维度)**.** 对横切关注点，成熟的处理方式是承认它们是*独立维度*（策略平面、能力模型、污点传播、服务网格），而不是硬塞进层次结构假装它们是某一层。

这一原则直接来自第三章的讨论：层级结构由近可分解性支撑，而近可分解性的前提是层间交互稀薄。当某个关注点在每个模块上都有非平凡的交互时，它按定义破坏了近可分解性，因此不可能被表示为”某一层”。强行表示的结果是层级结构失效，而失效往往被误诊为”分层不够彻底”。

6.7　设计检查项

以下检查表把本章内容转为可执行条目。每一项都应在设计评审中被明确回答。

表 6.1: 大型系统设计检查表

编号	检查项
A1	系统必须永远为真的命题有哪些？每条在何处被强制？绕过为何不可能？
A2	每条不变量的强制点是否唯一？若不唯一，代价（读集）是否已被显式接受？
A3	本层的原语是什么？它们是否是后继所有讨论的共享语言？
A4	数据模型是否已先于逻辑冻结？实体、关系、生命周期、所有权是否明确？
B1	每个模块的接口是否小到能一口气记住？
B2	每个模块是否深到可以整体替换？
B3	对每个声称的边界，是否存在泄漏对？（即：接口相同而外部行为不同的两次执行）
B4	每个模块的*无视集*是否已被显式写下？
B5	哪些模块会一起变化？该问题是否用共同变更历史验证过，而非仅凭直觉？
C1	添加一种新功能，需要修改多少个不相关的模块？（读集实测值）
C2	替换底层实现，是否会迫使业务逻辑大规模修改？
C3	一个子系统失败时，故障会传播到哪里？
C4	新增一种任务类型，是否需要重写公共调度逻辑？
C5	并发访问是否会破坏状态不变量？在何处被检测？
C6	性能瓶颈是否集中在少数关键路径？
D1	不变量是否已被编码为可执行形式（类型、断言、契约测试、性质测试）？
D2	每个模块的契约是否可被隔离验证？若不能，它是模块吗？
D3	每个关键子系统是否有可追溯的认知记录（表 5.1）？
D4	决策记录是否覆盖了”为什么这么选、放弃了什么”？

编号	检查项
D5	横切关注点是否被作为独立维度处理，而非伪装成某一层？
E1	系统是否需要统一基座？由 $n \times m$ 判断，而非代码行数。
E2	六职能中，哪些自建、哪些外借？判据是不变量的领域关键性。
E3	自建的部分，其不变量是通用宿主无法提供的吗？
E4	外借的部分，宿主提供的保证强度是否足以支撑本层契约？

定理 6.3、命题 6.8 与 6.16 在给定模型下成立。经验内容是假设 H_4 与 H_{12}：读集的标度指数优于读集水平与传统规模指标；不变量集中化（在绕过不可能的前提下）降低读集与缺陷率；率失真斜率预测重构收益。若三者被否定，本章的目标函数选择需要修改。

第七章　从意图到可验证执行

本章处理问题 1.4：如果编码成本趋近于零、人不再逐行审阅实现，正确性如何被机械地保证。答案由三部分构成：把正确拆成可分别处理的维度，把中间过程定义为一条可验证细化链，把信任组织成两种生成模式的复合结构。本章同时给出该链条上一个不可机械封闭的入口，以及它在何种意义上可以被共识逼近。

7.1　正确性的维度

一个 1 ns 的调用被 AI 改写成 1 μs 的调用，功能完全一致但性能差了一千倍，这个例子说明” 正确” 不是单一维度。

定义 7.1 (五个正确性维度)**.** 见表 7.1。

表 7.1： 五个正确性维度及其可机械检查性

维度	需要回答的问题	能否机械检查
功能正确性	输出与状态变化是否符合要求？	对明确规格，可证明或测试
副作用正确性	有没有执行未授权的 I/O、写入、网络请求、状态变更？	可通过效果规格、权限边界与运行时隔离限制
资源正确性	时间、内存、带宽、能耗是否满足预算？	部分可静态证明；部分只能测量或在指定环境下给出统计保证
系统正确性	组件组合后，并发、故障恢复、状态一致性是否仍正确？	部分可形式化证明，部分需模型检查、故障注入与运行验证
目标表达正确性	机器执行的规格是否真的是人想要的目标？	**不能从程序正确性证明中自动推出**

表 7.1 最后一行是整个问题的关键：*一份程序完全可能正确地实现了一份错误的规格*。编译器即使百分之百正确，也可以把一个错误的程序百分之百正确地编译成机器码；形式化证明同样无法拯救一个遗漏关键要求的规格。这一区分的历史表述是软件需求工程中的验证与确认之分 [52]。

7.2　可验证细化链

定义 7.2 (可验证细化链)**.** **可验证细化链**由七个阶段构成：

(G1) **目的与意图**：要实现什么；哪些结果是成功；哪些副作用不可接受；目标冲突时如何取舍。

(G2) **系统与环境模型**：把运行中的状态、事件、操作、资源、操作系统接口、外部环境与可观察行为表示出来。

(G3) **形式化行为契约**：按定义 2.22 给出允许行为、禁止行为、必须成立的不变量、进展条件、资源预算与环境假设。

(G4) **可实现性判定**：检查契约是否自相矛盾，目标在指定资源、硬件、操作系统与环境条件下是否真正可实现。

(G5) **实现综合与搜索**：由生成器、搜索算法或其他综合器产生候选实现。候选实现不预先获得信任。

(G6) **从抽象行为到二进制的正确性证明**：检查候选实现及其编译、链接与其他转换，证明生成的机器码在规定环境下满足行为契约。

(G7) **经认证的部署与实际执行**：核实正在运行的是已验证的二进制，在规定环境中执行，并对无法静态保证的条件进行监测、限制与故障处理。

这条链的意义不在于增加七个流程步骤，而在于*给每一步规定不同的数学任务*。(G2) 不应是自然语言描述，而是对系统状态、行为与环境建立可解释的模型；(G3) 判断要求是否可满足；(G4) 搜索可能的实现；(G5) 建立从实现到所要求行为的证明；(G6) 确保实际执行的对象与证明所对应的对象一致。

原则 7.3 (边界上传递什么)**.** *在每一道边界上，不仅传递一个产物，还要传递这个产物代表什么，以及它为什么可以被接受的证据。*

注 7.4 (不可机械替代的入口)**.** 验证结果与反例可以反向进入 (G2)、(G3)、(G4)；唯有 (G1) 中的意图审定，无法被一般性的代码正确性证明取代。理由见第 7.5 节。

7.3　转换边与证据

把定义 7.2 的七个阶段压成五条转换边，每条边有其典型证据与关键缺口。表 7.2 给出。

注 7.5 (关系 R_i 的异质性)**.** 按注 2.6，不同转换边上的解释关系不必是同一类。对 I→R 与 R→S 而言，可用的" 关系" 可能只是追溯映射而非形式化关系 [1, 2]；对 P→C→B 而言，它可能是编译器的语义保持定理 [22, 24]、单次翻译验证 [23] 或翻译认证 [26]；对 B→E 而言，它可能是运行时监控的判定 [34]。把它们强行统一为同一种关系是错误的做法。

注 7.6 (每条边上已有的工程实现)**.** 表 7.2 并非纯粹的设想：P→C→B 上的经验证编译 [22] 与经验证编译器后端 [24]、内核级的精化证明 [25]、以及证明携带代码 [15] 已经在特定语言、编译器、内核与系统中取得了重要进展。真正未解决的是：如何把这些高可信度的局部成果，扩展到更广泛的目标、更多种类的运行环境，以及实现可以被任意生成或动态调整的系统。

表 7.2: 五条转换边、典型证据与关键缺口

边	输入 → 输出	典型证明/证据	关键假设与主要缺口
I→R	自然语言/对话 → 结构化需求、领域事实与待澄清项	利益相关者审定、需求来源链接、例子与反例、冲突报告	意图没有自动给定的数学真值；外部事实与利益冲突不能只靠类型检查解决
R→S	需求 → 形式化性质/契约	追溯矩阵、需求级测试、一致/可满足/可实现分析、真空性与变异分析	形式化映射要同时保留禁例与正例；需求遗漏无法只从内部一致性推出
S→P	规格 → 程序/系统候选	综合算法、SMT、模型检查、Hoare/类型/依赖类型证明、反例引导归纳综合	有限 DSL、逻辑片段与搜索预算；必须防止前提过强与规格空洞
P→C→B	源程序 → 逐阶段中间表示 → 机器码/可部署制品	经验证编译器定理、逐次翻译验证/认证、证明携带代码	必须定义源/目标语义；链接、装载、汇编、运行时、第三方库与硬件未必在证明范围内
B→E	二进制与配置 → 真实环境行为	部署完整性、系统验证、运行时监控、隔离与能力机制、性能与调度分析	模型与真实 CPU、操作系统、网络、调度、故障与服务假设必须对齐

7.4　组合条件与失效模式

命题 2.4 已经给出组合的充分条件。这里补充三个具体的组合失效反例，它们的作用是证明” 局部正确” 确实不蕴含” 整体正确”。

例 7.7 (三个组合失效). (a) 源程序语义正确，但链接器选错了库。
(b) 每个服务本身满足接口契约，但契约对网络重试的假设互不兼容。
(c) 功能证明成立，但时延预算没有覆盖缓存争用。
三者的共同结构是：命题 2.4 的条件 (a) 或 (b) 之一被破坏。

用契约代数（定义 2.27、命题 2.29）读这三个反例，它们不再彼此无关，而是三类失败：

推论 7.8 (三个反例的共同结构). *(a) 链接器选错库：着色失配，被链接实现其颜色不等于链接点声明的颜色；*
(b) 契约对网络重试的假设互不兼容：接口图出现环，重试把无环调用图变成带环图，而着色未随环重新验证（注 2.32）；
(c) 时延预算未覆盖缓存争用：契约分量缺失，资源分量 R 未进入颜色，代价被当作实现细节。
三者分别是颜色外延不足、接口图改变、契约分量缺失。

结论是：组合规则必须把这些接口条件与非功能性条件显式纳入。

注 7.9 (保证的整体论性质). 推论 7.8(c) 与注 7.25 共同说明：任何验证结论实际断言的不是” 性质 φ 成立”，而是合取式

$$\varphi \wedge A_1 \wedge A_2 \wedge \cdots \wedge A_k,$$

其中 A_i 为全部环境假设。k 事先不可枚举，只有当某个 A_i 真被违反时它才显形。这是 Duhem–Quine 论题 [64] 在软件验证中的形态：不存在对单一命题的判决性检验，只有对整个合取式的检验。它的直接后果是，”测试通过”的强度严格受限于未被枚举的假设集。这也解释了表 7.3 为何必须为每类证据附一份”不应声称”清单，该清单就是已知的 A_i。

7.5　意图鸿沟

这是本章最需要谨慎的一节。

问题 7.10 (意图鸿沟)**.** 人最初用自然语言表达的意图 I，如何确保被转化为正确的形式化规格 Φ？我们希望有 $I \equiv \Phi$，这里的等价指”Φ 所要求的行为完整地反映了人的真实目标”。

一个流行的论证是：由 Gödel 不完备定理与 Rice 定理，意图与规格之间的裂缝是原则性的、永远无法机械封死的。该论证不成立。

命题 7.11 (本文不接受该论证的理由)**.** *Gödel 不完备定理讨论的是满足特定条件的形式系统中形式可证明性的限制，它并不直接说明机器无法判断一份需求规格是否符合某个人的真实意图。Rice 定理 [28] 说明对一般程序而言非平凡语义性质不可判定，这是对通用静态分析的根本限制，但它不是关于人的心理意图与形式规格之间关系的定理。要从中推出”意图永远无法完整形式化”，还需要额外的哲学与认知假设。Turing 的原始结果 [27] 同样只适用于明确编码的计算问题与一般算法主张。*

那么意图鸿沟存在吗？存在，但根据是认识论的，不是逻辑的。

命题 7.12 (意图鸿沟的三条认识论根据)**.**

(i) **需求是生成性的，而非枚举性的。**人在看到系统行为之前无法完整列出自己的需求。”不能泄露隐私”这条要求，在看到系统确实在收集数据之前可能根本没有浮现。这不是逻辑不可判定，而是人类认知的构造性局限：偏好是被环境激活的，不是预先存在的完整清单。

(ii) **规格是意图的有损压缩。**意图存在于一个因果的、具身的、演化塑造的心智中；规格是一个有限的符号对象。有损压缩意味着信息必然丢失，而丢掉的是哪些部分事先不可知。这是信息论层面的现象，不是元数学层面的。

(iii) **验证与确认的区分至今未被推翻。**验证（*verification*）回答”实现是否符合规格”；确认（*validation*）回答”系统是否满足真实需求”。这个区分之所以存在并沿用至今，正是因为后者无法被机械地归约为前者。

把命题 7.11 与 7.12 合起来，得到本文对该问题的准确表述：

原则 7.13 (意图鸿沟的准确表述)**.** 规格转换过程不是一个可以普遍机械保证的步骤，原因不在逻辑不可判定，而在人的偏好是动态的、情境性的、在遭遇之前不可全知的。

这一修正没有削弱命题，只是替换了承重墙。它同时说明：机器可以辅助澄清需求、提出反例、生成场景、寻找矛盾，并检查规格内部是否一致；而最终采用哪份规格、它是否完整表达了目标，仍然需要一个明确的责任主体。

注 7.14 (三个问题，而不是一个；以及自动化能为每一个做什么). 意图鸿沟常被当作单一障碍来讨论。它其实是三个，它们需要不同的工具，而把它们分开来看就会清楚：本文框架拒绝机械化的区域，比通常以为的要小。

(a) **语义表达**：给定事实上已经确定下来的需求，它们能否被写成形式化规格？在有界领域中这大体上可以自动化，其工具是类型系统、定理证明器，以及第十章讨论的规格综合方法。命题 7.12 没有任何一条否认这一点，把它读成否认这一点是过度解读。

(b) **需求发现**：利益相关者是否已经找到并表达了所有要紧的东西？这正是命题 7.12(i)，其补救手段是交互：反例、场景，以及第五章的探测循环。它部分可自动化，意义是机器可以生成探针；它不可自动化，意义是答案必须来自有切身利害的人。

(c) **规范性的取舍**：当需求彼此冲突——时延与安全性、隐私与分析能力、成本与性能——谁有资格作出决定？这根本不是认识论问题；它是第 7.13 节的权威轴，其工具是程序性的：一个被指定的责任机构，以及一份决定记录。

这样读这一鸿沟，也纠正了相反方向上一个常见的过度表述。因为 (a) 大体可机械化，”某些意图可以被精确定义并自动检查”是真的；不能被机械化的是残留部分——尚未被激活的偏好、尚未浮现的冲突，以及无人有资格独自作出的决定。因此本文所捍卫的准确主张比”意图无法被形式化”要窄：*一份关于实现正确性的形式化证明，本身无法确立该规格是否服务于所有相关各方的目标*，而开放世界中的剩余部分必须由一个独立的确认过程来处理。该过程可以使用自动化工具；它不得把一个工具的内部一致性当作”目标是对的”的证据。

注 7.15 (意图鸿沟是框架问题). 命题 7.12 的三条根据不是三项独立发现，它们共同指向一个更古老的问题。要形式化意图，必须预先判定哪些事实*相关*；而判定相关性需要已经知道哪些结论有待推出，这正是框架问题：不预先知道答案，就无法枚举相关前提。

因此意图鸿沟不是形式化方法的缺陷，而是框架问题的一个实例。这一识别有两项后果：

(a) 它解释了为什么更强的形式化工具不解决该问题，工具提高的是*推导*能力，而问题出在*前提的枚举*；

(b) 它解释了为什么*交互*是唯一有效的手段。框架问题在一般情况下未解，只在具备感知–行动闭环的系统中被回避：智能体无需预先枚举相关性，因为环境会对动作给出反馈。第五章的探测循环与本节的可追溯精化，都是把这一回避机制搬到需求层。

于是”交互式精化”是被逼出来的，而不是在若干候选方案中被选中的。

注 7.16 (无视集是框架问题被逐层级偿付之处). 注 7.15 把意图鸿沟与框架问题等同起来，就到此为止。这一等同还可以再推进一步，而推进之后就把本文此前彼此分离的两部分接了起来。

本文所使用的框架问题，其形式是*判定相关性的困难*：在还不知道要推出什么结论之前，就要说出哪些事实与结论有关。无视集（定义 2.8）正是一个这种形状的相关性判断，只不过它被显式写出、并且是局部的：$\mathrm{Obl}(\Lambda_i)$ 命名那些 Λ_i 声明与自己无关的差异。因此两者之间的关系不是类比，而是尺度上的关系——而把”得到了什么”与”没有得到什么”说清楚是值得的。

得到的第一样东西是*局部化*。为一个系统整体提出的框架问题，不会因为改成对它的某个层级提出而变小；但它被该层级所声明的观察映射约束住，并且用它能够陈述的”货币”来

清偿。全局问题——哪些事实与系统的目的相关——仍然没有一般解，本文也不声称有。层级结构换来的是：这个全局问题永远不会被一次性提出；每个层级回答自己有界的那一版本，并把答案公布出来。

得到的第二样东西是*可检验性*。仅仅被持有的相关性判断无法被批评；被写下来的则可以。第 6.2 节给出一个已声明的判断可能出错的两种方式——接口对外部行为可能不充分，或者观察映射相对于事实上可观察的东西可能不完备。两者都是检查，而不是意见。在这个意义上，无视集是框架问题的工程形态：它不是框架问题的解，而是一个为它付出代价、且这笔代价可以被审计的地方。

由此得到两项后果，它们也正是本条注放在这里、而不放在第五章的原因。第一，它解释了原则 2.21 的下潜协议为什么有那样一条停止规则：一直下潜到支配该变更的不变量变得可见为止，因为那个深度正是相关的无视集已被咨询过的深度，而继续下潜所换来的相关性，没有任何义务依赖其上。第二，它使意图层的困难可以用与模块边界相同的措辞读出来：意图鸿沟是 L_∞ 上的框架问题，而 L_∞ 恰恰是不能为机器声明任何无视集的层级，因为声明一个无视集本身就已经是在决定目的。

7.6　验证与生成的不对称

另一个需要修正的表述是把” 判卷比答卷容易” 称为”P 与 NP 的世界性红利”。这一说法不成立：P 是否等于 NP 仍是开放问题，我们并不知道一般而言找到解是否必然比检查解困难。

但必须把这一修正与另一件事区分开来：P 与 NP 的未决，*并不使*” 验证比生成便宜” 这一现象成为虚构。

命题 7.17 (结构性不对称)**.** *多项式时间证书检查的结构性不对称是 NP 类的定义本身：一个解可以被高效验证，这件事本身是关于计算的一类基本事实，它不依赖于我们是否证明了* $\mathrm{P} \neq \mathrm{NP}$。

论证. NP 的定义为” 存在多项式时间的验证器 V 与多项式长度的证书 w，使得 $x \in L \iff \exists w : V(x, w) = 1$”。这一定义本身断言了验证的存在性与高效性，而不对寻找 w 的复杂度作任何断言。因此验证器架构的有效性建立在定义层面，与 P 对 NP 问题的最终答案无关。 ■

这一成本量级提示了” 验证便宜” 的准确边界。限制不在 P 与 NP 的未决，而是*证书长度*：检查一份证明可以在多项式时间内完成，而这份证明本身可以长到天文数字。因此：

原则 7.18 (验证成本的双分量)**.** ” 验证比生成便宜” 必须被分解为两个独立命题：

(a) **检查时间便宜**：给定证书，判定是高效的（命题 7.17）；

(b) **证书尺寸便宜**：证书本身足够短，可以被存储、传输与复核。

(a) 由 NP 的定义保证；(b) 不由它保证，且在许多实际系统中是真正的瓶颈。一个只满足 (a) 的验证体系在工程上不可用，因为无人能复核一份无法读完的证书。

原则 7.18(b) 是可测量的：对给定的证明器与性质类，测量证书尺寸的分布即可。它同时解释了为什么” 小内核 + 大证明” 的架构在实践中总要配合一个*可读的证明摘要层*（不变量、

契约的散文陈述、反例），而不是只提供证明项。

例 7.19 (形式化验证的历史成本量级)**.** 微内核 seL4 的初始功能正确性证明投入了约 11 人年（不计可复用工具与框架的投资），当时的证明脚本约 20 万行，用于验证约 8500 行 C 代码[25]。这个量级说明形式化验证的历史成本结构。AI 可能显著降低编写证明、寻找不变量与修复证明的成本，但规格设计、模型选择、证明架构与可信基础仍然是实际工作。

7.7　证据等级

不同的证据提供不同强度的支持，混淆它们是这一领域最常见的错误。表 7.3 给出证据等级与相应的” 不应声称” 清单。

表 7.3: 证据等级

证据类型	能够支持的结论	不应声称的结论
机器检查的形式证明	对声明的命题，在给定逻辑、定义、公理、库及检查器假设下成立	不证明命题完整表达意图；不覆盖未建模环境；不自动说明时延或能耗
经验证编译器定理/翻译证书	特定源/目标语义间存在所定义的保持或精化关系，或单次翻译结果经验证	不意味着链接器、操作系统、硬件与外部服务都经过验证
静态分析与抽象解释	相对于可靠近似，证明或保守估计所分析的程序性质	不保证精确、无误报；不等于实际性能测量
测试与变异测试	发现反例、评估已列测试上的规格行为，提供有限样本证据	不证明所有输入与所有执行安全；通过率不是规格语义等价的通用证明
统计性能证据	指定数据、采样流程、硬件与置信方法下的经验分布	不等于硬实时最坏执行时间；不能无条件外推到分布变化后的部署
运行时监控	对被观察到的轨迹及监控器语义给出在线判定	若观察不完整，不得把” 未检测到” 表述为” 所有执行均满足”

原则 7.20 (三个必须区分的状态)**.** ” 检测到违反”、” 尚未观察到违反” 与” 所有执行均被证明安全” 是三个不同的状态，绝不能互相替代。

定义 7.21 (制品证据清单)**.** 每个制品应携带十项证据，见表 7.4。该清单的作用是使” 某个二进制是否可被信任” 成为一个可审计的问题，而不是一个关于作者身份的问题。

第 9 项与第 10 项是最容易缺失、也最重要的两项：前者把” 保证的边界” 显式化，后者把保证从一次性检查变成生命周期机制。

7.8　副作用与约束机制

问题 1.4 中的例子混合了不同种类的副作用。分开看才知道各自的机械机制。

定义 7.22 (副作用四分类)**.** 见表 7.5。

前三类已有机械机制覆盖，且是纵深配置（证明 > 类型 > 测试 > 计价 > 隔离 > 监控，每一层的信任基础都比上一层更简单）。只有第四类（规格偏离意图）永远需要人。

表 7.4: 制品证据清单

编号	证据项
1	规格版本与需求追溯标识
2	形式化规格与环境假设
3	候选源代码哈希
4	编译器、参数、目标架构与依赖版本
5	证明及检查器版本
6	资源分析模型与结果
7	测试与监控策略
8	可信计算基清单
9	**已知未覆盖行为**
10	**重新验证规则**（哪些变化触发重新计算受影响的证据）

表 7.5: 副作用分类与相应的机械机制

类别	例子	机械机制	验证时点
功能性	生成的函数修改了不该修改的全局状态	类型系统、效果类型（把” 会碰什么” 变成签名的一部分）、所有权系统	编译期，零运行时成本
资源性	1 ns 变 1 μs、内存暴涨	燃料/计价计量、最坏执行时间分析、配额	编译期 + 运行时双重
安全性	越权访问、逃逸沙箱	能力模型、沙箱、字节码验证器	准入时 + 运行时
语义漂移	规格本身就错了	**没有机械机制**	人的审定与共识

注 7.23 (三类工程原型的保证范围). 三种常见工程机制覆盖的是不同种类的性质，不能并列宣传：

(a) 浏览器沙箱保证的是*执行隔离与权限边界*，不保证功能正确性。它是地球上最大的不可信代码运行时：每天执行数以亿计来源不明、从未审查的代码，它从不审查，它隔离。

(b) 字节码验证器验证的是*其规则所覆盖的安全性质*（无越界访问、循环有界、无悬挂指针），不是任意业务逻辑正确性。一个几万行的确定性验证器裁决着百万行自由代码的准入，这正是" 编译器式的、不以意志为转移的保证" 的实例。

(c) 资源计价机制保证的是*抽象资源计量下的约束*，不是真实的墙上时钟时间。它把性能副作用从" 需要人看出来" 变成" 预算违规自动拒绝执行"，但 1 ns 到 1 μs 的退化它未必能捕获，因为它计量的是抽象操作成本而非实际硬件延迟。实时性能保证需要时序分析与受限的运行环境，这是另一套完全不同的工程。

7.9　运行时保证

形式化证明依赖环境假设，而实际环境可能变化。因此除静态证明外，还需要让执行系统在假设失效时仍然保持安全。

原则 7.24 (分层防御). 理想的架构应结合静态证明（针对已知确定性行为）与动态监控（针对未知或非确定性行为），形成分层防御：静态证明能证明的先证明；剩余风险由独立监控检测；严重违规由隔离或回退限制影响。

注 7.25 (监控自身的边界). 即使运行时监控器本身正确，也不代表它掌握了完整的执行事实。观测轨迹缺失、事件模糊、传输不可靠与并发不可见行为都会限制结论。因此监控系统的观测机制本身必须被纳入保证链。

注 7.26 (契约组合的四个条件). 局部性质组合依赖：环境确实满足子组件假设；连接拓扑是良置的；反馈条件可检查；观察与时间语义兼容。组件" 单独正确" 与系统" 整体正确" 不能互换。

原则 7.27 (不变量的条件式). 一条可用的假设–保证契约应当同时写明：它依赖的环境条件、它向外保证的行为、它可以产生的副作用、它必须消耗与占用的资源、以及它与其他子系统组合时必须满足的约束。

7.10　制品绑定

保证条件式 (2.12) 的最后一项，即实际执行的对象就是被验证的对象，在既有讨论中经常被遗漏。它不是边角细节：即使验证器正确地接受了某个二进制，如果部署系统使用的是另一个文件，或者加载时的环境改变了关键前提，前面的验证结论不能自动套用。供应链攻击恰好瞄准这条裂缝。

因此完整的信任链条必须包含：内容寻址的构建产物、验证证据与二进制的密码学绑定、以及部署时的再次校验。用式 (2.13) 的记号，即 $\mathrm{digest}(B_{\mathrm{deployed}}) = \mathrm{digest}(B_{\mathrm{verified}})$ 且 $E_{\mathrm{deployed}} \models A_{\mathrm{claimed}}$。

原则 7.28 (可复现构建的三层区分)**.** 应当区分三种不同强度的性质：

(a) **可复现**（reproducible）：同样的输入产生逐字节相同的输出；

(b) **可验证**（verifiable）：独立第三方可以从输入重新构建并比对；

(c) **已验证**（verified）：证明某个特定二进制满足其声明的性质。

(a) 与 (b) 是 (c) 得以生效的前提条件；(c) 不含 (a)。三者不可互相替代。

7.11　共识层的可靠性界

第 2.6 节指出，意图层不存在机械解。那么它存在什么？本节给出一个精确的答案，并证明一个反直觉的结论。

定理 7.29 (共识可靠性界)**.** *设 n 个验证者对"某规格是否忠实表达意图"这一命题独立作出判断，每人判断错误的概率至多为 $p < 1/2$。则多数表决出错的概率满足*

$$\Pr\big[\text{多数出错}\big] \;\le\; \exp\!\Big(-2n\Big(\tfrac{1}{2}-p\Big)^{2}\Big). \tag{7.1}$$

证明. 设 X 为出错人数，$X=\sum_{i=1}^{n} X_i$，X_i 独立且 $\Pr[X_i=1]\le p$，故 $\mathbb{E}[X]\le np$。多数出错等价于 $X \ge n/2$。由 Hoeffding 不等式 [58]，对 $t>0$，$\Pr[X-\mathbb{E}[X]\ge t]\le \exp(-2t^2/n)$。取 $t=n(1/2-p)$（此时 $X\ge n/2$ 蕴含 $X-\mathbb{E}[X]\ge t$），得

$$\Pr\Big[X \ge \tfrac{n}{2}\Big] \;\le\; \exp\!\Big(-\frac{2n^2(1/2-p)^2}{n}\Big) \;=\; \exp\!\Big(-2n\big(\tfrac{1}{2}-p\big)^2\Big). \qquad \blacksquare$$

这一结论是陪审团定理 [59] 的定量形式。但它依赖独立性，而独立性是脆弱假设；其失效有一个同样精确的形式。

命题 7.30 (相关错误模式支配界)**.** *设 n 个验证者的错误具有如下结构：以概率 q 发生一个共同错误模式 C（例如共享的训练数据偏差、共享的领域误解、共享的利益激励），此时全部验证者同时出错；以概率 $1-q$ 各验证者独立出错，错误率 $p' < 1/2$。则*

$$\Pr\big[\text{多数出错}\big] \;\ge\; q. \tag{7.2}$$

特别地，当 q 有正下界时，增加 n 不能把多数出错的概率压到 q 以下。

证明. 当 $C=1$ 时，全部 n 个验证者出错，多数必然出错。故 $\Pr[\text{多数出错}]\ge \Pr[C=1]=q$。对 n 取极限不改变该下界。 $\blacksquare$

推论 7.31 (多样性而非数量)**.** *由命题 7.30，共识层可靠性的支配变量是错误的独立性（即 q 的大小），而不是验证者的数量。因此共识设计的核心目标是降低 q：引入不同背景的参与者、不同训练来源的模型、不同利益立场的审查方，以及显式设计的对抗性角色。*

注 7.32 (这一结论的用途)*.* 推论 7.31 给出一个可直接用于工程判断的准则：当有人提议"再加几个智能体来审查"时，正确的问题是"新增的审查者与现有的审查者在错误模式上是否独立"。若不独立，增加数量只增加成本，不增加可靠性。这也解释了为什么"用另一个同源模型来审查"在结构上不构成保证。

注 7.33 (共识模型是什么，不是什么). 定理 7.29、命题 7.30 与下文的设计效应近似，都是已声明的概率模型之内的陈述。它们不应被读作对真实审查过程可靠性的预测；模型与实践之间的差距值得被明确说出，因为本文框架最可能被误用的地方正在这里。

三点保留。第一，这些模型假设有一个共同待判断的命题、在 q 之外错误来源相互独立，以及每人能力固定。真实的审定活动三条全都违反：什么算作忠实的规格本身就在争议之中，审定者会通过交谈相互影响，而能力随所判断的条款而变化。关于陪审团定理的文献在很大程度上就是在研究这些前提有多么脆弱——尤其是指出：在任何单一的概率概念下，独立性与能力无法同时被证成 [79]——而共享的证据体是使表面的独立性失效的最自然的机制，因为它使审定者的错误共享同一个原因 [78]。后一个结果正是原则 7.35 所称信息同构的形式对应物。

第二，单一参数 q 一般并不能刻画相关性。两位审定者可以在某一条款上相关、在另一条款上独立，可以在某个子领域上因共享训练而相关、在另一个子领域上因共享激励而相关，而一维的汇总把这一切平均掉了。q 是界所需要的正确参数，同时是对该现象的有损汇总。

第三，也是后果最重的一点：除非" 忠实于意图" 有一个稳定的评价标准、一个被指定的裁决者，以及一份关于什么算作错误的说明，p 与 q 不能从数据中估计。在这些东西存在的地方——验收标准可观察的有界领域——这些量可以从结果中估计，这正是假设 H_8 与 H_{11} 所处理的情形。在它们不存在的地方，界仍然是一个条件性陈述，而再多的审定数据也补不上那个缺失的标准。诚实的总结是：若测量可能，模型告诉你该测什么；若测量可能，它也告诉你答案的形状。

注 7.34 (部分相关：设计效应). 定理 7.29 与命题 7.30 描述两个极端：错误独立，给出随 n 指数改善的界；共同错误模式，给出任何 n 都无法突破的下限 q。真实的审定网络位于两者之间，其错误在成对的层面上部分相关。标准统计工具在此适用，值得写下来，因为它使推论 7.31 成为定量的，而不只是修辞性的。

设 $\bar{\rho}$ 为审定者错误的平均成对相关。惯用的*设计效应*近似 [80] 把名义审定者数量替换为有效数量，

$$n_{\mathrm{eff}} \;=\; \frac{n}{1+(n-1)\bar{\rho}}, \tag{7.3}$$

并以 n_{eff} 代替 n 来读取定理 7.29 的界。由此得到两项后果。

(a) 一旦 $n\bar{\rho} \gg 1$，式 (7.3) 就给出 $n_{\mathrm{eff}} \to 1/\bar{\rho}$，*与 n 无关*。越过该点之后，再增加审定者什么也买不到；天花板只由相关性设定。

(b) 降低 $\bar{\rho}$ 会抬高该天花板，而这一取舍是显式的：一旦 $n\bar{\rho}$ 很大，把 $\bar{\rho}$ 减半大致可以把可达到的 n_{eff} 翻倍。多样性是一条预算线，不是一句口号。

它与命题 7.30 的关系是：q 是 $\bar{\rho} \to 1$ 的退化情形——完全相关时 $n_{\mathrm{eff}} \to 1$，式 (7.3) 复现了下限。

本条注附带两点保留。第一，式 (7.3) 是可交换相关下均值方差的标准近似，而不是关于多数错误的定理；要为中间区域给出严格的界，需要一个显式的相关模型，本文不声称给出这样的模型。第二——而这一点正是该近似可用的原因——$\bar{\rho}$ 可以从本文框架已经在收集的数据中*估计*出来：审定者错误的历史成对相关，以随后观察到的缺陷与返工来结算。这并没有回答测量 q 的开放问题 (O4)；它提供了一个可测量的伴随量，而 q 仍然是那个不可测量的极限。

7.12　意图层的激励结构

命题 7.30 把共识可靠性归因于参数 q，但没有给出 q 的估计方法。本节指出这一遗漏的性质：q 的主要来源是合谋与共同利益，因此它不是一个统计参数，而是一个激励设计问题。

原则 7.35 (q 的构成)**.** 共同错误模式可分解为三类来源：

(a) **认知同构**：验证者共享训练来源、领域假设或思维定式；

(b) **利益同构**：验证者的收益与”通过”相关，而非与”正确”相关；

(c) **信息同构**：验证者读取同一份被审查材料，故其误差由同一处缺陷决定。

三类来源对应三种不同的干预：引入异质参与者（对 (a)）、改变收益结构（对 (b)）、引入独立信息通道（对 (c)）。只做第一类干预是常见做法，也是效用最低的做法。第 7.13 节指出同一族中的第四种共现，权威同构，并解释它为什么不是本分解的第四项，而是一条独立的失效轴。

原则 7.36 (适当评分规则)**.** 对第二类来源，正解是让验证者的报酬与其判断的准确性而非其判断的倾向挂钩。适当评分规则 [70] 提供了现成工具：报酬函数在真实信念处取唯一极大值，从而对”迎合多数”与”讨好提交方”两种行为同时去激励。具体形式可以是一个针对”该规格是否忠实表达意图”的预测市场，其结算依据是后续观察到的缺陷与返工数据。该机制有两个副产品：

(a) 它为降低 q 提供了除”换人”之外的手段；

(b) 它给出 q 的后验估计，测量验证者误差的历史相关性即可。

这一节补上了第 7.11 节留下的缺口。那里的界是条件性的，其条件（独立性）在此被归约为一个可设计、可测量、可付费的量。第九章把它写成假设 H_{11}。

7.13　谁可以审定：道义轴

第 7.11 节与第 7.12 节把审定当作一个认识论问题：规格是否忠实于意图，以及说”是”的多数有多可靠。还存在第二个问题，而本文框架自己的保证条件式已经包含了它，只是没有把它接上。式 (2.12) 的第二个合取项要求契约已明确且被授权采用。授权不是可靠性问题：一份规格可以被一个规模庞大且真正独立的专家组忠实地审定，同时仍然是无人具备采用资格的那一份。

原则 7.37 (授权是一条独立的失效轴)**.** 审定网络的可靠性关乎其裁决是否很可能是正确的。授权关乎该裁决是否决定得了任何事情。两者相互独立。

(a) 一个裁决可以既可靠又未被授权：一个完全来自同一利益相关者类别的、有能力的专家组，就另一类别有资格拒绝的一条需求达成一致。

(b) 一个裁决也可以既被授权又不可靠：单一责任主体在信息不足的情况下作出判断。

因此，q 的改善、n_{eff} 的改善或激励结构的改善，都无法修复一次授权失效。这种失效不是所有人一起错了，而是所有人加在一起并不是有权作出决定的那个机构。

这就是权威应当位于 q 的三类来源之旁、而不是其中的原因。原则 7.35 把共同错误模式分解为认知同构、利益同构与信息同构——审定者以同一种方式出错的三种途径。第四种共

现很容易命名，称为**权威同构**：全部审定者都来自同一个利益相关者类别。它确实会抬高 q，因为同一个类别往往既共享前提又共享利益。但降低 q 并不解决它。一个完全多样化的外部人专家组，恰恰是授权失效最干净的情形：他们可能是对的，却仍然没有资格。

这一区分有一种用旧了的制度形态，值得用一句话、也只值得一句话。分布式账本系统把*确立什么是真的*的机制，与*确立谁可以决定什么算作真的*的机制分开；共识与治理是不同的协议，一个共识极佳却没有治理的系统并不因此就是正当的。这个类比只是作为类比给出的。它指向的是本文的一个缺口，而不是本文的一个结果。

注 7.38 (对意图层设计的两项后果). (a) 第 7.11 节的审定记录，应在来源之外同时携带每位审定者的*利益相关者类别*。这多花一个字段，使权威同构变得可观察，并使授权失效在事后可以与可靠性失效区分开来——两者的补救手段不同。

(b) 被授权机构的选择不能从本文框架中推出。它是关于”谁承担决定的后果”的制度事实，并且由与第八章把 L_∞ 指派给人来承担的那同一个论证所确定——而那个论证的前提是现行的法律责任制度，正如该章明确指出的。

7.14　规格绑定

式 (2.12) 的第四支柱把已验证的制品绑定到实际执行的制品。到此为止还没有任何东西把规格绑定到它本应表达的那个意图，而这种不对称是要紧的，因为两者随时间的行为相反。一个二进制要么逐字节相同、要么不相同：它的绑定是静态的，并且一次判定即定。而规格所要忠实的是一个会移动的意图。命题 7.12(i) 就是原因——需求是生成性的，偏好由规格写定时尚不存在的情境激活——所以忠实性不是规格所拥有的一种性质，而是一个会衰减的量。

表 7.4 已经要求每个制品携带第 10 项，即重新验证规则：哪些变化触发重新计算受影响的证据。规格一侧没有对应的东西，而命题 7.42 的条件 (i) 被陈述得好像它不需要任何对应物。修法是让规格自己携带它的绑定记录。

定义 7.39 (规格绑定)**.** 一个**规格绑定**是一个四元组

$$\mathrm{SB}(\Phi) \;=\; \big(\text{规格版本},\ \text{审定记录},\ \text{有效视界},\ \text{重新审定触发条件}\big),$$

其中审定记录是第 7.11 节的共识制品，它在规格写定时确立了 Φ 的忠实性；有效视界是该审定被当作仍然现行的那个区间；重新审定触发条件则是在视界到期之前使其失效的那些事件。

触发条件才是有意思的部分，而且它们不是随意的。命题 7.12 命名了三种漂移来源，每一种都提示出自己的触发条件。

(a) *生成性需求*：出现了新的一类用户、场景或部署环境，而这恰恰是激活了无人曾表达过的偏好的那种情境。

(b) *有损压缩*：写 Φ 所用的领域模型、本体或受控语言发生变化，以致同一段文字所指的东西变了。

(c) *确认*：一起被观察到的事故，其处置无法从 Φ 推出，这是 Φ 对意图表达不足、而不是实现出了错的证据。

触发条件 (c) 值得单独指出，因为它是免费得到的：一起需要人来作出判断的事故，按构造就是规格没有解决的事故。

因此，命题 7.42 的条件 (i) 应当带着一个时间下标来读。记 $\varepsilon_{\mathrm{cons}}(\Delta t)$ 为自审定起经过区间 Δt 之后的共识错误，

$$\Pr\big[\Phi \text{ 在时刻} t \text{ 忠实于} I\big] \;\geq\; 1-\varepsilon_{\mathrm{cons}}(\Delta t), \qquad \Delta t = t - t_{\mathrm{attest}}, \tag{7.4}$$

它在 $\Delta t = 0$ 时退化为原来的条件。本文不给出 $\varepsilon_{\mathrm{cons}}(\cdot)$ 的形式：不提出任何衰减模型，该量被留作一个可测量的对象，而不是被建模的对象。本节所确立的是结构上的对称性——规格需要一条重新审定规则，正如二进制需要一条重新验证规则——以及这一对称性是由命题 7.12(i) 逼出来的，而不是为了完备而添上的。

注 7.40 (制品绑定缺口的规格一侧对偶). 假设 H_9 关乎制品绑定缺口的可测量风险。它的规格一侧对偶是" 依据一份陈旧但仍被审定的规格执行" 的风险：审定记录仍在有效视界之内，二进制仍与其摘要相符，而两者一起错了，因为意图移动了。这两个缺口相互独立，一次部署可以堵住其中一个而让另一个敞着。

7.15　综合保证条件

把本章各部分组合起来，得到一个完整的、条件性的保证陈述。

定义 7.41 (双层信任结构). 从意图到执行的链条是一个双层信任结构的复合体：

$$\underbrace{I \xrightarrow{\text{共识}} \Phi}_{\text{不可形式化层}} \longrightarrow \underbrace{\Phi \xrightarrow{\text{机械}} P \xrightarrow{\text{机械}} B \xrightarrow{\text{机械+监控}} E}_{\text{可形式化层}} \tag{7.5}$$

在可形式化层，信任由机械验证器保证；在不可形式化层，信任由对抗性共识保证；两层之间的边界、共识的记录以及整个链条的可追溯性，由密码学锚定的基础设施机械保证。

命题 7.42 (综合保证：充分条件). 设意图 I、已审定规格 Φ、候选实现（二进制）B、运行环境 E，A 为声明的环境假设。若下列条件同时成立，则 " 在所声明的范围 Σ 内满足性质 φ" 成立：

$$\textit{(i)}\quad \Pr\big[\Phi \text{ 在时刻} t \text{ 忠实于} I\big] \;\geq\; 1-\varepsilon_{cons}(\Delta t); \tag{7.6}$$

$$\textit{(ii)}\quad \mathrm{Traces}(B,E) \;\subseteq\; \mathrm{Allowed}(\Phi); \tag{7.7}$$

$$\textit{(iii)}\quad E \models A; \tag{7.8}$$

$$\textit{(iv)}\quad \mathrm{digest}(B_{deployed}) = \mathrm{digest}(B_{verified}); \tag{7.9}$$

$$\textit{(v)}\quad \varphi \text{ 在所用的解释关系} R \text{ 下被保持}; \tag{7.10}$$

$$\textit{(vi)}\quad \Phi \text{ 由一个具备采用资格的机构授权}, \tag{7.11}$$

其中 $\varepsilon_{cons}(\cdot)$ 是式 (7.1) 的界，由 q 与 $\bar{\rho}$ 决定，并在 $\Delta t = t - t_{attest}$ 处取值。

这一陈述是刻意单向的。它在别处曾被写成等价式，而那是错的；其理由值得说出来，因为只要提出这种形状的框架，这个理由就会重现。这六个条件是证据条件：它们说的是手上必

须有什么。它们不是真值条件，而真值并不等待它们。一条性质可以在没有任何证据被汇集起来的情况下成立，所以反方向之所以失败，理由是最基本的那一条：一个系统可以是正确的却没有留下文档。本命题所断言的是有用的那个方向——六条全部成立就足够了——而它刻意不断言的是：少于六条就不可能。

每个合取项都带有自己的保留，而它们没有一个与其余各项在相同的意义上属于机械性的。

(a) 条件 (i) 是一个概率，在忠实性被定义、裁决者被指定、错误类别被说明之前，它没有自然的定义。在这些东西存在之前，ε 是模型的参数，而不是测量到的频率。它的可靠性受 q 以及第 7.11 节与第 7.12 节的相关 $\bar{\rho}$ 支配，而两者都是条件模型的量。

(b) 条件 (ii) 是一条语义性质，不是例行检查。$\mathrm{Traces}(B, E) \subseteq \mathrm{Allowed}(\Phi)$ 只有在受限语言、有限模型或携带可检查证书的程序上才是直接可判定的；对一般环境中的一般程序，它是一项证明义务，靠类型化、模型检查、契约验证与不同强度的测试组合来清偿——前提是它真被清偿了。

(c) 条件 (iii) 与 (iv) 是那对机械性的条件，其强度受可信计算基与环境模型限制，而不是受本章中任何东西限制。

(d) 条件 (v) 正是注 2.5 的关系性前提被花掉的地方。

(e) 条件 (vi) 是道义性的，根本不是可靠性问题（第 7.13 节）。

因此，定义 2.24 的六个合取项中，三个是机械性的，一个是概率性的，一个是道义性的，而还有一个——契约必须明确这一要求——正是本章其余部分的用途所在。

综合陈述还必须携带另一个区分，即经典的安全性（safety）与活性（liveness）之分。条件 (ii) 约束的是有限轨迹：被禁止的事情不会发生。进展性质——一个请求最终得到应答、一把锁最终被释放、一个队列最终排空——是活性性质，它们不是” 没有有限前缀违反 Φ” 的实例，也不能由同一个论证得到。当系统的契约包含活性时，这些义务属于能够表达它们的层级与装置：对协议模型是公平性假设与良基性论证，对已部署的系统则是带界视界的监控。一份把活性悄悄吸收进轨迹包含的综合陈述，会以一种容易被忽略的方式出错，而本陈述没有这样做。

式 (7.5) 与命题 7.42 一起给出了对问题 1.4 的完整回答：

原则 7.43 (对问题1.4 的回答)**.** ” 人不再读代码” 是可达的；” 人不必再为目标的表达负责” 是不可达的。可行的分工是：人确立并承担目标与约束的责任；生成器负责大规模生成、搜索与优化；机械化的验证与执行边界负责判断与限制实现；系统始终携带可追溯的保证证据。

问题	本文的判断
Q1 能否证明意图与规格一致？	不能依靠通用自动方法。受控语言、领域本体、例子与人工审定可缩小问题范围，但无法覆盖未表达的隐含需求（命题 7.12）
Q2 如何证明规格充分？	可检查一致性、可满足性、可实现性、空洞成立与形式化覆盖指标；这些都不能证明规格包含了所有未声明的禁令

问题	本文的判断
Q3 局部保证如何推出端到端保证？	需要关系可复合、环境假设闭合、观察映射兼容（命题 2.4）；活性与实时性质通常还需额外条件
Q4 能否统一功能与资源保证？	可以在联合行为–成本语义中表达多种约束，但时间、内存、能耗与信息流需要不同的分析模型；联合表达不等于统一求解
Q5 如何降低对生成模型的信任？	生成器提出候选，独立检查器验证。这降低了对生成器正确性的依赖，但不消除对逻辑内核、定理陈述、外部假设与制品对应关系的信任要求
Q6 给定规格是否总能有效综合？	不能。有限语法、受限逻辑与特定模板可使部分问题可解；任意程序语义性质存在不可判定性与搜索空间爆炸的边界
Q7 实际延迟能否严格保证？	在特定硬件、输入、调度与成本模型假设下可得到可用的最坏执行时间上界；基准测试与渐近复杂度本身不足以证明纳秒级上界
Q8 是否需要统一理论？	更有根据的选择是建立共享的证明接口与证据格式，同时保留各领域不同的语义、假设与分析工具。统一形式体系可以是目标之一，但不是必要的先验条件

注 7.44 (中心问题 Q1–Q8 的结论摘要).

本章的经验内容是方向 1–3（第九章）与假设 H_7、H_8、H_{11}。命题 7.11 与 7.12 是论证而非定理，其可挑战之处在于三条根据是否穷尽。定理 7.29 与命题 7.30 是标准概率论结果的应用，其结论依赖独立性假设。

第八章　层级即验证义务的分解单元

前七章把层级分别用作理解工具（第五章）、设计工具（第六章）与保证工具（第七章）。本章说明这三者是同一个对象的三种用法，并论证其中最值得继续推进的方向：把层级从” 理解的辅助” 升级为*验证义务的分解单元*。

8.1　认知尺度

定义 8.1 (认知尺度)**.** 给定研究对象 S 与任务 T，S 的**认知尺度** $\mathrm{L}_0, \mathrm{L}_1, \mathrm{L}_2, \ldots$ 按” 该层回答什么类型的问题” 划分，而不是按代码的物理位置划分：

L_0 **计算原理** 什么在计算？什么约束了它?：内存、指令执行、函数调用、并发、同步、虚拟内存、I/O、缓存、网络、持久化、类型与内存模型。

L_1 **基础设施与运行时** 系统提供哪些公共能力？如何保证它们正确协作?：分配器、任务调度器、事件循环、对象系统、执行器、存储引擎接口、资源管理器、生命周期机制。即第四章的六职能所在层。

L_2 **组件与子系统** 有哪些职责？边界在哪里？它们如何协作?：事务子系统、渲染引擎、实体-组件系统、网络协议栈、复制模块、查询执行器。

L_3 **领域功能与业务语义** 系统对外承诺什么？什么行为才是正确的?：查询语义、游戏规则、布局规则、事务隔离语义、权限与业务不变量。

L_4 **及以上：场景与系统级行为** 整个系统在真实场景中如何表现?：端到端工作流、故障恢复、系统级性能、部署拓扑、业务目标。

L_∞ **目的与责任** 应当要什么？哪些取舍可接受?；这一层属于人，理由见第 8.4 节。

原则 8.2 (认知尺度不是物理层级)**.** 定义 8.1 的层级代表*认知尺度*，而不一定是代码的物理层级。游戏引擎的 L_3 领域功能可能直接调用 L_1 的运行时接口；数据库的事务语义也可能对存储层提出约束。

原则 8.3 (层级的相对性与递归性)**.** 同一个组件在不同研究任务中可以处于不同层级。一个异步运行时对普通应用开发者是基础设施（L_1），对其自身开发者则是需要深入修改的复杂系统（即它自己有一套完整的 $\mathrm{L}_0 \ldots \mathrm{L}_4$）。因此层级是相对于研究对象与当前任务定义的。

原则 8.3 有一个重要推论：*每个大型系统内部都有一份完整的层级阶梯*，这份内嵌阶梯与外部的技术栈阶梯是两回事。这直接解释了第四章的结论：每个大系统都在自己的域内长出一个” 小操作系统”，而这正是它的内嵌 L_1。

注 8.4 (与因果层级的区别)**.** 本文的层级与因果层级 [66] 有形式相似之处：对低层的干预不自动传递为高层的干预，正如对实现层的修改不自动成为对契约层的修改。但二者不同：因果层

级区分的是查询类型（观测、干预、反事实），本文的层级区分的是可观察域与保证义务（定义 2.1）。混同二者会导致一个具体错误：以为” 能在高层提问” 就等于” 能在高层干预”，而后者需要前者并不提供的接口。

8.2　层契约

定义 8.5 (层契约四元组)**.** 一个层级要被承认，必须能写出四元组

$$\mathrm{LC}(\Lambda) \;=\; \big(\text{参与者},\ \text{保证的不变量},\ \text{契约形式},\ \text{验证方式}\big), \tag{8.1}$$

以及本文额外要求的第五项，即**无视集** $\mathrm{Obl}(\Lambda)$（定义 2.8），即该层被允许永远不关心的内容。

原则 8.6 (层级的真实性判据)**.** 一个层级是真的，当且仅当它能写出定义 8.5 的五项。写不出第五项（无视集）的” 层级” 不是层级，只是一个目录或一个命名空间。

把定义 8.5 应用到表 8.1 给出的实例，可以看到同一组问题在不同层级上的具体形态。

表 8.1: 层契约四元组在一个关系数据库引擎上的实例化

层	参与者	保证的不变量	契约形式	可无视
L_0	页、缓存行、指令	局部性原理；缓存行的原子性	硬件时序与内存模型	上层语义；事务概念不存在
L_1	缓冲页、日志记录、锁、内存上下文	脏页在刷盘前不被复用；日志先于数据落盘	palloc 契约、缓冲区固定/释放协议	SQL 语义；查询计划形状
L_2	解析器、规划器、执行器、访问方法	执行器消费的元组满足当前快照的可见性	执行器接口（打开/取下一个/关闭）	存储的物理布局；日志格式
L_3	事务、语句、会话	隔离级别所声明的可见性与原子性	SQL 语义与隔离级别定义	缓冲区管理策略；索引结构
L_4	应用、运维、客户端	端到端工作流在故障下仍然可恢复	服务等级目标与恢复点目标	内部执行计划的形状

表 8.1 最有信息量的一列是最后一列。它把” 这个层级负责什么” 之外的另一半，即” 这个层级被允许不知道什么”，显式化了。这一列是大型系统设计文档中最常缺失、也最有价值的一列。

8.3　验证义务的分解

这是本章的核心主张。

原则 8.7 (层级的三重身份)**.** 同一套层级结构同时承担三种功能：

(i) **理解单元：**它规定了” 理解一个系统需要多少信息” 的下界，即各层商的复合（命题 2.9）；

(ii) **设计单元：**它规定了” 一次典型变更的读集” 的结构（定理 6.3）；

(iii) **验证义务单元：**它规定了哪些性质可以被机械保证、哪些必须由运行时监控、哪些必须留给人的责任划分。

(iii) 是既有讨论触及而恰好未展开的部分。

为什么层级天然是验证义务的分解单元？因为它同时提供了两样东西：一个可观察域（$\mathcal{O}_i$，即该层能看到什么）与一个不变量集合（$\mathcal{I}_i$，即该层必须保证什么）。而验证义务的形式恰好是” 在某个观察域上判定某个性质”，因此每一条验证义务都自然地归属于某个层级。

定义 8.8 (验证义务的层级归属)**.** 给定性质 φ 与层级 Λ，称 φ **归属于** Λ，若 φ 可以完全由 Λ 的观察域 $\mathcal{O}_\Lambda$ 表达。对 φ，其归属层为能表达它的最低层级。

由此得到三类义务的分派：

(a) **可在本层机械判定的：** φ 在 Λ 的观察域内可表达，且存在可穷尽的判定程序。归入机械保证。

(b) **可在本层监控但不可判定的：** φ 在观察域内可表达，但只能在执行时对已观测轨迹判定。归入运行时保证，并按原则 7.20 区分三个状态。

(c) **在本层不可表达的：** φ 需要跨层信息或涉及目的本身。归入人的审定或共识保证。

表 8.2: 保证类型在各层级上的分布（示例）

性质	归属层	保证方式	证据类型
内存安全	L_0–L_1	机械（类型系统 / 所有权 / 沙箱）	编译期检查、验证器
无越界与循环有界	L_0–L_1	机械（字节码验证器）	准入时静态证明
无未授权 I/O	L_1–L_2	机械（能力模型 / 效果类型）	编译期 + 运行时边界
功能后置条件	L_2–L_3	机械（Hoare / 精化类型 / 契约）	证明项、SMT 判定
资源上界	L_1–L_2	机械（资源分析）+ 环境假设	符号界、最坏执行时间分析
实时截止期	L_2–L_4	机械（在硬件模型内）+ 运行时	时序分析、监控
隔离级别语义	L_3	机械（模型检查 / 契约）	模型检查结果
端到端可用性	L_4	监控 + 共识	运行指标、服务等级目标
规格忠实于意图	L_∞	共识	对抗性审查、审定记录

表 8.2 是本章主张的具体化。它的实践含义是：对一个性质，先问” 它归属于哪一层”，再问” 该层能提供哪种保证”，最后问” 该保证的假设是什么”。这三个问题替代了” 我们是否需要形式化验证” 这类无法回答的问题。

8.4　责任主体

原则 8.9 (责任不可转移)**.** [**A**] 决定” 要什么” 与承担该决定的后果，是同一个主体性行为的两个侧面。因此 L_∞ 不属于机器，不是因为机器不够聪明，而是因为责任必须由一个能够承担后果的主体来承担。

强度标注为 [**A**]，而它所依赖的假设必须写明，因为若不写明，这一论证就是本文中唯一一处完全不携带前提的地方。前提是：*在现行的法律责任制度之下*。这一论证所用的不是人的某种形而上学属性，而是一种制度属性——被问责、被起诉、被制裁或以其他方式被要求作出交代的能力。这种能力由法律与约定构造出来，而不是由生物学给定：法律人格在历史上既被扩展过也被褫夺过，而今天它已被扩展到非自然人。如果某一制度把法律责任归于一个人工主体，前提就会改变，结论也必须重新推导。在这一改变中存活下来的是那个结构性要点：无论什么占据 L_{∞}，它都必须是一个能够承担后果的东西，而哪些东西属于此类，是由制度回答的，不是由本框架回答的。

但这一原则需要按第 7.11 节的结果精确化。人的残余职责*不是个人的，而是集体的*：不是” 一个人审阅规格”，而是” 一个认知网络通过对抗性辩论达到对规格的共识”。这个网络可以包括多个不同背景的人类专家、不同训练来源的模型，以及显式设计的对抗性角色。

原则 8.10 (多样性不可替代)**.** AI 时代人的不可替代性，不在于人的个体判断比 AI 更好，而在于人是认知网络中不可还原的*多样性来源*。由推论 7.31，共识可靠性由错误模式的独立性支配；而独立性的最终来源是*不同的问题形成历史*。因此人的价值在于其视角的多样性，而不在于其计算的精确性。

8.5　层级语言的错配

原则 8.11 (层级语言一致性)**.** 读代码时最大的错误，是用错层级的语言理解当前层级。

例 8.12 (具体形态)**.** (a) 用指针分析的方式理解业务校验逻辑，把” 这笔交易是否允许” 的问题当成内存别名问题。

(b) 用业务语义理解缓冲区管理，试图在页面替换策略中寻找商业规则。

(c) 用调度器的公平性语言理解数据库隔离级别：二者都是并发问题，但前者的正确性判据是吞吐与延迟分布，后者是可见性与原子性。

(d) 用类型系统解决多租户隔离，因为隔离是横切关注点（原则 6.22），类型系统只能表达其中的一部分。

形态 (d) 还有第二种读法，而且它是更锋利的那一种。类型系统约束的是该层级*已声明*的通道——即它能够命名的数据通路——而它无法命名的那些通道（分配时机、缓存占用、错误行为）仍然敞开。这不是充分性的失败（差分测试会抓住那种失败），而是完备性的失败：所声明的观察映射并不等于可观察之物的全部（原则 6.11、注 2.12）。按这种读法，(d) 并不是” 为一个层级选错语言” 的例子；它是一个层级的无视集不真实的例子，而再多的类型层表达能力也修补不了它。

反模式例 8.12(d) 的形态更严重：它不只是” 用错语言”，而是试图把横切关注点压入层级结构。由原则 6.22，这种尝试在结构上注定失败，而其失败常被误诊为” 分层不够彻底”。

本章的经验内容是定义 8.8 的可复现性：不同分析者是否对同一性质给出相同的层级归属，以及表 8.2 的分派是否减少” 应当机械保证的性质却只靠人工审查” 的漏项。以下降的漏项率作为指标，该主张可被检验。

第九章　可证伪的研究纲领

本文的全部主张若不转化为可检验的形式，就只是一组解释框架。本章给出这一转化，其做法遵循研究纲领方法论中”硬核与保护带”的区分 [65]：不可承重的主张应被移入保护带，而不是加固为硬核。

9.1　方法论：从解释框架到可检验假设

原则 9.1 (降格程序)**.** 对每一条主张，执行下述操作：写出它的可观察后果；写出什么观察会否定它；写出获得该观察的最小实验；写出该实验的对照条件。凡是无法完成这四步的主张，应当被明确标注为解释框架而非经验断言。

按原则 9.1，本文的主张被分为三类：

- **已证明的**：命题 2.9、3.5、2.20、6.8、2.4、5.8、7.17、2.29、4.17、5.10、6.16；定理 4.4、4.9、6.3、7.29；命题 7.30；推论 4.5、4.12、7.31、7.8。
- **已论证但依赖明确假设的**：原则 1.5、1.6、1.7、1.8；命题 7.11、7.12。
- **作为可证伪假设提出的**：下述 H_1–H_{14}。此外，注 4.19 的结构提案（把层族提升为严格纤维化）与注 4.23 的重整化类比均未被计入定理，其地位是提案与启发式。

9.2　十四条可证伪假设

假设 9.2 (结构收敛高于随机基线)**.** 在控制系统规模、领域、平台、编程语言与框架之后，来自不同领域的大型软件系统在其抽象骨架（定义 2.16）上呈现的结构相似性显著高于随机基线。操作化：以第 3.6 节的五步设计构造架构图集合与骨架映射，以第三节的结构指标为特征向量，检验跨领域配对的相似性分布是否随机优于此零模型（如度序列保持的图重连）。证伪条件：相似性分布与零模型无显著差异。风险：骨架映射的选择自由度；需预注册映射与指标。

假设 9.3 (收敛强度关于领域绑定度单调)**.** 猜想 3.10 成立：结构相似性关于领域绑定度 Dom（定义 3.9）单调递减。操作化：对每个层级标注 Dom（由该层语义所依赖的领域公理占比估计），回归相似性对 Dom。证伪条件：单调性不成立或系数不显著。

假设 9.4 (相变阈值由 $n \times m$ 决定)**.** 系统是否建立统一基座，由活跃实体数与共享资源数的乘积（以及重复问题频率与演化频率）预测得比代码行数更好。操作化：对一组开源项目，人工或半自动标注（是否出现六职能中的自建实现），以 $n \times m$ 的代理量（并发实体数量级、共享资源种类数）与代码行数分别做逻辑回归，比较解释力。证伪条件：代码行数的解释力不劣

于 $n \times m$。风险：n 与 m 的自动估计困难；可先用并发实体与互斥量/连接池/缓存等的数量作为代理。

假设 9.5 (读集的标度优于传统规模指标)**.** 变更读集 $\mathbb{E}[\rho]$（定义 6.1）关于系统规模的标度指数 α（命题 6.16）与缺陷率、变更失败率的相关性，优于模块数、代码行数、圈复杂度等传统规模指标，也优于读集的当前水平 $\mathbb{E}[\rho]$ 本身。操作化：在若干有完整提交历史与缺陷追踪的开源项目上，按时间窗口同时估计读集的水平与标度，分别与缺陷密度做面板回归，比较两者的解释力。证伪条件：标度不提供超越水平与既有指标的预测力。

假设 9.6 (分层模型加主动探测优于穷尽阅读)**.** 在单位成本（token 预算或人时）约束下，采用分层认知模型加主动探测的理解过程，其任务成功率（以定义 5.3 的 (U1)–(U5) 为评分标准）优于穷尽式阅读。操作化：给定同一陌生代码库与同一组任务（预测影响面、定位缺陷、添加功能），比较三种策略：顺序阅读、仅层次摘要、层次摘要加探测循环。测量任务成功率与单位成本成功率。证伪条件：主动探测策略不显著更优，或优势完全来自探测次数而非策略结构。

假设 9.7 (泄漏对可被自动发现)**.** 命题 6.8 的泄漏对可以被自动化差分测试发现，且发现率与模块边界质量负相关。操作化：对若干库，生成" 接口相同、内部状态不同" 的执行对，检测外部行为分歧；与人工标注的边界质量评分比较。证伪条件：自动生成的执行对无法有效触发泄漏，或发现率与边界质量无关。

假设 9.8 (规格质量是当前的约束瓶颈)**.** 在当前技术水平下，限制" 意图到可验证执行" 的主要瓶颈是规格质量，而非代码生成能力或证明搜索能力。操作化：以既有基准证据（规格生成任务、代码正确率与规格质量分离评测、证明成功率）为初始证据，并在新的受控基准上比较三个环节的改进对端到端成功率的边际贡献。初始证据：在 189 个 Lean 任务上的分项评测报告最佳模型的代码正确率 72.6%、规格健全性与完备性 52.3%、证明成功率 4.9%（单次试验）[8]；245 个操作系统内核规格生成任务的评测显示模型在长上下文与规格完备性上仍显不足 [3]；以可信测试实例化规格的评测指出证明难度会干扰对规格质量的评价 [4]；可追溯精化方法报告在规格生成子任务上的准确率提升 [5]；变异驱动方法报告精确率与召回率 [6]。这些是特定基准上的实证结果，不是通用可靠性定理，但它们支持" 三个维度应分别评估" 这一判断。证伪条件：在控制其他环节能力后，规格质量的边际贡献不显著高于其他环节。

假设 9.9 (共识可靠性受相关错误模式支配)**.** 在意图审定任务上，增加验证者数量对可靠性的提升受相关错误模式 q 的上限约束（命题 7.30）；引入异质验证者（不同背景的人、不同来源的模型、对抗性角色）对可靠性的提升优于单纯增加同质验证者。操作化：构造意图审定任务，配置若干组验证者（同质/异质），测量多数判断的错误率随组大小与异质性的变化，并与式 (7.1) 与 (7.2) 的预测比较。证伪条件：同质验证者的错误率随 n 下降的速率与异质验证者无显著差异（即 q 实际上很小）。

假设 9.10 (制品绑定缺口是可测量的风险)**.** " 被验证的工件" 与" 实际执行的工件" 之间的绑定缺口（式 (2.13)）在真实部署中是可测量且可显著降低的风险，且其降低带来可观测的可靠性收益。操作化：在若干部署流水线上引入内容寻址、可复现构建与部署时校验，测量构建不

确定性比例、部署漂移事件率与相关事故率的变化。*证伪条件*：缺口在实践中不可测量，或修复成本远超收益。

假设 9.11 (六职能是粗粒化下的存活集)**.** 在定义 4.20 的粗粒化序列下，存活的结构恰为 $\mathcal{K}_6$ 的槽位及其关系型（猜想 4.21）。*操作化*：按原则 4.22 对若干仓库实施机械粗粒化，记录存活集随粒度的变化。*证伪条件*：存活集包含六职能之外的结构，或六职能中的某些在中等粒度上即被冲掉。*风险*：粗粒化的阈值选择会引入人为参数；应对阈值做敏感性分析。

假设 9.12 (激励结构降低相关错误模式)**.** 在意图审定任务上，采用适当评分规则（原则 7.36）相比仅增加验证者数量，能更显著地降低多数判断的错误率，并可测得错误相关性的下降。*操作化*：对同一组规格审定任务，比较三种配置：少量无激励验证者、大量无激励验证者、少量带评分规则验证者；测量错误率与验证者间误差相关系数。*证伪条件*：评分规则配置的错误率与误差相关系数均不优于大量无激励配置。

假设 9.13 (率失真斜率预测重构收益)**.** 率失真曲线的斜率 $\partial L_T/\partial \ell$（注 6.14）能预测哪些重构在实际中降低缺陷率或返工率。*操作化*：对一组已完成的重构，事前估计其沿曲线的位移方向与斜率，事后以缺陷率与变更失败率的变化作为结果变量做回归。*证伪条件*：斜率不提供超越" 改动规模" 等朴素变量的预测力。

假设 9.14 (单位代码量的证明成本轨迹)**.** 第十一章并不主张形式化验证会*自动*变便宜，而是要求一个可测量的成本指标。该假设是：比值

$$\kappa_{\text{proof}} \;=\; \frac{\text{确立一项已陈述保证所需的工作量}}{\text{承载该保证的代码规模}}$$

处在一条可测量的轨迹上，并且决定机械化验证会变成常规工程、还是仍被局限在安全关键工作中的，是这条轨迹——而不是它的当前取值。*初始证据*：seL4 的功能正确性证明据报告约用十一个人年，对应约 8,500 行 C 代码，即约每千行 1.3 人年，且不含对可复用工具与框架的投入 [25]；把该投入计入的数字要大好几倍。这两个数字之间的差距值得点明，因为可复用工具所占的那一份，恰恰是该假设成立时应当下降的部分。*操作化*：对固定的性质类与固定的证明器，按项目、按年度记录 κ_{proof}，把可复用基础设施所占份额与单项目所占份额分开；在观测窗口上拟合这条轨迹。*证伪条件*：测不出趋势，或者趋势完全归因于可复用基础设施份额、而单项目份额持平。由于本文拒绝预测它朝哪个方向走，两种结果都是有信息量的。*风险*：分母是有争议的——代码行数、规格行数还是证明行数——这一选择必须预注册，否则轨迹在不同项目之间不可比。*解释区间*：为读取这条轨迹而按惯例提出的阈值，例如" 常规" 的 0.1 与" 专家" 的 0.5–1，是约定的判定区间而非测量量，本文在此照此标注它们。

假设 9.15 (智能体式生成能否降低读集的标度指数)**.** 命题 6.16 使读集的标度指数 α 成为决定一个系统的治理成本随其规模线性增长还是二次增长的量。上述每一个假设涉及的都是由人、或由人指挥的工具所构建的系统。这一条直接问出 AI 时代的问题：一个智能体式生成器在被允许重构的条件下，能否*发现* α 低于人类基线的架构，还是它只是在所继承的架构内部移动常数？这一区分不是修饰性的——它是" 加速工作的自动化" 与" 改变规模上仍然可能之事的自动化" 之间的区别。*操作化*：让一个智能体组与一个人工对照组从同一代码库、同一功

能待办清单、同样的重构自由度出发；在同一窗口内跟踪 $\mathbb{E}[\rho]$ 与拟合出的 α 关于系统规模的变化；比较两条轨迹。证伪条件：α 在两组之间无显著差异，或者表面差异完全由所写代码的数量而非其结构解释。风险：α 是通过定义 6.1 的读集代理估计的，而该代理的有效性本身就是假设 H_4；H_4 的代理若不成立，会使本假设变得不可证伪而不是为假，两者不可混为一谈。

表 9.1：假设与实验设计汇总

编号	假设	主要测量	主要风险
H_1	结构收敛高于随机基线	骨架相似性 vs 零模型	骨架映射选择自由度过大
H_2	收敛强度关于领域绑定度单调	相似性对 Dom 的回归	Dom 难以独立标注
H_3	阈值由 $n \times m$ 决定	逻辑回归解释力比较	n、m 的自动估计
H_4	读集标度优于传统规模指标	读集标度与缺陷密度的面板回归	读集代理精度
H_5	分层加主动探测优于穷尽阅读	单位成本任务成功率	任务与评分的主观性
H_6	泄漏对可被自动发现	差分测试发现率	触发有效执行对的难度
H_7	规格质量是瓶颈	三环节的边际贡献	基准的真值与覆盖
H_8	共识受 q 支配	同质/异质验证者错误率	异质性的构造与度量
H_9	制品绑定缺口可测量	构建不确定性与漂移事件率	真实部署的可得性
H_{10}	六职能是粗粒化存活集	粗粒化序列下的存活集	粗粒化阈值的人为性
H_{11}	激励结构降低相关错误	错误率与误差相关系数	激励机制的实验伦理与成本
H_{12}	率失真斜率预测重构收益	重构前后的缺陷率回归	斜率的可估计性
H_{13}	单位代码量的证明成本呈可测量轨迹	κ_{proof} 的拟合轨迹，工具份额单列	分母有争议；跨项目可比性
H_{14}	智能体式生成可降低读集标度指数 α	拟合 α 关于系统规模的两组轨迹	依赖 H_4 的读集代理

9.3　三个优先研究方向

以下三个方向分别连接需求工程与形式化方法、编译正确性与资源分析、程序契约与运行时安全。它们的共同特征是：都有明确的精确问题、可证伪假说、检验方法与主要风险。

方向一：面向真实意图的规格质量证据链

精确问题。　给定自然语言需求集合 R、领域模型 D 与生成规格 S，如何构建独立评估流程，分别检查：需求覆盖；禁止行为排除；合法行为保留；环境假设显式性；规格的可满足性与可实现性？

已有基础。　需求追溯研究、需求形式化与验证、空洞性检测、以及近期的规格生成与联合评测基准。关键连接是需求工程的语义追溯与形式化方法的规格与程序反例。

可证伪假说。　与单一”证明通过”或单一”参考规格相似度”相比，原子需求追溯加正反向测试加规格强度与变异检查加独立领域审查，能更早识别规格的过弱、过强与意图遗漏（即假

设 H_7 的具体化）。

检验方法。 在支付、权限控制与幂等接口等小型基准上，建立由多利益相关者标注的需求原子、允许与禁止轨迹集以及环境前提；比较四类方案：纯生成、需求分解、追溯测试、变异精化加专家辅助。测量精确率与召回率、错误类别、漏掉的禁例率、过度约束率、专家一致性与成本。

证伪条件。 若加入追溯与变异层并未显著降低隐藏负向要求的漏报率，或收益只来自泄露的测试参考答案，则假说不成立。

主要风险。 真实意图的”金标准”本身可能存在争议。

方向二：面向实际制品的功能–资源联合翻译证书

精确问题。 给定源程序 P、功能契约 φ、资源预算 B 与固定目标平台模型 H，能否为每次编译生成证书 C，使独立验证器同时检查语义保持与可分析的资源约束？

已有基础。 经验证编译器的语义保持证明、逐次翻译验证、翻译认证、证明携带代码、自动摊还资源分析与最坏执行时间分析。实质连接是把目前分别验证的”行为关系”与”成本关系”置于带成本的行为语义中。

工作模型。 程序语义扩展为二元组 (τ, c)，其中 τ 是可观察迹线、c 是成本；翻译证明应保证目标迹线可由源语义解释，并在明确成本模型下证明 $c_{\text{target}} \leq B$，或给出相对于源成本的保守关系。

注 9.16 (一个必须避免的假设). 不能预设优化编译后的成本一定单调下降。优化可能提高平均性能而恶化最坏情况，因此成本关系必须是双向可比较的保守关系，而非单调性假设。

检验方法。 选定可复现的子语言与目标架构，比较三条路线：经验证编译、翻译验证、普通编译加事后验证。记录证书大小、验证时间、程序类别覆盖、资源上界紧致度、误拒率以及对编译优化变化的稳健性。

证伪条件。 若资源证明在真实平台校准后无法保持可靠，或证书生成与验证成本远高于基线且无安全收益，则应缩小到特定安全关键领域。

主要风险。 目标处理器微架构与调度模型不够精确。

方向三：不可信组件的组合契约与运行时保障

精确问题。 当一个服务由已验证组件、未完全验证的生成组件与外部服务组成时，如何用假设–保证契约、信息流策略、资源预算与运行时监控，给出可审计的整体安全命题及故障影响上界？

已有基础。　模块化假设–保证推理、动态系统契约组合、信息流非干扰、已验证内核与运行时验证的不确定性研究。实质连接是把子组件的形式化契约与运行时对契约前提与保证的检测及强制执行放到同一个系统论证中。

可证伪假说。　当所有组件都有显式假设、出入口事件可观测、且违反契约可由独立监控触发隔离或回退时，能在不证明每个组件内部实现正确的条件下，为指定安全性质给出可组合的条件性保证。

检验方法。　搭建多组件支付流程：请求幂等服务、身份权限服务、生成式风险决策组件、外部数据库与网络模拟器。对比三组（无契约、静态契约、契约加监控与回退）在故障注入下测量重复扣款、跨用户数据泄露、检测延迟、隔离成功率与性能代价。

证伪条件。　若监控所能观测的事件不足以判断关键性质，或回退机制自身无法保持安全，则架构不能提供宣称的保证。

主要风险。　对于未观测的微妙语义性质，运行时保障仅能约束可检测与可强制执行的部分。

9.4　六阶段实施路线

表 9.2: 研究实施路线

阶段	核心任务	交付物
1	定义基准与威胁模型：选择支付服务与幂等接口作为贯穿案例；明确资产、角色、可观察事件、禁止行为、环境假设与时限	需求原子集、正负轨迹集、环境假设清单（经利益相关者审定）
2	规格质量评估：比较纯生成、追溯精化、变异与反例驱动方案；检查过弱规格、过强规格、空洞成立、不可满足与不可实现	规格质量评估器；每个需求标识对应规格条款、测试与审定记录
3	有限语言程序综合：选择明确子语言与判定性可控的逻辑；分离候选生成与证明检查	候选程序、证明义务、反例分类；区分不可满足、未知、超时与不可表达
4	制品与资源证明：选择成熟编译或翻译验证路径；指定可复现实验硬件或仿真模型	编译链元数据、语义保持或翻译证书、资源上界与模型边界
5	组合与运行时保障：为外部服务与未完全验证模块写假设–保证契约；添加独立监控、沙箱、审计与安全回退	系统级契约组合结果、故障注入报告、剩余风险清单
6	端到端评价：测量保证覆盖、漏报、误报、开发成本、证明成本、制品验证成本、资源预算与失效恢复	一个能证伪研究假说的公开基准、工具原型与实验报告

9.5　成功判据

以下判据评估该路线是否成功。它们被设计为可独立检验的形式：

(S1) **对象明确**：每一段保证的对象（哪个工件、哪个语义、哪个环境）可以被唯一标识。

(S2) **假设真实**：所依赖的环境假设可以被检验，而非被默认。

(S3) **证据独立**：证据的判定不依赖生成者的自我评价。

(S4) **失败类别可区分**：不可满足、未知、超时、不可表达与候选被反例否定必须被分别报告，不能合并为一个” 失败”。

(S5) **成本可测量**：生成成本、证明成本、验证成本与运行成本可以被分别计量。

(S6) **可证伪**：至少有一条 H_i 的否定会导致该纲领的核心主张被削弱。

注 9.17 (纲领的自我检验). 本文的一个核心主张是” 生成不可信、验证须独立”。该原则同样适用于本文：本文的承重命题必须可被独立检验，且不得依赖任何未证明猜想。注 7.44 与第十一章的不作主张清单，是这一自查的具体结果。

9.6　最小可行子集与采纳风险

上述假设共享一个它们都没有覆盖的失败模式。它们各自会被实验否证或确证，而本框架还可能干脆没有被采纳，此时根本不会有实验被运行。这一风险值得明确写出，因为它与可证伪性正交，也因为它有具体的形状：形式对象是有维护成本的——声明必须被写出、保持更新并被辩护——如果这个成本超过收益，这套实践就会死于被忽视，而不是死于被否证。

原则 9.18 (最小可行子集). 如果本框架中只有三个要素能够被采纳，它们应当是：

(a) **无视集声明**：每个层级或模块声明它被允许永远不必关心什么（定义 2.8）；

(b) **泄漏对测试**：对每一条被声称的边界，用差分测试寻找两条接口轨迹相同而外部行为不同的执行（命题 6.8、推论 6.9）；

(c) **验证义务的层级归属**：每个性质被分派给能够表达它的最低层级，这一分派决定它是机械可判定的、可监控的，还是需要审定的（定义 8.8）。

这三者构成本框架中最小的闭环：一个声明、一个对该声明的测试，以及一个消费前两者的路由决定。第 2.7 节的契约操作元与第 7.11 节的共识界是精化层——在相应困难存在之处有价值，在不存在之处可以省略。原则 2.21 与 6.11 使这个闭环可用：前者说明要看得多深，后者说明声明必须满足什么。

采纳主张的可测量形式被刻意写得谦逊，而它正是第八章已经为其自身经验内容提出的指标：本应被机械保证的性质却被留给人工审查的比率，以及被声称的边界未能通过其自身测试的比率。如果在真实项目中采纳上述三个要素并不能让这些比率移动，那么本框架的问题就不是某个假设为假，而是这副药比病本身更贵。

把采纳风险写成第十五条假设也是站得住的，以采纳率与成本收益比作为它的测量量。本文没有这样做，理由本身也是本框架纪律的一部分：一条假设的否证应当取决于关于世界的观察，而未被采纳取决于实践者的决定，让同一个词同时覆盖两者，会模糊第七章着力保持清晰的” 验证” 与” 确认” 之分。这一选择被记录在此，而不是被默默做出。

第十章　文献地图与定位

本章把本文的主张放入文献背景，指出其与既有理论的关系，并标出空缺所在。

10.1　六组文献

第一组：现代规格生成与联合评测。　这是最接近” 意图 → 规格” 这一步的前沿。相关工作报告了可测量的结果：对操作系统内核验证任务的规格生成基准在长上下文与规格完备性上仍显示模型局限 [3]；把代码正确性、规格质量与证明生成拆开评估的基准报告了三个维度之间的显著差距（在特定任务集上，代码正确率、规格健全性与完备性、单次证明成功率分别为 72.6%、52.3% 与 4.9%）[8]；以可信测试实例化规格的评测框架指出证明难度会干扰对规格质量的评价 [4]；可追溯精化方法把自然语言要求分解为原子需求并建立需求级追溯 [5]；变异驱动方法用语义不等价变体检验规格强度 [6]；从代码生成规格的路线应与从意图生成规格的路线分开研究 [7]。空洞性检测 [9] 则提供了识别” 验证通过但规格空洞成立” 的理论路径。这些结果共同支持一个重要判断：*代码生成能力、规格表达能力与形式化证明能力是不同的研究变量，应当分别评估。*

第二组：程序语义、细化与可观察行为。　操作语义、指称语义、抽象解释的格模型 [20]、单子语义 [17, 18]、域理论、精化关系 [16] 与分离逻辑 [21]，共同提供了” 何时两个不同抽象层级的系统实现了同一个行为规格” 的数学工具。语义细化不是简单的信息压缩，而是要定义抽象层之间的*行为关系*。

第三组：类型论、构造性数学与证明提取。　构造性类型论 [19]、证明与程序的对应 [47]、以及把证明对象与构造性计算联系起来的机制，为”AI 提出候选证明、机械内核检查证明” 提供了数学基础。但必须避免一个推论：*如果一个规格已经被翻译成类型，那么规格就必然忠实地表达了人的意图*。对应关系只保证在选定逻辑与类型系统中命题、类型、证明与项之间的规定结构对应；人类意图如何转化成命题是另一个需要验证的过程。精化类型 [44] 与 SMT 求解 [45] 是这一方向在工程上的代表；自动定理证明环境与检索增强的证明搜索 [13, 14] 是它在 AI 时代的延续。

第四组：从证明到机器码。　这是最成熟的一组。经验证编译器把编译转换本身变成被证明的对象 [22]；经验证编译器的后端覆盖多个目标架构 [24]；操作系统内核的精化证明展示了大规模低层系统软件的形式化验证能达到的程度 [25]；逐次翻译验证对*单次编译结果*独立检查

[23]；翻译认证为特定制品建立可验证的关系证书 [26]；证明携带代码让代码提供者附带机器可独立检查的证明 [15]。

这一组的关键教训是：验证编译器正确性与验证一个特定二进制正确性是两种不同的策略。前者证明编译器满足普遍性质，后者对某次产物独立验证。二者可以组合，而不必把全部信任押在单一工具上。

第五组：资源正确性。　自动摊还资源分析能在特定语言与成本模型下推导程序的符号化资源上界并具有可靠性证明；最坏执行时间分析针对特定硬件与假设推导时序上界；静态缓存分析说明硬件模型对上界推断的影响。必须区分三种时间概念：算法复杂度给出随输入规模变化的成本增长界；最坏执行时间分析给出特定平台与假设下的上界；实际测量提供特定实验条件下的延迟数据。三者不能互相替代。同样，" 线性使用" 不等于" 最优使用"：一个程序可以只消耗每份输入资源一次，却仍然使用了不好的算法或引入了过多延迟。

第六组：运行时验证、契约组合与层级涌现。　这一组包含两类内容，性质不同，必须分开处理。

第一类是运行时保证与契约：假设–保证推理要求子系统在输入假设满足时给出相应保证 [35]；动态系统的契约组合给出了任意互连拓扑下的组合规则与检查方法 [36, 37, 38]；运行时验证的不确定性研究说明观测、事件传输与轨迹完整性是监控可靠性的组成部分 [34]；时序逻辑的在线监控 [46] 与信息流安全 [42] 分别处理轨迹性质与访问限制。

第二类是解释框架：复杂系统的近可分解性为层级结构提供了解释 [39]；最小描述长度为" 抽象即压缩" 提供了统计与信息论背景 [56, 40, 41]；计算热力学给出计算的物理下界 [32]。这一组可以为架构、压缩、模型选择与复杂系统提供解释框架，但不能自动提供具体程序的正确性证明。

10.2　按数学功能重排的五层

把上述文献按" 它解决链条上哪一段问题" 重排，得到一个比"AI 写代码、验证器检查" 更深的结构：

最重要的发现是：这些领域并没有共同使用一个现成的、完全统一的数学对象。但它们已经分别解决了这条链上的不同问题。因此研究机会在于定义它们之间的组合关系，而不是从零发明全部数学工具。

10.3　空缺的交叉点

原则 10.1 (文献地图的形状)**.** 从规格到二进制的每一步都有成熟文献；从证明到实现的自动化综合有活跃前沿；但从意图到规格的鸿沟没有形式化解，只有共识结构，而社会认识论、拜占庭容错与对抗性评审这些文献存在于完全不同的学科，且尚未与形式化验证系统系统性地耦合。

据此，最值得占据的位置是：

表 10.1: 按数学功能重排的五个层次

层	功能	代表方向
A	目标的可表达性：形式化规格生成、意图追溯、需求分解、反例生成	规格生成基准、可追溯精化、变异驱动强度检验、空洞性检测、需求形式化
B	目标的可实现性：逻辑一致性、程序综合、约束求解、可实现性判定	反例引导归纳综合、语法引导综合、程序骨架补全、构造性类型论、SMT 求解
C	跨层语义保持：操作语义、精化关系、证明检查、编译器正确性、二进制验证	经验证编译器、经验证编译器后端、内核精化证明、证明携带代码、逐次翻译验证
D	资源与环境保证：复杂度、资源上界、运行时监控、假设–保证契约、执行约束	自动摊还资源分析、最坏执行时间分析、运行时验证、契约组合、信息流安全
E	结构与物理极限：模型压缩、复杂系统层级、热力学与物理实现	最小描述长度、复杂系统近可分解性、计算热力学、分子自组装

原则 10.2 (建议的研究位置)**.** 把社会认识论式的对抗性共识与证明携带代码式的机械准入整合在同一个框架内：让”经共识审定的规格”直接成为”携带证明的制品”的输入，并把共识记录、规格版本与制品身份用密码学锚定在一起。据本文检视，目前很少有工作同时处理这两个文献群。

这一位置之所以有价值，是因为它同时满足三个条件：它对应一个真实且尚未解决的技术缺口（意图层无机械解）；它的两个组成部分都已有成熟的理论基础；它的组合规则可以被明确写出（定义 7.41 与命题 7.42），因而可以被检验。

10.4　与既有理论的关系

表 10.2: 本文与既有理论的关系

理论	本文如何使用它，以及不如何用它
复杂系统的近可分解性	使用：作为层级结构之所以稳定的解释，并把它改造成可检验的充分统计量条件（定义 6.7、命题 6.8）。不使用：不作为”所有复杂系统必然具有相同层级”的证据；原文献本身并不证明这一点，也不给出验证成本的普遍定理。
抽象解释与 Galois 连接	使用：作为层间语义保持的形式工具（定义 2.2）。不使用：不声称所有层间关系都可用 Galois 连接表达；不能表达时退回到模拟关系或人工论证。

理论	本文如何使用它，以及不如何用它
最小描述长度	使用：作为” 理解即压缩” 的形式背景与下潜停止准则的来源（定义 2.19、命题 2.20）。不使用：不声称最短描述即最正确的描述。MDL 是模型选择原则，其结论依赖所选的模型族、编码方式与数据，不能单独决定系统应该实现什么目的。
信息隐藏与深模块	使用：作为模块边界的规范性依据。不使用：本文补充了其可检验版本，即接口是否为行为的充分统计量（定义 6.7），因为” 信息隐藏得好不好” 本身不可测量。
组织结构与系统结构的对应	使用：作为高层收敛的机制之一，也是发散的原因之一。不使用：不作为技术必然性；它是约束的来源，不是解的唯一性来源。
程序逻辑与精化	使用：作为规格–实现关系的数学接口。不使用：不声称安全精化自动保证活性、时限与公平性性质。
证明携带代码	使用：作为” 生成自由、准入严格” 信任结构的经典实现与本文第四支柱的先行者。不使用：不忽略其前提：接收方必须预先规定安全策略，而策略本身的正确性是一个外部问题。
假设–保证推理	使用：作为局部保证组合成全局保证的规则来源（命题 2.4）。不使用：不忽略其四个组合条件；组件” 单独正确” 与系统” 整体正确” 不互换。
资源分析	使用：作为资源正确性可进入形式化分析的证据。不使用：不把符号上界等同于特定处理器上的时间上界；不把复杂度阶、最坏执行时间、统计分位数与实际测量混为一谈。
计算热力学	使用：作为” 计算的物理代价不可压缩” 这一边界条件的依据。不使用：不用它预测某段普通代码的耗电；不把它并入语义正确性。（本文只在第十一章提及，不作为承重结构。）
停机问题与 Rice 定理	使用：作为通用综合与通用判定算法不可能存在的边界。不使用：不用它们证明” 意图永远无法形式化”（命题 7.11）。
P 与 NP	使用：只使用 NP 类的定义所蕴含的结构性不对称（命题 7.17）。不使用：不用”$P \neq NP$” 或”$P = NP$” 的任何未决断言作为承重结构。

原则 10.3 (本文的定位)**.** 本文的贡献位于*接口层*，而非新的语义理论：把层级从理解辅助升级为验证义务的分解单元；把无视集、近可分解性与读集形式化为可计算或可测量的对象；把内部基础设施化的必然性归因于操作语义的完备性而非代码量；把信任结构写成两种生成模式的复合并给出共识可靠性的界。这些接口把既有理论连接起来，而不是取代它们。

第十一章　反对意见与限度

本章执行一项自查：把不可承重的主张移出结构，并显式记录移出了什么。

11.1　不作主张清单

以下清单是硬性的：本文的任何其他部分都不得被解读为支持下列主张。

(N1) 本文不主张所有大型软件最终收敛到同一种高层架构（假设 3.3 被明确拒绝）。

(N2) 本文不主张代码量本身导致内部基础设施化；本文主张的是实体数与资源数的乘积、以及重复问题与演化频率。

(N3) 本文不主张六职能必须自建；本文主张的是它们必须被定义，可以外借给宿主。

(N4) 本文不主张 Simon 的近可分解性是一条普遍定律。本文把它改造为一个有条件、可检验的充分统计量判据（定义 6.7），并承认原文献不提供验证成本的普遍定理。

(N5) 本文不主张最小描述长度可以决定系统应当实现什么目的。MDL 是模型选择原则，不是意图理论。

(N6) 本文不主张” 理解就是压缩” 在所有领域都是定理。它在归纳推断范围内是可辩护的理论立场，在超出该范围时是有力的启发式。

(N7) 本文不主张机械系统绝不犯错。本文主张的是：机械判定不依赖生成者的主观意愿；机械系统仍然可能有缺陷、故障与不完整的模型。

(N8) 本文不主张信任可以被完全机械化。TCB 可以被缩小、验证与隔离，但不能被凭空消除。

(N9) 本文不主张意图层可以被机械封闭。它可以被对抗性共识逼近，且逼近的可靠性受相关错误模式支配。

(N10) 本文不主张” 加更多智能体来审查” 能提高可靠性。按推论 7.31，只有降低相关错误模式才能提高可靠性。

(N11) 本文不主张分层可以容纳一切。横切关注点在结构上不能被局部化到任何一层（原则 6.22）。

(N12) 本文不主张形式化验证会在 AI 时代自动变得便宜。这是一个值得研究的假说，而不是已成立的事实（第 7.6 节）。

(N13) 本文不主张编译正确性等于系统正确性。链接、装载、运行时、外部库、硬件与操作系统未必在证明范围内。

(N14) 本文不主张资源保证可以统一为一种方法。时间复杂度、符号资源上界、最坏执行时间、统计分位数与实际测量是不同等级的证据。

(N15) 本文不主张” 未检测到违反” 等于” 所有执行均满足”。三个状态必须被区分（原则 7.20）。

(N16) 本文不主张判定谁被授权采用一份规格。原则 7.37 确立的是：授权是一条独立于可靠性的失效轴；至于哪个机构拥有这种资格，是一项制度事实，本框架为它定位，但不为它作结。

(N17) 本文不主张给出了规格保真度如何随时间衰减的模型。第 7.14 节论证的是：该量必须带时间下标，且由命题 7.12(i) 可强制推出一条重新审定的规则；$\varepsilon_{\mathrm{cons}}(\cdot)$ 的形式则保持开放。

(N18) 本文不主张设计效应表达式 (7.3) 是一条关于多数错误的定理。它是可交换相关之下对均值的标准近似，本文用它把推论 7.31 定量化，而不是用它确立一个界。

11.2　被移出承重结构的论证

本文主动拒绝若干在同类讨论中常见、但不成立的论证。拒绝它们不是因为它们无用，而是因为它们把启发式写成了定理，从而使整个框架依赖未经证明的猜想。

表 11.1：被移出承重结构的论证

被拒绝的论证	为什么不成立	本文的替代
由 Gödel 不完备定理推出”意图与规格之间必然存在永久裂缝”	不完备定理讨论特定形式系统中的可证明性限制，不是关于人的心理意图与形式规格之间关系的定理	用三条认识论根据替代：需求是生成性的；规格是有损压缩；验证与确认的区分（命题 7.12）
由 Rice 定理推出”意图无法形式化”	Rice 定理讨论程序的非平凡语义性质的不可判定性，与人的意图无关	只把它用作通用综合与通用判定算法的边界（注 7.44 的 Q6）
把”判卷比答卷容易”称为 $\mathrm{P} \neq \mathrm{NP}$ 的红利	P 是否等于 NP 仍开放；该论证依赖未证明猜想	只使用 NP 类定义所蕴含的结构性不对称（命题 7.17）
断言”验证器必须极小、确定、绝不同构”是数学定理	小型可信内核是有价值的架构原则，小不等于绝无缺陷	TCB 最小化：让不可信部分可被检查，让不可避免的信任集中在尽可能小、性质尽可能明确的基础上
断言”死的东西不会背叛”，故机械保证绝对可靠	机械系统可能有缺陷、故障与不完整的模型	区分”不背叛”（不依赖生成者意愿）与”无误”（不可能）
断言 Simon 层级定律证明所有复杂系统必然层级化	原文献提供解释，不提供普遍定律，也不给出验证成本的普遍定理	改造为有条件、可检验的充分统计量判据（定义 6.7）

被拒绝的论证	为什么不成立	本文的替代
断言线性逻辑使资源浪费在结构上不可能	线性逻辑的指数结构仍允许特定形式的复制与忽略；线性使用不等于最优使用	采用组合结构：使用约束 + 资源上界分析 + 性能模型 + 运行时计量 + 符合条件的最坏执行时间分析
断言形式化验证在 AI 时代必然变得便宜	历史成本量级显示规格设计、模型选择、证明架构仍是实际工作（例 7.19）	降为假说，并给出可测量的成本指标（第九章判据 S5）
断言” 理解即压缩” 是普遍定律	其严格形式化适用于归纳推断，超出该范围退化为启发式	限定适用范围，并把它与任务相关的充分性绑定（定义 2.18）
断言本框架与 Langlands 纲领存在实质联系	该纲领研究算术对象与自守表示之间的函子性对偶；本框架所需的” 生成自由 / 准入确定” 只是对偶的气味，不是函子。写入它会以修辞替代结构	采用能付账的对偶：LP/SMT 对偶。Farkas 证书、不可满足核与不可实现证书给出了综合问题与其不可行性证明之间的定理级对应，且已在工程中使用
断言 Landauer 原理约束软件设计	软件尺度与该下界相差约十二个数量级；擦除下界不是架构的约束面	采用存储墙与局部性：每比特搬运的能耗比擦除下界高约 10^9 倍，这才是决定内存层级、缓存策略与数据布局的实际物理
断言同伦类型论是本框架的必要基础	该理论扩展了数学基础，但未给出资源分析或已验证编译尚未提供的软件保证	将类型论的作用限于已被证明的部分：依赖类型表达后置条件与不变量 [19]

11.3　可能的反对意见与回应

反对意见一：读集与无视集在真实系统中无法精确测量，因此这些形式化是装饰性的。　回应：定义 6.1 的读集可以从版本历史中以” 每次变更触及的模块集合” 作为代理直接统计，这是一个可计算的量（假设 H_4 正是要检验它作为代理的有效性）。无视集在本文中并非要求被自动提取，而是要求被显式声明，它是设计产物而非分析产物。声明之后，它可以被泄漏对诊断程序（推论 6.9）部分地检验。

反对意见二：核心结论” 大系统会长出一个内核” 过于宽泛，几乎无法证伪。　回应：定理 4.9 有明确前提（并发实体多于一个、资源有限且受争用、故障可达），且其结论是可证伪的：给出一个满足前提却缺少六项之一的良定义系统即可推翻它。经验部分则由假设 H_3 承担，其证伪条件是” 代码行数的解释力不劣于 $n \times m$”。

反对意见三：双层信任结构把太多责任推给共识，而共识在实践中常常失效。　回应：这正是命题 7.30 要处理的问题。该命题不是为共识辩护，而是给出它的失效形式与设计变量。它同

时解释了一个常见现象：由同源模型组成的”审查团”在结构上不构成保证，无论其规模多大。

反对意见四：把层级当作验证义务的分解单元，是否只是在重新命名已有的”分层验证”实践？　回应：部分是的，本文承认这一点。区别在于三点：(a) 本文给出的归属规则（定义 8.8）是一个可操作的判定，而不是一个口号；(b) 本文显式引入了”该层不可表达的义务”这一类，并要求它被分派给共识或人的审定，而不是被忽略；(c) 本文给出共识层可靠性的定量界，从而使”分派给共识”成为一个可以被评估而非只能被信任的决定。

反对意见五：把技术债定义为描述长度差是不可计算的，因此没有用。　回应：该定义的价值不在于数值计算，而在于它给出的方向与判据（原则 6.13(c)）：判断一次重构是否值得，等价于判断它是否让系统更容易被更短的描述生成。这是一个可以在评审中被实际使用的问题，即使 $\ell^\star$ 不可计算。

反对意见六：本文混合了工程、数学与社会认识论，是否超出了单一学科的可辩护范围？　回应：这一批评成立。缓解措施有两项。第一，对每一类主张标注其强度（第九章的三分法）；第二，明确区分哪些引用是被使用的、哪些是被拒绝的（表 10.2）。跨学科本身不是缺陷，前提是每个学科的结论都不被越界使用。

11.4　尚未解决的问题

以下问题是开放的：

(O1) **Dom 与 Conv 的独立可测性**。猜想 3.10 要成为可检验命题，需要两个量都能被独立于结论地度量。本文只给出了方向，没有给出可复现的标注规程。

(O2) **六职能完备性论证的形式化**。定理 4.9 的证明是结构性枚举，不是形式系统中的推导。把它写成对一个给定的操作语义框架的完备性定理，是一项尚未完成的工作。

(O3) **跨层语义的组合是否总能成立**。命题 2.4 给出充分条件，但在包含非确定性、实时约束与自反性的系统中，这些条件是否可同时满足尚不清楚。

(O4) **共识层的 q 如何度量**。命题 7.30 把可靠性归因于相关错误模式，但没有给出 q 的可操作估计方法。这是该方向能否落地的关键。第 7.11 节给出了一个配套量，即审定者错误的平均成对相关 $\bar{\rho}$，它可以由本框架已经在收集的数据估计出来；而 q 仍然是那个无法这样估计的极限。

(O5) **资源与功能的联合语义是否可判定**。带成本的迹语义可以表达联合约束，但其可判定片段有多大、紧致度如何，尚无定论。

(O6) **制品绑定与动态生成实现的兼容**。如果实现可以在运行时根据负载重新生成，式 (2.13) 的静态同一性断言需要被替换为”哪些变化保持原有契约、哪些要求重新验证、验证失败时如何回退到已知安全状态”这一更强的问题。

(O7) **层级归属的一致性**。定义 8.8 是否能在不同分析者之间给出一致的归属，需要实证检验。

11.5　方法论局限

(L1) 本文引用的全部文献，其核验深度限于正式出版页面、作者或项目主页、以及预印本元数据，未逐篇通读全文。因此文中每一个具体数字都应被视为待复核的二手报告；标称年份较新的若干条目按原始记录保留，本文未做独立核验。针对第一轮修订，本文补做了一轮核验。支撑假设 H_7 的五条经验类引用被对照原始摘要页面逐一核对：所引的四个数字（189 个 Lean 任务；最佳模型的 72.6%、52.3% 与 4.9%）以及第二个基准中的 245 个操作系统任务，均见于摘要本身，而不仅仅见于二手综述。由此做出两处出处更正：可追溯精化的那项工作是研讨会海报而非主会论文，变异驱动方法则是未经评审的预印本。新增的引用按同一深度核对，其中两篇未读全文的条目在参考文献中标为” 仅元数据”。这一轮未确立的是：没有任何数字是从某篇论文的表格或图中重新推导出来的，也没有任何全文被从头读到尾，因此数字类主张的等级仍然是” 引自原始摘要”，比最初的二手报告高一级，但尚未达到独立复现的程度。

(L2) 本文的形式化是为本文目的而建立的工作定义，不应被误称为已有统一标准。特别是” 无视集”” 层契约四元组”” 验证义务的层级归属” 三项，是本文提出的工作概念。

(L3) 本文的三条认识论根据（命题 7.12）是论证而非定理；其可挑战之处在于这三条是否穷尽。

(L4) 本文未进行任何实验。第九章的全部假设均处于” 待检验” 状态。

原则 11.1 (自查结论)**.** 上述审视之后，承重结构剩下三条可独立检验的最小命题：

(i) 验证与生成之间存在由 NP 类定义所蕴含的结构性不对称；

(ii) 层级是保证义务的分解单元，因为每条义务都归属某个可观察域；

(iii) 意图与规格之间的鸿沟有认识论根据，因而不可被普遍机械封闭。

其余主张（包括” 压缩”” 涌现”” 内部基础设施化” 等）要么已被降为可证伪假设，要么已被限定在明确的适用范围内。

第十二章　结论

12.1　四个问题的回答

问题 12.1 (底层确定性的可推广性)**.** 结论分三层。底层之所以近乎唯一，是因为它被过约束：约束数不少于设计自由度，解集退化为离散点集（命题 3.5）。高层之所以多解，是因为问题层面的约束远少于自由度，解集是一个高维流形。但高层仍然收敛，因为存在第二组约束（来自制造与维护该系统的人类组织）把流形压缩成少数几个尖峰（原则 3.8）。

因此准确的回答是：存在收敛，但不是朝向工程最优，而是朝向有界组织可导航的结构。收敛的载体是关系而非组件（原则 3.11）；收敛的强度关于领域绑定度单调递减（猜想 3.10）。

问题 12.2 (规模的强制性)**.** 结论是一个条件必然性结果。决定是否需要统一基座的自变量是活跃实体数与共享资源数的乘积，而不是代码行数（定理 4.4、推论 4.5）。而六项职能（身份与生命周期、分配与回收、调度、命名与寻址、通信与同步、故障隔离与恢复）是操作语义良定义的必要条件（定理 4.9）。它们可以被自建，也可以被外借；判据是其所承载的不变量是否是领域关键的（推论 4.12）。

问题 12.3 (智能的理解与设计)**.** 结论是：理解是逆向压缩，设计是正向压缩，二者是同一结构的两个读取方向。理解的最优下潜深度由率失真驻点条件给出（命题 2.20）；设计的可测量目标是读集期望（定义 6.1），其最小化要求不变量的单点强制（定理 6.3）。抽象是否泄漏是可测的，而不只是可感的（命题 6.8、推论 6.9）。对 AI 而言，优势不在于读得更多，而在于探测得更快（命题 5.8）。

问题 12.4 (无人审阅条件下的正确性)**.** 结论是：人不再逐行审阅实现是可达的，人不必再为目标的表达负责是不可达的。”正确”必须被拆成五个维度（定义 7.1）；中间过程必须被定义为七段可验证细化链（定义 7.2）；信任必须是两种生成模式的复合（定义 7.41）；制品绑定是不可省略的第四支柱（式 (2.13)）。综合保证条件式（命题 7.42）给出了完整的回答。

12.2　压缩作为主线

把全文收拢，得到一条单一的主线：**压缩**。

- 复杂系统用层级压缩自己，于是每个大系统都长出一个域内内核（第四章）；
- 有限智能用理解压缩信息，于是有限的认知装得下无限的世界（第五章）；
- 工程用架构压缩变更成本，于是局部的修改不需要全局的理解（第六章）；
- 文明用规则压缩信任，于是来源不明的百万行代码可以被安全运行（第七章）。

而压缩的机制在四种情形下是同一个：按抽象映射的核做商（命题 2.9）。一个层级暴露给上层的全部信息，就是下层行为在这个核下的商。

12.3　核心命题

如果要把全文压缩为一句可被检验的话，本文选择这一句：

原则 12.5 (核心命题)**.** 一个复杂系统能被有限智能理解，当且仅当它可被压缩；而它可被压缩，当且仅当它存在一套尺度分离，使得每一层都拥有一个明确的、被声明的无视集。

这句话有三个直接后果。

第一，它把”可理解性”从一个主观感受变成了一个结构性质：可理解性等价于存在一个商映射，在该映射下任务相关的预测能力被保留。这使可理解性成为设计的目标函数而非事后评价。

第二，它把”架构”重新定义为人为制造的尺度分离。自然界的分层由尺度分离与重整化免费获得；软件的分层不会自然涌现，必须由接口、封装与不变量付费制造，并且随时会漏。因此架构的核心交付物不是模块列表，而是每一层的无视集声明。

第三，它对 AI 给出一个具体的行动准则：不要试图穷尽底层信息量，而应维护一个分层的、可证伪的、能被反例更新的压缩模型；用探测而非阅读获取梯度；以满足任务充分性作为停止条件。

12.4　结语

全文始于三个工程直觉，落在一个关于结构与保证的联合命题上。确定的不是答案，而是哪些问题没有答案，以及为什么：

- 意图与规格之间的鸿沟没有机械解，其根据是认识论的而不是逻辑的；它实质上是框架问题，因而只在交互中可回避；
- 机械保证的信任基础可以被缩小，但不能被消除；且”验证便宜”必须分解为检查时间与证书尺寸两个独立命题，后者才是常见瓶颈；
- 共识保证的可靠性受相关错误模式支配，因此多样性而不是数量才是设计变量，而多样性最终是激励设计问题，不是统计问题；
- 大系统的结构收敛是概率性的、分领域的、受组织约束塑形的，而不是唯一的；
- 架构的职责不是降低单个变更的成本，而是降低该成本随规模增长的指数。

把这几条并列起来看，共同的形状是：复杂度压缩是规模驱动的必然，压缩的形态是领域驱动的偶然；而保证的边界，恰好落在”能够被机器判定的”与”必须由承担后果的主体确认的”之间。

代价是放弃一个可以用一条定理表达的图景；收益是每一个主张都落在它能被检验的位置上。对一个关于”如何在不可信的复杂性之上建立可信结论”的框架，这是唯一自洽的结局：它必须通过自己的检验。

附录 A　补充证明与论证

本附录给出正文中若干结果的完整证明或更细致的论证。凡正文中已给出完整证明者，此处不重复。

A.1　命题 3.5 的说明

正文给出的证明基于横截性定理。这里补充两点。

为什么”独立约束”是必要的。　若两个约束不独立（例如第二个是第一个的推论），则它们对余维数的贡献不叠加，$\operatorname{codim}\mathcal{S} < c$，解集维数高于 $d-c$。因此命题 3.5 的结论应在”约束独立且横截”的前提下理解。这一前提在底层设计中通常成立（每条约束来自不同的既有对象：二进制兼容性、调试器、异常展开、系统调用接口），而在高层设计中常常不成立（多个”约束”实际上是同一偏好的不同表述）。

为什么”解集退化”不等于”唯一解”。　$c \geq d$ 时解集是有限集合或空集，有限集合可以有多个元素。底层之所以在实践中常常”接近唯一”，是因为除了形式约束之外还有隐式的经济约束：同时维护两套不兼容的调用约定会带来人力与工具链成本，这一成本在可选方案之间形成严格的偏好序。

A.2　定理 4.4 的完整论证

正文给出的是主论证。这里补充成本函数的选择与假设的合理性。

成本的定义。　设单位交互成本为 c，指”正确处理一次特定实体与特定资源之间的交互”所需的全部工作（设计、实现、测试与长期维护）。在无基座情形下，每个实体必须独立完成这一工作，且必须独立处理资源的不变量。

一致性义务的量级。　对资源 r，n 份独立实现必须在 r 的不变量上取得一致。若一致性不被强制，则任一份实现的偏离都会破坏全局不变量。若被强制，则强制本身需要 n 个位置都知道该不变量，即强制点数为 $\Omega(n)$。这两种情形都不优于”读集 $\geq k$”的结论（定理 6.3）。换言之，无基座不仅成本更高，而且它把不变量强行散落，从而永久提高了系统的读集下界。这是统一基座的第二个收益，独立于摊销。

假设 (b) 的松弛。　假设 (b)（每个实体参与全部 m 个资源）是最坏情形。若实体 e 仅参与 $|\mathcal{R}(e)|$ 个资源，则无基座成本为 $\sum_e |\mathcal{R}(e)|$。但只要 $\mathbb{E}_e[|\mathcal{R}(e)|]$ 随 m 增长（即资源的使用是分散而广泛的），乘积形式的增长仍然成立。因此式 (4.3) 的定性结论不依赖最坏情形假设。

A.3　定理 4.9 的逐项论证补充

正文以" 缺失导致未定义行为" 的方式逐项论证。这里补充两点对论证强度的说明。

(F3) 调度的论证强度。　该论证依赖"$\to$ 应当是函数，或其非确定性应被显式规定"。若一个系统的语义被定义为所有交错的集合（即把非确定性当作语义的一部分而不加选择），那么" 选择函数" 确实可以缺失。但这种情形下，系统的实现仍然必须在每个具体执行中选择一个交错，因此调度职能在实现层面仍然存在，只是没有出现在抽象语义中。本文的表述" 或者在该系统自身的代码中，或者在其宿主中" 覆盖了这一情形。

(F6) 隔离的论证强度。　该论证依赖" 故障可达"。若一个系统的运行环境保证无故障（例如纯函数式计算、或在被验证为无故障的受限环境中执行），则 (F6) 可以缺失或退化为平凡的。这正是一些极简系统能够不拥有完整六职能的原因，也说明定理 4.9 的前提不是多余的。

A.4　定理 6.3 的读集下界

正文证明了 $|\rho(c)| \geq |\mathrm{S}(\iota)|$。这里说明为什么这一不等式是紧的。

若 $\mathrm{S}(\iota) = \{M_0\}$，且 M_0 的接口是充分的（定义 6.7），则任何可能破坏 ι 的变更都必须在 M_0 的接口上被表达，因此变更者只需理解 M_0 与其调用点。此时 $|\rho(c)| = 1 + O(\text{调用点数})$，而下界 1 被达到（当变更本身就发生在 M_0 内部时）。

若 $\mathrm{S}(\iota) = \{M_0, M_1\}$ 且这两个模块都对 ι 有部分责任，则变更者必须理解两者以及它们之间的一致性规则，后者的理解成本严格高于前两者之和。这说明读集关于 $|\mathrm{S}(\iota)|$ 的增长实际上是超线性的，而不仅是不小于。正文中的线性下界因此是保守的。

A.5　命题 7.12 的三条根据是否穷尽

这一论证有限度。三条根据的形式如下：

(g1) 偏好是生成性的：存在意图 I 与情形 s，使得 I 的某个成分只有在意图主体遭遇 s 之后才被确定。

(g2) 规格是有损压缩：存在意图 I 与规格 Φ，使得 Φ 无法承载 I 的全部相关信息。

(g3) 确认不可归约为验证：存在需求 R 与实现 P，使得"P 满足 Φ" 为真而"P 满足 R" 为假，且判定后者的过程不能被归约为判定前者。

三条根据不是互相独立的：(g1) 是 (g2) 的一个来源，(g3) 是 (g2) 的一个后果。因此更准确的表述可能是：意图鸿沟的根源是 *(g1)* 与 *(g2)* 的复合。本文保留三条并列的表述，是因为它们在工程上对应不同的缓解手段：(g1) 对应交互式精化与反例生成；(g2) 对应追溯与覆盖分析；(g3) 对应独立的确认流程与责任划分。

是否存在第四条根据，是一个开放问题（第十一章 O 项）。

A.6　定理 7.29 的界是否紧

式 (7.1) 是 Hoeffding 界，它在大 n 下不是紧的。对 $p < 1/2$，多数错误的概率实际上由中心极限定理给出，$\Pr[\text{多数出错}] \approx \Phi\big(-\sqrt{n}\,(1/2-p)/\sigma\big)$，其指数为 $\Theta(n(1/2-p)^2)$ 但常数更优。

本文选择 Hoeffding 界而非更紧的估计，原因是：

(a) 它不需要对个体错误的分布作任何假设，只需要独立性、有界性与均值；

(b) 它的形式更简单，因而不容易被误读为” 精确值”；

(c) 本文的核心结论（命题 7.30）不依赖该界的紧致性：相关错误模式给出的下界 q 与上界的常数无关。

A.7　命题 7.30 的一个更强的形式

正文给出了 $\Pr[\text{多数出错}] \geq q$。可以给出一个更细致的形式。设共同错误模式 C 与个体独立错误 I_i 满足：验证者 i 出错当且仅当 $C = 1$ 或 $I_i = 1$。则多数出错的事件为

$$\{C = 1\} \;\cup\; \Big\{C = 0,\ \sum_{i=1}^{n} I_i \geq \tfrac{n}{2}\Big\},$$

故

$$\Pr[\text{多数出错}] \;=\; q + (1-q)\,\Pr\Big[\sum_{i=1}^{n} I_i \geq \tfrac{n}{2}\Big] \;\geq\; q.$$

并且当 $q > 0$ 固定时，$\lim_{n\to\infty} \Pr[\text{多数出错}] = q$。这给出了一个比” 下界为 q” 更强的陈述：*共识的可靠性存在一个由相关错误模式决定的渐近地板，增加验证者数量只能逼近该地板，不能突破它*。这正是推论 7.31 的定量依据。

A.8　命题 5.8 的说明

正文的论证使用了信息增益的定义。这里补充一个说明：命题 5.8 并不是说被动阅读没有价值。被动阅读提供的是*先验*（在贝叶斯意义上），而主动探测提供的是*似然*。一个没有先验的探测者不知道去探测什么；一个只有先验的阅读者无法更新。因此第 5.4 节的七步协议前四步是阅读（建立先验），后三步是探测（获取梯度）。二者的次序不是任意的。

附录 B　模板

本附录给出三份可直接使用的模板。它们把正文中的形式对象转换为工程文档。

B.1　架构地图模板

用于记录一个系统的层级阶梯。每一行是一层。

表 B.1: 架构地图

层	参与者	保证的不变量	假设下层	可以无视	验证方式
L_4					
L_3					
L_2					
L_1					
L_0					

填写要求：

(1) ” 可以无视” 一列不得留空。留空意味着该层不是层级（原则 8.6）。

(2) ” 保证的不变量” 必须写成可以被检验或至少可以被判定的命题。

(3) ” 验证方式” 必须指明该不变量归入表 8.2 中的哪一类保证（机械、监控、共识）。

B.2　模块契约模板

身份 模块名、所属层、唯一标识。

职责 它解决什么问题（一句话，不超过一行）。

边界 它负责什么、明确不负责什么。

对外接口 上层通过哪些函数、类型或接口使用它。

前置条件 P 调用者必须满足什么。

后置条件 Q 返回时保证什么。

不变量 I 在何处被强制；为什么绕过不可能。

副作用 E^+ / E^- 允许发生什么；明确禁止什么。

资源预算 R 时间、空间、带宽、并发度的上界及其成本模型。

失败语义 F 出错、取消、重试与恢复时会发生什么。

环境假设 A 结论成立所依赖的硬件、操作系统、负载与外部服务条件。

所有权与生命周期 关键对象如何创建、初始化、关闭与销毁；谁拥有。
并发模型 线程安全保证、锁序、可重入性。
依赖 依赖哪些模块；依赖抽象接口还是具体实现。
无视集 本模块被允许永远不关心的内容（必填，见正文定义 2.8）。
验证要求 哪些测试或检查能证明上述结论；是否存在可隔离的验证单位。
泄漏对风险 已知的” 接口相同而外部行为不同” 的场景。
未决问题 尚未确定的设计问题。

B.3 决策记录模板

标识与标题 ADR-XXXX: 决策标题。
状态 草案 / 已接受 / 已取代 / 已废弃。
背景 什么情况迫使必须作出这一决策。
决策驱动因素 影响该决策的约束与目标（性能、一致性、团队规模、演化速度、成本）。
候选方案 逐一列出，每个方案写明其假设与代价。
决策 选择哪一个，以及一句话理由。
影响 该决策使什么变容易了，使什么变难了。
被排除方案的完整论证 为什么其他方案被拒绝。这一项是决策记录最有价值的部分：它记录了当时可得的信息与判断，使未来的读者不必重新推导。
验证 该决策的后果如何被检验；哪些外部证据会表明它是错的。
重新评估条件 出现什么情况时应当重新审视这一决策。

原则 B.1. 若一个架构像底层 ABI 一样是物理事实，就没有必要写决策记录。*决策记录的普遍存在，本身就是高层架构没有唯一正确答案的证据。*

附录 C　术语表

术语	定义与所在位置
层级 (level)	六元组 $(X, \mathcal{B}, \mathcal{O}, \mathcal{A}, \mathcal{C}, \mathcal{I})$；认知尺度而非物理目录。定义 2.1。
抽象映射	把下层行为投影为上层行为的映射 $\mathcal{O}_i$。定义 2.2。
具体化映射	把上层行为展开为下层实现集合的映射 γ_i。定义 2.2。
Galois 连接	$\mathcal{O}_i \dashv \gamma_i$，刻画抽象化丢失了什么、保住了什么。定义 2.2。
层间可靠性	$\mathcal{O}_i(\mathcal{B}_{i+1} \restriction A) \subseteq \mathcal{B}_i$。定义 2.3。
无视集 (oblivious set)	抽象映射的核 $\ker \mathcal{O}_i$，即该层在原理上永远无法区分的差异集合。定义 2.8。
压缩率	在抽象映射下不可恢复的信息量。定义 2.14。
架构关系型	带类型的多重图，边分为抽象、依赖、组合、控制、契约五类。定义 2.15。
抽象骨架	把构件替换为职责角色、保留关系类型而抹去实例后的图。定义 2.16。
任务	二元组 $T = (Q, d)$，Q 为查询族，d 为答案损失。定义 2.17。
T-充分性	模型对所有查询给出与真实系统一致的答案。定义 2.18。
描述长度	模型在固定编码方案下的长度 $\ell(M)$。定义 2.19。
行为契约	八元组 $(P, Q, E^+, E^-, R, I, F, A)$。定义 2.22。
可观察轨迹	工件在环境中可能产生的可观察执行序列集合 $\mathrm{Traces}(B, E)$。定义 2.23。
保证	四元组（性质、对象、前提、证据）；其成立条件为式 (2.12)。定义 2.24。
可信计算基 (TCB)	必须被无条件信任才能使保证成立的组件集合。定义 2.25。
机械保证	由单个确定性、可穷尽检视的判定器给出的保证。定义 2.26。
共识保证	由多个独立参与方的对抗性审查给出的保证。定义 2.26。
内部基础设施化	共性识别 $\to$ 机制统一 $\to$ 上层组合的三阶段过程。定义 4.1。
参与者-资源结构	五元组 $(E, \mathcal{R}, \to, \mathrm{Ready}, \mathrm{Fault})$。定义 4.2。
统一基座	使所有实体与资源的交互都经过其声明接口的机制集合。定义 4.3。
内核六职能	身份与生命周期、分配与回收、调度、命名与寻址、通信与同步、故障隔离与恢复。定义 4.8。

术语	定义与所在位置
变更读集	为正确完成一次变更而必须阅读的模块集合 $\rho(c)$。定义 6.1。
不变量强制点	必须正确实现才能保证某不变量成立的模块集合 $\mathrm{S}(\iota)$。定义 6.2。
近可分解	模块接口观察是其外部行为的充分统计量。定义 6.7。
泄漏对	接口相同而外部行为不同的两次执行。命题 6.8。
技术债	代码描述长度与最短生成描述之差。定义 6.12。
可验证细化链	从目的到经认证部署的七段链 (G1)…(G7)。定义 7.2。
五个正确性维度	功能、副作用、资源、系统、目标表达。定义 7.1。
副作用四分类	功能性、资源性、安全性、语义漂移。定义 7.22。
制品证据清单	每个制品应携带的十项证据。定义 7.21。
认知尺度	L_0 计算原理、L_1 基础设施与运行时、L_2 组件与子系统、L_3 领域功能与业务语义、L_4 及以上场景与系统级行为、L_∞ 目的与责任。定义 8.1。
层契约四元组	（参与者、保证的不变量、契约形式、验证方式）加无视集。定义 8.5。
验证义务的层级归属	性质 φ 归属于能表达它的最低层级。定义 8.8。
领域绑定度	领域特有公理在该层全部公理中的占比 Dom。定义 3.9。
双层信任结构	意图层由共识保证、可形式化层由机械保证的复合结构。定义 7.41。

符号索引

符号	含义
Λ_i	第 i 层
$\mathcal{B}_i$	第 i 层的行为集合
$\mathcal{O}_i, \gamma_i$	抽象与具体化映射
$\mathcal{A}_i, \mathcal{C}_i, \mathcal{I}_i$	环境假设、成本模型、不变量集合
$\mathrm{Obl}(\Lambda)$	无视集
κ	压缩率
$\mathcal{G}, \phi$	架构关系型、抽象骨架
T, Q, d	任务、查询族、损失泛函
ℓ, L_T	描述长度、任务损失
S	行为契约
$\mathrm{Traces}(B, E)$, $\mathrm{Allowed}(S)$	可观察轨迹集、允许轨迹集
$\mathfrak{A}$, TCB	保证四元组、可信计算基

符号	含义
Π	参与者-资源结构
$\mathcal{K}_6$	内核六职能
n, m, k, F	实体数、资源数、每实体接口数、基座固定成本
$\rho(c), \mathrm{S}(\iota)$	读集、强制点集
D	技术债
$\mathrm{Dom}, \mathrm{Conv}$	领域绑定度、收敛强度
$q, p, \varepsilon_{\mathrm{cons}}$	相关错误概率、个体错误率、共识错误上界

参考文献

[1] J. Mucha, A. Kaufmann, and D. Riehle. A systematic literature review of pre-requirements specification traceability. *Requirements Engineering*, 29:119–141, 2024. https://doi.org/10.1007/s00766-023-00412-z

[2] K. Mokos and P. Katsaros. A survey on the formalisation of system requirements and their validation. *Array*, 7:100030, 2020. https://doi.org/10.1016/j.array.2020.100030

[3] S. Li, J. Jiang, T. Zhao, and J. Shen. OSVBench: Benchmarking LLMs on specification generation tasks for operating system verification. In *Proceedings of AAAI 2026*, 40(37):31698–31707, 2026. https://doi.org/10.1609/aaai.v40i37.40437

[4] F. Yang, X. Li, S. Wang, J. An, Z. Sun, S. Feng, W. Wang, W. Wang, N. Zhan, and F. Xu. How powerful are LLMs in generating formal program specifications? In *Proceedings of ICML 2026*, PMLR 306:145221–145249, 2026. https://proceedings.mlr.press/v306/yang26am.html

[5] Z. Ye, A. Z. H. Yang, H. Su, Z. Liao, S. Tenka, Z. Qin, U. Ghai, D. Song, and S. Kong. Intent-aligned formal specification synthesis via traceable refinement (VeriSpecGen). arXiv:2604.10392, 2026. https://arxiv.org/abs/2604.10392

[6] L. Ma, S. Liu, Y. Li, Q. Wu, H. Wang, and L. Bu. SpecSyn: LLM-based synthesis and refinement of formal specifications for real-world program verification. arXiv:2604.21570, 2026. https://arxiv.org/abs/2604.21570

[7] L. Ma, S. Liu, Y. Li, X. Xie, and L. Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proceedings of ICSE 2025*, 2025. https://doi.org/10.1109/ICSE55347.2025.00129

[8] Z. Ye, Z. Yan, J. He, T. Kasriel, K. Yang, and D. Song. VERINA: Benchmarking verifiable code generation. In *Proceedings of ICLR 2026*, 2026.

[9] O. Kupferman and M. Y. Vardi. Vacuity detection in temporal model checking. In *CHARME 2003*, 2003. https://doi.org/10.1007/3-540-48153-2_8

[10] A. Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis / Technical Report EECS-2008-177, UC Berkeley, 2008. https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-177.html

[11] R. Alur et al. Syntax-guided synthesis. In *Proceedings of CAV 2013*, 2013. https://www.sygus.org/

[12] T. Kurabayashi, H. Tanno, H. Kirinuki, and T. Oshima. A survey of program synthesis. *Computer Software*, 40(2):95–128, 2023. https://doi.org/10.11309/jssst.40.2_95

[13] K. Yang, A. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. J. Prenger, and A. Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *NeurIPS 2023 Datasets and Benchmarks*, 2023.

[14] P. Song, K. Yang, and A. Anandkumar. Towards large language models as copilots for theorem proving in Lean. arXiv:2404.12534, 2024. https://arxiv.org/abs/2404.12534

[15] G. C. Necula. Proof-carrying code. In *Proceedings of POPL 1997*, pages 106–119, 1997. https://doi.org/10.1145/263699.263712

[16] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969. https://doi.org/10.1145/363235.363259

[17] E. Moggi. Notions of computation and monads. *Information and Computation*, 93(1):55–92, 1991. https://doi.org/10.1016/0890-5401(91)90052-4

[18] G. D. Plotkin and J. Power. Notions of computation determine monads. In *FOSSACS 2002*, LNCS 2303:342–356, 2002. https://doi.org/10.1007/3-540-45931-6_24

[19] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013. https://homotopytypetheory.org/book/

[20] P. Cousot and R. Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of POPL 1977*, pages 238–252, 1977. https://doi.org/10.1145/512950.512973

[21] A. W. Appel and S. Blazy. Separation logic for small-step Cminor. In *TPHOLs 2007*, LNCS 4732:5–21, 2007. https://doi.org/10.1007/978-3-540-74591-4_3

[22] X. Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009. https://doi.org/10.1145/1538788.1538814

[23] G. C. Necula. Translation validation for an optimizing compiler. In *Proceedings of PLDI 2000*, 2000. https://doi.org/10.1145/349299.349314

[24] Y. K. Tan, M. O. Myreen, R. Kumar, A. Fox, S. Owens, and M. Norrish. The verified CakeML compiler backend. *Journal of Functional Programming*, 29, 2019. https://doi.org/10.1017/S0956796818000229

[25] G. Klein et al. seL4: Formal verification of an OS kernel. In *Proceedings of SOSP 2009*, pages 207–220, 2009. https://sel4.systems/Research/publications.html

[26] J. O. G. Krijnen, M. M. T. Chakravarty, G. Keller, and W. Swierstra. Translation certification for smart contracts. *Science of Computer Programming*, 233:103051, 2024. https://doi.org/10.1016/j.scico.2023.103051

[27] A. M. Turing. On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 42:230–265, 1936. https://doi.org/10.1112/plms/s2-42.1.230

[28] H. G. Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the AMS*, 74(2):358–366, 1953.

[29] J. Hoffmann and S. Jost. Two decades of automatic amortized resource analysis. *Mathematical Structures in Computer Science*, 32(6), 2022. https://doi.org/10.1017/S0960129521000487

[30] R. Wilhelm et al. The worst-case execution-time problem—overview of methods and survey of tools. *ACM Transactions on Embedded Computing Systems*, 7(3), 2008. https://doi.org/10.1145/1347375.1347389

[31] M. Lv, N. Guan, J. Reineke, R. Wilhelm, and W. Yi. A survey on static cache analysis for real-time systems. *LITES*, 3(1), 2016. https://doi.org/10.4230/LITES-v003-i001-a005

[32] R. Landauer. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development*, 5:183–191, 1961. https://doi.org/10.1147/rd.53.0183

[33] I. Georgescu. 60 years of Landauer's principle. *Nature Reviews Physics*, 3:770, 2021. https://doi.org/10.1038/s42254-021-00400-8

[34] R. Taleb, S. Hallé, and R. Khoury. Uncertainty in runtime verification: A survey. *Computer Science Review*, 50:100594, 2023. https://doi.org/10.1016/j.cosrev.2023.100594

[35] T. A. Henzinger, S. Qadeer, and S. K. Rajamani. You assume, we guarantee: Methodology and case studies. In *Proceedings of CAV 1998*, 1998.

[36] M. Sharf, B. Besselink, K. H. Johansson, et al. Assume/guarantee contracts for dynamical systems: Theory and computational tools. *IFAC-PapersOnLine*, 54(5):25–30, 2021. https://doi.org/10.1016/j.ifacol.2021.08.469

[37] M. Sharf, B. Besselink, and K. H. Johansson. Contract composition for dynamical control systems: Definition and verification using linear programming. *Automatica*, 164:111637, 2024. https://doi.org/10.1016/j.automatica.2024.111637

[38] B. Shali, A. J. van der Schaft, and B. Besselink. Design and control for implementation of simulation-based assume-guarantee contracts. *IEEE Transactions on Automatic Control*, 71(1):65–80, 2026. https://doi.org/10.1109/TAC.2025.3587874

[39] H. A. Simon. The architecture of complexity. *Proceedings of the American Philosophical Society*, 106(6):467–482, 1962.

[40] P. D. Grünwald. The minimum description length principle in coding and modeling. *IEEE Transactions on Information Theory*, 52(2), 2006. https://doi.org/10.1109/TIT.2005.862128

[41] P. Grünwald and T. Roos. Minimum description length revisited. arXiv:1908.08484, 2019. https://arxiv.org/abs/1908.08484

[42] J. A. Goguen and J. Meseguer. Security policies and security models. In *IEEE Symposium on Security and Privacy 1982*, pages 11–20, 1982. https://doi.org/10.1109/SP.1982.10014

[43] M. C. Feser, S. Chaudhuri, and I. Dillig. Synthesizing data structure traversals from local specifications. In *Proceedings of PLDI 2015*, 2015. https://doi.org/10.1145/2737924.2738007

[44] P. M. Rondon, M. Kawaguchi, and R. Jhala. Liquid types. In *Proceedings of PLDI 2008*, 2008. https://doi.org/10.1145/1375581.1375602

[45] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *TACAS 2008*, LNCS 4963, 2008. https://doi.org/10.1007/978-3-540-78800-3_24

[46] M. Leucker and C. Schallhart. Runtime verification for LTL and TLTL. In *TACAS 2009*, 2009. https://doi.org/10.1007/978-3-642-00768-2_17

[47] W. A. Howard. The formulae-as-types notion of construction. In *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus and Formalism*, 1980.

[48] On the generation of requirements specifications from software engineering models: A systematic literature review. *Information and Software Technology*, 51(9):1291–1307, 2009. https://doi.org/10.1016/j.infsof.2009.04.001

[49] D. L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972.

[50] M. E. Conway. How do committees invent? *Datamation*, 14(4):28–31, 1968.

[51] F. P. Brooks. *The Mythical Man-Month*. Addison-Wesley, 1975.

[52] B. W. Boehm. Verifying and validating software requirements and design specifications. *IEEE Software*, 1(1):75–88, 1984.

[53] J. Ousterhout. *A Philosophy of Software Design*. Yaknyam Press, 2018.

[54] L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.

[55] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27:379–423, 1948.

[56] J. Rissanen. Modeling by shortest data description. *Automatica*, 14(5):465–471, 1978.

[57] A. N. Kolmogorov. Three approaches to the quantitative definition of information. *Problems of Information Transmission*, 1(1):1–7, 1965.

[58] W. Hoeffding. Probability inequalities for sums of bounded random variables. *Journal of the American Statistical Association*, 58(301):13–30, 1963.

[59] M. de Condorcet. *Essai sur l'application de l'analyse à la probabilité des décisions rendues à la pluralité des voix*. Paris, 1785.

[60] L. C. Paulson. The foundation of a generic theorem prover. *Journal of Automated Reasoning*, 5(3):363–397, 1989.

[61] M. T. Nygard. *Release It! Design and Deploy Production-Ready Software*. Second edition, Pragmatic Bookshelf, 2018.

[62] E. Evans. *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.

[63] D. C. Dennett. Real patterns. *The Journal of Philosophy*, 88(1):27–51, 1991.

[64] W. V. O. Quine. Two dogmas of empiricism. *The Philosophical Review*, 60(1):20–43, 1951.

[65] I. Lakatos. Falsification and the methodology of scientific research programmes. In *Criticism and the Growth of Knowledge*, Cambridge University Press, 1970.

[66] J. Pearl. *Causality: Models, Reasoning, and Inference*. Second edition, Cambridge University Press, 2009.

[67] T. Leinster. *Higher Operads, Higher Categories*. London Mathematical Society Lecture Note Series 298, Cambridge University Press, 2004.

[68] B. Jacobs. *Categorical Logic and Type Theory*. North-Holland, 1999.

[69] P. Alvaro, N. Conway, J. M. Hellerstein, and W. R. Marczak. Consistency analysis in Bloom: A CALM and collected approach. In *Proceedings of CIDR 2011*, 2011.

[70] G. W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.

[71] C. E. Shannon. Coding theorems for a discrete source with a fidelity criterion. *IRE National Convention Record*, 7(4):142–163, 1959.

[72] R. E. Kalman. On the general theory of control systems. In *Proceedings of the First International Congress on Automatic Control*, 1960.

[73] A. Saoud, A. Girard, and L. Fribourg. Assume-guarantee contracts for continuous-time systems. *Automatica*, 134:109910, 2021. https://doi.org/10.1016/j.automatica.2021.109910

[74] A. Benveniste, B. Caillaud, D. Nickovic, R. Passerone, J.-B. Raclet, P. Reinkemeier, A. Sangiovanni-Vincentelli, W. Damm, T. A. Henzinger, and K. G. Larsen. Contracts for system design. *Foundations and Trends in Electronic Design Automation*, 12(2–3):124–400, 2018. (metadata only) https://doi.org/10.1561/1000000053

[75] R. H. Coase. The nature of the firm. *Economica*, 4(16):386–405, 1937. https://doi.org/10.1111/j.1468-0335.1937.tb00002.x

[76] O. E. Williamson. *The Economic Institutions of Capitalism*. Free Press, New York, 1985.

[77] W. K. G. Assunção, J. Krüger, S. Mosser, and S. Selaoui. How do microservices evolve? An empirical analysis of changes in open-source microservice repositories. *Journal of Systems and Software*, 204:111788, 2023. https://doi.org/10.1016/j.jss.2023.111788

[78] F. Dietrich and C. List. A model of jury decisions where all jurors have the same evidence. *Synthese*, 142(2):175–202, 2004. https://doi.org/10.1007/s11229-004-1276-z

[79] F. Dietrich. The premises of Condorcet's jury theorem are not simultaneously justified. *Episteme*, 5(1):56–73, 2008. https://doi.org/10.3366/E1742360008000233

[80] L. Kish. *Survey Sampling*. John Wiley & Sons, New York, 1965.

[81] J. Ladyman and D. Ross, with D. Spurrett and J. Collier. *Every Thing Must Go: Metaphysics Naturalized*. Oxford University Press, Oxford, 2007.

[82] ISO/IEC/IEEE. *Systems and software engineering — Systems and software assurance — Part 2: Assurance case*, ISO/IEC/IEEE 15026-2:2022. ISO, Geneva, 2022. https://www.iso.org/standard/80625.html

[83] OpenSSF. *SLSA: Supply-chain Levels for Software Artifacts*, specification v1.2, approved. Open Source Security Foundation, 2025. https://slsa.dev/spec/v1.2/

[84] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *28th USENIX Security Symposium*, pages 1393–1410, 2019. https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias

[85] L. Cucu-Grosjean. Invited paper: Statistical, stochastic or probabilistic (worst-case execution) execution time? What impact on the multicore composability. In *22nd International Workshop on Worst-Case Execution Time Analysis (WCET 2024)*, volume 121 of *OASIcs*, pages 6:1–6:10, 2024. https://doi.org/10.4230/OASIcs.WCET.2024.6